

Verification Continuum™

Verdi® Python-Based NPI Language Model

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	21
Synopsys Statement on Inclusivity and Diversity	22

1. Introduction to Python Based NPI	23
Packages and Modules	23
Packages	23
Modules	23
Module Functions and Class Objects	24
L0 Module Functions	24
L1 Module Functions	24
Class Objects	24
User Interface and Use Flow	24
Environment and Library Setting	24

2. Module npisys	27
Overview	27
L0 APIs	27
npisys.init(<i>pyArgvList</i>)	27
npisys.load_design(<i>pyArgvList</i>)	27
npisys.end()	28

3. Python-Based NPI Language Model	29
Overview	29
Quick Start	29
Demo Case	30

4. Class Objects	33
Supported Properties and Methods	33
Mixed Design Objects	34
npiterator	34

Contents

<i>class lang.IteratorHandle(cps_obj)</i>	34
<i>npilangInterface</i>	36
<i>class lang.LangInterfaceHandle(cps_obj)</i>	36
<i>npiport</i>	39
<i>class lang.PortHandle(cps_obj)</i>	39
SystemVerilog Objects	45
<i>npirealvar</i>	48
<i>class lang.RealVarHandle(cps_obj)</i>	48
<i>npireturnstmt</i>	55
<i>class lang.ReturnStmtHandle(cps_obj)</i>	55
<i>npimodulearray</i>	58
<i>class lang.ModuleArrayHandle(cps_obj)</i>	58
<i>npimethodtaskcall</i>	62
<i>class lang.MethodTaskCallHandle(cps_obj)</i>	62
<i>npilmmmediateassert</i>	67
<i>class lang.ImmediateAssertHandle(cps_obj)</i>	67
<i>npitypepattern</i>	72
<i>class lang.TypePatternHandle(cps_obj)</i>	72
<i>npipropformaldecl</i>	75
<i>class lang.PropFormalDeclHandle(cps_obj)</i>	75
<i>npiclassobj</i>	79
<i>class lang.ClassObjHandle(cps_obj)</i>	79
<i>npitaskcall</i>	85
<i>class lang.TaskCallHandle(cps_obj)</i>	85
<i>npiclasstypespec</i>	90
<i>class lang.ClassTypespecHandle(cps_obj)</i>	90
<i>npilntvar</i>	97
<i>class lang.IntVarHandle(cps_obj)</i>	97
<i>npidefaultpattern</i>	104
<i>class lang.DefaultPatternHandle(cps_obj)</i>	104
<i>npinamedbegin</i>	107
<i>class lang.NamedBeginHandle(cps_obj)</i>	107
<i>npifunction</i>	114
<i>class lang.FunctionHandle(cps_obj)</i>	114
<i>npilnttypespec</i>	121
<i>class lang.IntTypespecHandle(cps_obj)</i>	121
<i>npifork</i>	125
<i>class lang.ForkHandle(cps_obj)</i>	125
<i>npitimespec</i>	131
<i>class lang.TimeTypespecHandle(cps_obj)</i>	131

Contents

<code>npiclassaccessobj</code>	135
<code>class lang.ClassAccessObjHandle(<i>cps_obj</i>)</code>	135
<code>npienumconst</code>	139
<code>class lang.EnumConstHandle(<i>cps_obj</i>)</code>	139
<code>npistructnet</code>	143
<code>class lang.StructNetHandle(<i>cps_obj</i>)</code>	143
<code>npitimenet</code>	150
<code>class lang.TimeNetHandle(<i>cps_obj</i>)</code>	150
<code>npistructtypespec</code>	157
<code>class lang.StructTypespecHandle(<i>cps_obj</i>)</code>	157
<code>npisArraytypespec</code>	161
<code>class lang.ArrayTypespecHandle(<i>cps_obj</i>)</code>	161
<code>npideassign</code>	166
<code>class lang.DeassignHandle(<i>cps_obj</i>)</code>	166
<code>npilintegertypespec</code>	169
<code>class lang.IntegerTypespecHandle(<i>cps_obj</i>)</code>	169
<code>npinamedfork</code>	173
<code>class lang.NamedForkHandle(<i>cps_obj</i>)</code>	173
<code>npienumtypespec</code>	179
<code>class lang.EnumTypespecHandle(<i>cps_obj</i>)</code>	179
<code>npistringvar</code>	184
<code>class lang.StringVarHandle(<i>cps_obj</i>)</code>	184
<code>npipackedarraytypespec</code>	191
<code>class lang.PackedArrayTypespecHandle(<i>cps_obj</i>)</code>	191
<code>npigatearray</code>	195
<code>class lang.GateArrayHandle(<i>cps_obj</i>)</code>	195
<code>npiconstrlfeelse</code>	200
<code>class lang.ConstrIfElseHandle(<i>cps_obj</i>)</code>	200
<code>npiconstrforeach</code>	203
<code>class lang.ConstrForeachHandle(<i>cps_obj</i>)</code>	203
<code>npichandlevar</code>	206
<code>class lang.ChandleVarHandle(<i>cps_obj</i>)</code>	206
<code>npieportitem</code>	213
<code>class lang.ExportItemHandle(<i>cps_obj</i>)</code>	213
<code>npiextref</code>	216
<code>class lang.ExtRefHandle(<i>cps_obj</i>)</code>	216
<code>npilintegernet</code>	220
<code>class lang.IntegerNetHandle(<i>cps_obj</i>)</code>	220
<code>npishortintvar</code>	226
<code>class lang.ShortIntVarHandle(<i>cps_obj</i>)</code>	226

Contents

npiUdp	234
class lang.UdpHandle(<i>cps_obj</i>)	234
npiSysTaskCall	240
class lang.SysTaskCallHandle(<i>cps_obj</i>)	240
npiAssignment	244
class lang.AssignmentHandle(<i>cps_obj</i>)	244
npiClockingBlock	248
class lang.ClockingBlockHandle(<i>cps_obj</i>)	248
npiRestrict	256
class lang.RestrictHandle(<i>cps_obj</i>)	256
npiPackage	260
class lang.PackageHandle(<i>cps_obj</i>)	260
npiIndexedPartSelect	272
class lang.IndexedPartSelectHandle(<i>cps_obj</i>)	272
npiNetSelect	277
class lang.NetSelectHandle(<i>cps_obj</i>)	277
npiRepeat	282
class lang.RepeatHandle(<i>cps_obj</i>)	282
npiInterface	285
class lang.InterfaceHandle(<i>cps_obj</i>)	285
npiVirtualInterfaceVar	299
class lang.VirtualInterfaceVarHandle(<i>cps_obj</i>)	299
npiPackedArrayVar	305
class lang.PackedArrayVarHandle(<i>cps_obj</i>)	305
npiPropertyInst	312
class lang.PropertyInstHandle(<i>cps_obj</i>)	312
npiBitVar	316
class lang.BitVarHandle(<i>cps_obj</i>)	316
npiRegBit	323
class lang.RegBitHandle(<i>cps_obj</i>)	323
npiTableEntry	331
class lang.TableEntryHandle(<i>cps_obj</i>)	331
npiConstraintOrdering	335
class lang.ConstraintOrderingHandle(<i>cps_obj</i>)	335
npiIfElse	338
class lang.IfElseHandle(<i>cps_obj</i>)	338
npiParameterSelect	342
class lang.ParameterSelectHandle(<i>cps_obj</i>)	342
npiPropertySpec	347
class lang.PropertySpecHandle(<i>cps_obj</i>)	347

Contents

npiParamater	356
class lang.ParameterHandle(<i>cps_obj</i>)	356
npIDistItem	363
class lang.DistItemHandle(<i>cps_obj</i>)	363
npIPrimTerm	367
class lang.PrimTermHandle(<i>cps_obj</i>)	367
npIAlways	370
class lang.AlwaysHandle(<i>cps_obj</i>)	370
npIArrayParameter	379
class lang.ArrayParameterHandle(<i>cps_obj</i>)	379
npIImmediateCover	381
class lang.ImmediateCoverHandle(<i>cps_obj</i>)	381
npIDisable	386
class lang.DisableHandle(<i>cps_obj</i>)	386
npIUnionTypespec	389
class lang.UnionTypespecHandle(<i>cps_obj</i>)	389
npIInterfaceArray	393
class lang.InterfaceArrayHandle(<i>cps_obj</i>)	393
npIBreak	398
class lang.BreakHandle(<i>cps_obj</i>)	398
npIGenScope	401
class lang.GenScopeHandle(<i>cps_obj</i>)	401
npIShortRealVar	409
class lang.ShortRealVarHandle(<i>cps_obj</i>)	409
npISysFuncCall	417
class lang.SysFuncCallHandle(<i>cps_obj</i>)	417
npINettypeTypespec	423
class lang.NettypeTypespecHandle(<i>cps_obj</i>)	423
npIFuncCall	427
class lang.FuncCallHandle(<i>cps_obj</i>)	427
npIByteTypespec	433
class lang.ByteTypespecHandle(<i>cps_obj</i>)	433
npIDoWhile	437
class lang.DoWhileHandle(<i>cps_obj</i>)	437
npIRepeatControl	440
class lang.RepeatControlHandle(<i>cps_obj</i>)	440
npINullOrStmt	444
class lang.NullStmtHandle(<i>cps_obj</i>)	444
npIImmediateAssume	447
class lang.ImmediateAssumeHandle(<i>cps_obj</i>)	447

Contents

npiSequenceInst	451
class lang.SequenceInstHandle(<i>cps_obj</i>)	451
npiNet	455
class lang.NetHandle(<i>cps_obj</i>)	455
npiCase	461
class lang.CaseHandle(<i>cps_obj</i>)	461
npiUdpDefn	465
class lang.UdpDefnHandle(<i>cps_obj</i>)	465
npiCaseItem	469
class lang.CaseItemHandle(<i>cps_obj</i>)	469
npiNetBit	472
class lang.NetBitHandle(<i>cps_obj</i>)	472
npiConstant	479
class lang.ConstantHandle(<i>cps_obj</i>)	479
npiIODecl	483
class lang.IODeclHandle(<i>cps_obj</i>)	483
npiBitSelect	487
class lang.BitSelectHandle(<i>cps_obj</i>)	487
npiImplication	491
class lang.ImplicationHandle(<i>cps_obj</i>)	491
npiStructVar	494
class lang.StructVarHandle(<i>cps_obj</i>)	494
npiStringTypespec	502
class lang.StringTypespecHandle(<i>cps_obj</i>)	502
npiRelease	506
class lang.ReleaseHandle(<i>cps_obj</i>)	506
npiShortRealTypespec	509
class lang.ShortRealTypespecHandle(<i>cps_obj</i>)	509
npiFor	512
class lang.ForHandle(<i>cps_obj</i>)	512
npiMpPortBit	520
class lang.MpPortBitHandle(<i>cps_obj</i>)	520
npiAliasStmt	523
class lang.AliasStmtHandle(<i>cps_obj</i>)	523
npiProgram	526
class lang.ProgramHandle(<i>cps_obj</i>)	526
npiForever	540
class lang.ForeverHandle(<i>cps_obj</i>)	540
npiMpPort	544
class lang.MpPortHandle(<i>cps_obj</i>)	544

Contents

<code>npIAssume</code>	548
<code>class lang.AssumeHandle(cps_obj)</code>	548
<code>npIPropertyDecl</code>	552
<code>class lang.PropertyDeclHandle(cps_obj)</code>	552
<code>npIByteVar</code>	559
<code>class lang.ByteVarHandle(cps_obj)</code>	559
<code>npIWait</code>	566
<code>class lang.WaitHandle(cps_obj)</code>	566
<code>npIRange</code>	570
<code>class lang.RangeHandle(cps_obj)</code>	570
<code>npIClassDefn</code>	572
<code>class lang.ClassDefnHandle(cps_obj)</code>	572
<code>npIForeachStmt</code>	581
<code>class lang.ForeachStmtHandle(cps_obj)</code>	581
<code>npIProgramArray</code>	588
<code>class lang.ProgramArrayHandle(cps_obj)</code>	588
<code>npIAssert</code>	592
<code>class lang.AssertHandle(cps_obj)</code>	592
<code>npIAssignStmt</code>	596
<code>class lang.AssignStmtHandle(cps_obj)</code>	596
<code>npIInterfaceTypespec</code>	601
<code>class lang.InterfaceTypespecHandle(cps_obj)</code>	601
<code>npITypeParameter</code>	603
<code>class lang.TypeParameterHandle(cps_obj)</code>	603
<code>npIArrayVar</code>	608
<code>class lang.ArrayVarHandle(cps_obj)</code>	608
<code>npIEnumVar</code>	616
<code>class lang.EnumVarHandle(cps_obj)</code>	616
<code>npIContinue</code>	623
<code>class lang.ContinueHandle(cps_obj)</code>	623
<code>npISwitchArray</code>	626
<code>class lang.SwitchArrayHandle(cps_obj)</code>	626
<code>npIArrayNet</code>	630
<code>class lang.ArrayNetHandle(cps_obj)</code>	630
<code>npIEnumNet</code>	637
<code>class lang.EnumNetHandle(cps_obj)</code>	637
<code>npIShortIntTypespec</code>	645
<code>class lang.ShortIntTypespecHandle(cps_obj)</code>	645
<code>npIUnionVar</code>	649
<code>class lang.UnionVarHandle(cps_obj)</code>	649

Contents

npibegin	656
class lang.BeginHandle(<i>cps_obj</i>)	656
npigenvar	666
class lang.GenVarHandle(<i>cps_obj</i>)	666
npimodule	671
class lang.ModuleHandle(<i>cps_obj</i>)	671
npilf	682
class lang.IfHandle(<i>cps_obj</i>)	682
npitudparray	686
class lang.UdpArrayHandle(<i>cps_obj</i>)	686
npidelaycontrol	690
class lang.DelayControlHandle(<i>cps_obj</i>)	690
npifinal	694
class lang.FinalHandle(<i>cps_obj</i>)	694
npiclassvar	697
class lang.ClassVarHandle(<i>cps_obj</i>)	697
npioperation	702
class lang.OperationHandle(<i>cps_obj</i>)	702
npigate	708
class lang.GateHandle(<i>cps_obj</i>)	708
npinamedeventarray	713
class lang.NamedEventArrayHandle(<i>cps_obj</i>)	713
npilintegervar	717
class lang.IntegerVarHandle(<i>cps_obj</i>)	717
npidistribution	725
class lang.DistributionHandle(<i>cps_obj</i>)	725
npilinitial	728
class lang.InitialHandle(<i>cps_obj</i>)	728
npiparamassign	731
class lang.ParamAssignHandle(<i>cps_obj</i>)	731
npilongintvar	734
class lang.LongIntVarHandle(<i>cps_obj</i>)	734
npitypespecmember	741
class lang.TypespecMemberHandle(<i>cps_obj</i>)	741
npiparameterbit	745
class lang.ParameterBitHandle(<i>cps_obj</i>)	745
npinamedevent	753
class lang.NamedEventHandle(<i>cps_obj</i>)	753
npiconstraint	757
class lang.ConstraintHandle(<i>cps_obj</i>)	757

Contents

<code>npISwitch</code>	762
<code>class lang.SwitchHandle(cps_obj)</code>	762
<code>npIMemberPattern</code>	768
<code>class lang.MemberPatternHandle(cps_obj)</code>	768
<code>npIForce</code>	771
<code>class lang.ForceHandle(cps_obj)</code>	771
<code>npIRefObj</code>	774
<code>class lang.RefObjHandle(cps_obj)</code>	774
<code>npIReg</code>	779
<code>class lang.RegHandle(cps_obj)</code>	779
<code>npIWhile</code>	787
<code>class lang.WhileHandle(cps_obj)</code>	787
<code>npISequenceDecl</code>	790
<code>class lang.SequenceDeclHandle(cps_obj)</code>	790
<code>npIConstrIf</code>	797
<code>class lang.ConstrIfHandle(cps_obj)</code>	797
<code>npILongIntTypespec</code>	800
<code>class lang.LongIntTypespecHandle(cps_obj)</code>	800
<code>npIImportItem</code>	804
<code>class lang.ImportItemHandle(cps_obj)</code>	804
<code>npIVarSelect</code>	807
<code>class lang.VarSelectHandle(cps_obj)</code>	807
<code>npIModport</code>	812
<code>class lang.ModportHandle(cps_obj)</code>	812
<code>npITimeVar</code>	816
<code>class lang.TimeVarHandle(cps_obj)</code>	816
<code>npILogicTypespec</code>	823
<code>class lang.LogicTypespecHandle(cps_obj)</code>	823
<code>npIPackedArrayParameter</code>	828
<code>class lang.PackedArrayParameterHandle(cps_obj)</code>	828
<code>npISeqFormalDecl</code>	834
<code>class lang.SeqFormalDeclHandle(cps_obj)</code>	834
<code>npIEventControl</code>	838
<code>class lang.EventControlHandle(cps_obj)</code>	838
<code>npIEventStmt</code>	841
<code>class lang.EventStmtHandle(cps_obj)</code>	841
<code>npIPartSelect</code>	844
<code>class lang.PartSelectHandle(cps_obj)</code>	844
<code>npIClockingIODecl</code>	848
<code>class lang.ClockingIODeclHandle(cps_obj)</code>	848

Contents

npiChandleTypespec	853
class lang.ChandleTypespecHandle(<i>cps_obj</i>)	853
npiMethodFuncCall	857
class lang.MethodFuncCallHandle(<i>cps_obj</i>)	857
npiPositionPattern	863
class lang.PositionPatternHandle(<i>cps_obj</i>)	863
npiTask	866
class lang.TaskHandle(<i>cps_obj</i>)	866
npiBitTypespec	873
class lang.BitTypespecHandle(<i>cps_obj</i>)	873
npiPackedArrayNet	877
class lang.PackedArrayNetHandle(<i>cps_obj</i>)	877
npiRealTypespec	884
class lang.RealTypespecHandle(<i>cps_obj</i>)	884
npiCover	888
class lang.CoverHandle(<i>cps_obj</i>)	888
npiDefParam	893
class lang.DefParamHandle(<i>cps_obj</i>)	893
npiContAssign	896
class lang.ContAssignHandle(<i>cps_obj</i>)	896
npiCovergroup	899
class lang.CovergroupHandle(<i>cps_obj</i>)	899
npiCoverageOption	904
class lang.CoverageOptionHandle(<i>cps_obj</i>)	904
npiCoverPoint	907
class lang.CoverPointHandle(<i>cps_obj</i>)	907
npiCoverCross	911
class lang.CoverCrossHandle(<i>cps_obj</i>)	911
npiBins	915
class lang.BinsHandle(<i>cps_obj</i>)	915
VHDL Objects	918
npiVhConst	920
class lang.VhConstHandle(<i>cps_obj</i>)	920
npiVhProcedure	926
class lang.VhProcedureHandle(<i>cps_obj</i>)	926
npiVhOperation	931
class lang.VhOperationHandle(<i>cps_obj</i>)	931
npiVhGeneric	935
class lang.VhGenericHandle(<i>cps_obj</i>)	935
npiVhSimpAttrName	941

Contents

<i>class lang.VhSimpAttrNameHandle(cps_obj)</i>	941
<i>npivhRoot</i>	944
<i>class lang.VhRootHandle(cps_obj)</i>	944
<i>npivhMapElem</i>	953
<i>class lang.VhMapElemHandle(cps_obj)</i>	953
<i>npivhPackDecl</i>	956
<i>class lang.VhPackDeclHandle(cps_obj)</i>	956
<i>npivhForeverLoop</i>	961
<i>class lang.VhForeverLoopHandle(cps_obj)</i>	961
<i>npivhEnumLiteral</i>	964
<i>class lang.VhEnumLiteralHandle(cps_obj)</i>	964
<i>npivhNullLiteral</i>	968
<i>class lang.VhNullLiteralHandle(cps_obj)</i>	968
<i>npivhBitStrLiteral</i>	971
<i>class lang.VhBitStrLiteralHandle(cps_obj)</i>	971
<i>npivhUserAttrName</i>	975
<i>class lang.VhUserAttrNameHandle(cps_obj)</i>	975
<i>npivhReturn</i>	978
<i>class lang.VhReturnHandle(cps_obj)</i>	978
<i>npivhRange</i>	981
<i>class lang.VhRangeHandle(cps_obj)</i>	981
<i>npivhPackage</i>	984
<i>class lang.VhPackageHandle(cps_obj)</i>	984
<i>npivhProcDecl</i>	991
<i>class lang.VhProcDeclHandle(cps_obj)</i>	991
<i>npivhWait</i>	994
<i>class lang.VhWaitHandle(cps_obj)</i>	994
<i>npivhConcAssert</i>	998
<i>class lang.VhConcAssertHandle(cps_obj)</i>	998
<i>npivhTypeConv</i>	1002
<i>class lang.VhTypeConvHandle(cps_obj)</i>	1002
<i>npivhPhysLiteral</i>	1006
<i>class lang.VhPhysLiteralHandle(cps_obj)</i>	1006
<i>npivhParamAttrName</i>	1010
<i>class lang.VhParamAttrNameHandle(cps_obj)</i>	1010
<i>npivhForLoop</i>	1014
<i>class lang.VhForLoopHandle(cps_obj)</i>	1014
<i>npivhCase</i>	1018
<i>class lang.VhCaseHandle(cps_obj)</i>	1018
<i>npivhSig</i>	1022

Contents

<i>class lang.VhSigHandle(cps_obj)</i>	1022
<i>npivhSimpleSigAssign</i>	1027
<i>class lang.VhSimpleSigAssignHandle(cps_obj)</i>	1027
<i>npivhProcess</i>	1031
<i>class lang.VhProcessHandle(cps_obj)</i>	1031
<i>npivhIndexVar</i>	1035
<i>class lang.VhIndexVarHandle(cps_obj)</i>	1035
<i>npivhWhileLoop</i>	1039
<i>class lang.VhWhileLoopHandle(cps_obj)</i>	1039
<i>npivhSelectSigAssign</i>	1043
<i>class lang.VhSelectSigAssignHandle(cps_obj)</i>	1043
<i>npivhIf</i>	1047
<i>class lang.VhIfHandle(cps_obj)</i>	1047
<i>npivhIntTypeDecl</i>	1051
<i>class lang.VhIntTypeDeclHandle(cps_obj)</i>	1051
<i>npivhEntity</i>	1054
<i>class lang.VhEntityHandle(cps_obj)</i>	1054
<i>npivhCompDecl</i>	1059
<i>class lang.VhCompDeclHandle(cps_obj)</i>	1059
<i>npivhSliceName</i>	1062
<i>class lang.VhSliceNameHandle(cps_obj)</i>	1062
<i>npivhConstParam</i>	1067
<i>class lang.VhConstParamHandle(cps_obj)</i>	1067
<i>npivhSelectWaveform</i>	1071
<i>class lang.VhSelectWaveformHandle(cps_obj)</i>	1071
<i>npivhIntLiteral</i>	1074
<i>class lang.VhIntLiteralHandle(cps_obj)</i>	1074
<i>npivhCondWaveform</i>	1077
<i>class lang.VhCondWaveformHandle(cps_obj)</i>	1077
<i>npivhLibraryDecl</i>	1079
<i>class lang.VhLibraryDeclHandle(cps_obj)</i>	1079
<i>npivhRecordTypeDecl</i>	1081
<i>class lang.VhRecordTypeDeclHandle(cps_obj)</i>	1081
<i>npivhAttrSpec</i>	1084
<i>class lang.VhAttrSpecHandle(cps_obj)</i>	1084
<i>npivhConcProcCall</i>	1088
<i>class lang.VhConcProcCallHandle(cps_obj)</i>	1088
<i>npivhBlock</i>	1092
<i>class lang.VhBlockHandle(cps_obj)</i>	1092
<i>npivhIndexedName</i>	1098

Contents

<i>class lang.VhIndexedNameHandle(cps_obj)</i>	1098
<i>npivhSeqSigAssign</i>	1103
<i>class lang.VhSeqSigAssignHandle(cps_obj)</i>	1103
<i>npivhFuncCall</i>	1107
<i>class lang.VhFuncCallHandle(cps_obj)</i>	1107
<i>npivhBranch</i>	1112
<i>class lang.VhBranchHandle(cps_obj)</i>	1112
<i>npivhNull</i>	1115
<i>class lang.VhNullHandle(cps_obj)</i>	1115
<i>npivhArrayTypeDecl</i>	1119
<i>class lang.VhArrayTypeDeclHandle(cps_obj)</i>	1119
<i>npivhFloatTypeDecl</i>	1122
<i>class lang.VhFloatTypeDeclHandle(cps_obj)</i>	1122
<i>npivhRealLiteral</i>	1125
<i>class lang.VhRealLiteralHandle(cps_obj)</i>	1125
<i>npivhSigParam</i>	1129
<i>class lang.VhSigParamHandle(cps_obj)</i>	1129
<i>npivhElemDecl</i>	1133
<i>class lang.VhElemDeclHandle(cps_obj)</i>	1133
<i>npivhCharLiteral</i>	1136
<i>class lang.VhCharLiteralHandle(cps_obj)</i>	1136
<i>npivhSeqProcCall</i>	1140
<i>class lang.VhSeqProcCallHandle(cps_obj)</i>	1140
<i>npivhStringLiteral</i>	1145
<i>class lang.VhStringLiteralHandle(cps_obj)</i>	1145
<i>npivhAttrDecl</i>	1148
<i>class lang.VhAttrDeclHandle(cps_obj)</i>	1148
<i>npivhAggregate</i>	1151
<i>class lang.VhAggregateHandle(cps_obj)</i>	1151
<i>npivhSeqAssert</i>	1154
<i>class lang.VhSeqAssertHandle(cps_obj)</i>	1154
<i>npivhEnumTypeDecl</i>	1158
<i>class lang.VhEnumTypeDeclHandle(cps_obj)</i>	1158
<i>npivhArchBody</i>	1161
<i>class lang.VhArchBodyHandle(cps_obj)</i>	1161
<i>npivhGenerate</i>	1165
<i>class lang.VhGenerateHandle(cps_obj)</i>	1165
<i>npivhWaveformElem</i>	1169
<i>class lang.VhWaveformElemHandle(cps_obj)</i>	1169
<i>npivhVarParam</i>	1172

Contents

class lang.VhVarParamHandle(<i>cps_obj</i>)	1172
npivhVarAssign	1176
class lang.VhVarAssignHandle(<i>cps_obj</i>)	1176
npivhQualifiedExpr	1180
class lang.VhQualifiedExprHandle(<i>cps_obj</i>)	1180
npivhPhysUnit	1184
class lang.VhPhysUnitHandle(<i>cps_obj</i>)	1184
npivhElemAssoc	1187
class lang.VhElemAssocHandle(<i>cps_obj</i>)	1187
npivhComponent	1190
class lang.VhComponentHandle(<i>cps_obj</i>)	1190
npivhVar	1199
class lang.VhVarHandle(<i>cps_obj</i>)	1199
npivhExit	1204
class lang.VhExitHandle(<i>cps_obj</i>)	1204
npivhFuncDecl	1208
class lang.VhFuncDeclHandle(<i>cps_obj</i>)	1208
npivhCondSigAssign	1212
class lang.VhCondSigAssignHandle(<i>cps_obj</i>)	1212
npivhPhysTypeDecl	1216
class lang.VhPhysTypeDeclHandle(<i>cps_obj</i>)	1216
npivhUseClause	1219
class lang.VhUseClauseHandle(<i>cps_obj</i>)	1219
npivhReport	1222
class lang.VhReportHandle(<i>cps_obj</i>)	1222
npivhSubTypeDecl	1225
class lang.VhSubTypeDeclHandle(<i>cps_obj</i>)	1225
npivhOthers	1230
class lang.VhOthersHandle(<i>cps_obj</i>)	1230
npivhNext	1232
class lang.VhNextHandle(<i>cps_obj</i>)	1232
npivhPackBody	1236
class lang.VhPackBodyHandle(<i>cps_obj</i>)	1236
npivhFunction	1239
class lang.VhFunctionHandle(<i>cps_obj</i>)	1239
SystemC Objects	1244
npiscMethodProcess	1245
class lang.ScMethodProcessHandle(<i>cps_obj</i>)	1245
npiscInstance	1249
class lang.ScInstanceHandle(<i>cps_obj</i>)	1249

5. L0 APIs	1254
handle_by_name	1254
<i>lang.handle_by_name(name, hdl)</i>	1254
handle_by_index	1256
<i>lang.handle_by_index(hdl, index)</i>	1256
handle_by_range	1258
<i>lang.handle_by_range(hdl, left, right)</i>	1258
get_top_inst_list	1259
<i>lang.get_top_inst_list()</i>	1259
release_handle	1261
<i>lang.release_handle(hdl)</i>	1261
release_all_handles	1263
<i>lang.release_all_handles()</i>	1263
compare_objects	1264
<i>lang.compare_objects(hdl1, hdl2)</i>	1264
objects_overlap	1268
<i>lang.objects_overlap(hdl1, hdl2)</i>	1268

6. L1 APIs	1272
Hierarchy Tree	1272
hier_tree_dump_txt	1272
<i>lang.hier_tree_dump_txt(root_scope, file)</i>	1272
hier_tree_dump_csv	1274
<i>lang.hier_tree_dump_csv(root_scope, file, is_skip_lib_cell=False)</i>	1274
hier_tree_trv_inst	1276
<i>lang.hier_tree_trv_inst(root_scope)</i>	1276
hier_tree_trv_begin	1278
<i>lang.hier_tree_trv_begin(root_scope)</i>	1278
hier_tree_trv_register_cb	1280
<i>lang.hier_tree_trv_register_cb(obj_type, cb_func, cb_data)</i>	1280
hier_tree_trv_reset_cb	1283
<i>lang.hier_tree_trv_reset_cb()</i>	1283
hier_tree_trv	1286
<i>lang.hier_tree_trv(scope_hier_name=None, depth=0)</i>	1286
Module	1288
mod_define_get_inst	1288

Contents

<i>lang.mod_define_get_inst(mod_name)</i>	1288
Component	1289
arch_body_get_inst	1289
<i>lang.arch_body_get_inst(arch_body_name)</i>	1289
entity_get_inst	1291
<i>lang.entity_get_inst(entity_name)</i>	1291
Signal	1293
trace_driver2	1294
<i>lang.trace_driver2(sig_hier_name, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1294
trace_driver_by_hdl2	1297
<i>lang.trace_driver_by_hdl2(sig_hdl, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1297
trace_driver_dump2	1299
<i>lang.trace_driver_dump2(sig_hier_name, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1299
trace_driver_by_hdl_dump2	1301
<i>lang.trace_driver_by_hdl_dump2(sig_hdl, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1301
trace_load2	1304
<i>lang.trace_load2(sig_hier_name, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1304
trace_load_by_hdl2	1306
<i>lang.trace_load_by_hdl2(sig_hdl, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1306
trace_load_dump2	1309
<i>lang.trace_load_dump2(sig_hier_name, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1309
trace_load_by_hdl_dump2	1311
<i>lang.trace_load_by_hdl_dump2(sig_hdl, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)</i>	1311
get_signal_define_typespec_by_hdl	1313
<i>lang.get_signal_define_typespec_by_hdl(sig_hdl)</i>	1313
get_signal_define_typespec	1314
<i>lang.get_signal_define_typespec(sig_hier_name)</i>	1314
dump_signal_define_typespec	1315
<i>lang.dump_signal_define_typespec(sig_hier_name, file)</i>	1315
dump_signal_define_typespec_by_hdl	1316
<i>lang.dump_signal_define_typespec_by_hdl(sig_hdl, file)</i>	1316
get_bit_blasted_signal	1317
<i>lang.get_bit_blasted_signal(sig_hdl)</i>	1317

Contents

List	1319
list_all_module_define	1319
lang.list_all_module_define(<i>root_scope</i> , <i>file</i>)	1319
Find	1320
find_inst_wildcard	1321
lang.find_inst_wildcard(<i>root_scope</i> , <i>inst_wildcard_list</i>)	1321
find_inst_wildcard_dump	1323
lang.find_inst_wildcard_dump(<i>root_scope</i> , <i>inst_wildcard_list</i> , <i>file</i>)	1323
find_inst_regex	1326
lang.find_inst_regex(<i>root_scope</i> , <i>inst_regex_list</i>)	1326
find_inst_regex_dump	1328
lang.find_inst_regex_dump(<i>root_scope</i> , <i>inst_regex_list</i> , <i>file</i>)	1328
find_inst_with_def_wildcard	1330
lang.find_inst_with_def_wildcard(<i>root_scope</i> , <i>inst_def_wildcard_list</i>)	1330
find_inst_with_def_wildcard_dump	1332
lang.find_inst_with_def_wildcard_dump(<i>root_scope</i> , <i>inst_def_wildcard_list</i> , <i>file</i>)	1332
find_inst_with_def_regex	1334
lang.find_inst_with_def_regex(<i>root_scope</i> , <i>inst_def_regex_list</i>)	1334
find_inst_with_def_regex_dump	1337
lang.find_inst_with_def_regex_dump(<i>root_scope</i> , <i>inst_def_regex_list</i> , <i>file</i>)	1337
find_signal_wildcard	1339
lang.find_signal_wildcard(<i>root_scope</i> , <i>sig_wildcard_list</i>)	1339
find_signal_wildcard_dump	1341
lang.find_signal_wildcard_dump(<i>root_scope</i> , <i>sig_wildcard_list</i> , <i>file</i>)	1341
find_signal_regex	1343
lang.find_signal_regex(<i>root_scope</i> , <i>sig_regex_list</i>)	1343
find_signal_regex_dump	1346
lang.find_signal_regex_dump(<i>root_scope</i> , <i>sig_regex_list</i> , <i>file</i>)	1346
Utilities	1348
dump_hdl_info	1348
lang.dump_hdl_info(<i>hdl</i> , <i>file</i> , <i>is_compcose_full_name=False</i> , <i>const_size=False</i>)	1348
get_hdl_info	1350
lang.get_hdl_info(<i>hdl</i> , <i>is_compose_full_name=False</i> , <i>const_size=False</i>)	1350
dump_hdl_vec_info	1351
lang.dump_hdl_vec_info(<i>hdl_list</i> , <i>file</i> , <i>is_release=False</i> , <i>is_compose_full_name=False</i> , <i>const_size=False</i>)	1351

Contents

expr_decompile	1353
<i>lang.expr_decompile(expr_hdl, const_size=False,</i> <i>res_const_opr=False)</i>	1353
expr_trv_register_cb	1354
<i>lang.expr_trv_register_cb(cb_func, cb_data)</i>	1354
expr_trv_reset_cb	1356
<i>lang.expr_trv_reset_cb()</i>	1356
expr_trv	1358
<i>lang.expr_trv(expr_hdl)</i>	1358
scope_explore_register_cb	1359
<i>lang.scope_explore_register_cb(obj_type, cb_func, cb_data)</i>	1359
scope_explore_reset_cb	1364
<i>lang.scope_explore_reset_cb()</i>	1364
scope_explore	1367
<i>lang.scope_explore(scope_hier_name=None, depth=0)</i>	1367
A. Appendix	1371
Data Structure	1371
TrcOption	1371
<i>class lang.TrcOption(edge_check=True, trc_val_idx=False,</i> <i>report_data=True, report_control=True, trc_stmt_prop=False)</i>	1371
DrvLoadStmt	1374
<i>class lang.DrvLoadStmt(src_hdl, scope_hdl, is_pass_thr, num_sig_use,</i> <i>use_hdl, sig_hdl_list)</i>	1374

Preface

The Python Based NPI Language Model User Guide helps you to traverse designs from the HDL language perspective.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

- You can call for support at:

<https://www.synopsys.com/support/global-support-centers.html>

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Python Based NPI

Python-Based NPI APIs support six models. They are as follows:

- Language
- Netlist
- Waveform
- Text
- Coverage
- Waveform Writer

Each model have their own APIs to let you be able to traverse data objects and obtain objects' properties like the existing C-Based or Tcl-Based NPI APIs.

In this guide, the environment setting for using **Python-Based NPI APIs for Language** is demonstrated.

Packages and Modules

Packages

The Python-based NPI package name is “pynpi”, and it is placed at `$VERDI_HOME/share/NPI/python`.

Modules

There are seven modules inside the “pynpi” package: npisys, lang, netlist, text, cov waveform and waveformw. The first module, npisys, is the system model for initialization, loading design and exit. The other modules represent language model, netlist model, text model, coverage model, wave model and waveform writer model respectively

Module Functions and Class Objects

L0 Module Functions

Every module provides some L0 (level 0) functions to let you get the class objects. These functions return a class object or a list of class objects, and they follow the specification of the existing L0 APIs provided in C or Tcl.

L1 Module Functions

Similar to L0 module functions, every module also provides some L1 (level 1) functions to let you get advanced information based on the results obtained by L0 module functions. These functions follow the specification of the existing L1 APIs provided in C or Tcl.

Class Objects

The class object is similar to the so-called handle in NPI C APIs. The most difference is that some basic L0 APIs in C and Tcl will become class method function. These L0 APIs are usually to get integer value, string value, 1-to-1 method to get a handle, and 1-to-many method to get handle iterator.

User Interface and Use Flow

This chapter describes the user interface and use flow for Python-Based NPI APIs.

Environment and Library Setting

The python library setting flow of using Python-Based NPI APIs contains four parts:

1. Check your Python's version:

Python-Based NPI APIs need the Python version greater than 3.6.0.

2. Environment setting for "VERDI_HOME" is required for Python-based NPI. Ensure that you set it up before running program.
3. Add python library path into your python code before loading Python-Based NPI by the following commands:

```
rel_lib_path = os.environ['VERDI_HOME'] + '/share/NPI/python'  
sys.path.append(os.path.abspath(rel_lib_path))
```

4. Import module “npisys” for using the function of NPI initialization and exit from pynpi package.

```
from pynpi import npisys
```

5. Import the module you need from pynpi package. For example, if you want to use language model, you can import the module as follows:

```
from pynpi import lang
```

6. Note that initialization function `npisys.init()` must be called before writing your code by using any other modules. Besides, `npisys.end()` should be called after finishing your code. Following is a simple example to demonstrate how to use language model by Python-Based NPI APIs.

Python program to use NPI language model: (demo.py)

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(rel_lib_path))
from pynpi import npisys
from pynpi import lang

# Initialize NPI
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI")
    assert 0

# Beginning of your code here -----
#
# Example code can be found in later chapters
#
# End of your code -----

# End NPI
npisys.end()
```

C shell script to setup environment and execute Python program on 64-bit machine:
(run_demo)

```
#!/bin/csh -f
# Setup your $VERDI_HOME here
setenv VERDI_HOME [YOUR_VERDI_HOME_PATH]
# run the python program
# - Input arguments depend on your program design
# - If loading design is required, you can pass the options like
./demo.py -sv demo.v
```

To run the example, put the above files in the same directory and execute the C shell script `run_demo`.

Chapter 1: Introduction to Python Based NPI User Interface and Use Flow

```
./run_demo
```

2

Module npisys

This chapter contains the following topics:

- [Overview](#)
- [L0 APIs](#)

Overview

Module npisys is for setting Python-based NPI. You must call `npisys.init()` before using any other NPI modules and call `npisys.end()` after using any other NPI modules.

L0 APIs

Following are the public L0 APIs for system module:

`npisys.init(pyArgvList)`

System initialization for Python-Based NPI.

Parameters: `pyArgvList` (str list) – input argument list, for example, `sys.argv`

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.init(sys.argv)
```

`npisys.load_design(pyArgvList)`

Load design for Python-Based NPI.

Parameters: `pyArgvList` (str list) – input argument list, for example, `sys.argv`

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.load_design(sys.argv)
```

npisys.end()

Clean NPI-related settings and data.

Parameters: none

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.end()
```


3

Python-Based NPI Language Model

This chapter contains the following topics:

- [Overview](#)
- [Quick Start](#)

Overview

NPI Language Model contains three language models:

- SystemVerilog Model
- VHDL Model
- SystemC Model

It lets users traverse designs from the HDL language perspective. Every construct in the language will be regarded as an object.

For NPI Language - SystemVerilog Model, the IEEE Std 1800 -2009 SystemVerilog Language Reference Manual (LRM) is applied as the foundation of it. Most of the SystemVerilog object model diagrams are similar to VPI Object Model Diagram in LRM Chapter 37. For NPI Language - VHDL Model, the IEEE Std1076 -2008 VHDL Language Reference Manual (LRM) is applied as the foundation of it. However, the style of the VHDL object model diagrams will follow NPI Language - SystemVerilog Model.

Refer to API Reference to see all the APIs that are used in the Language Model. Refer to supported properties and methods of each object type to understand the HDL language structure and to learn how to traverse designs via the Language Model.

Quick Start

Environment and library setting:

1. Use Python version greater than 3.6.0.
2. Update LD_LIBRARY_PATH to access NPI shared library put in Verdi release package.

If there exists any error like “cannot open shared object file”, please add the shared library path to LD_LIBRARY_PATH before running your Python-Based NPI program.

For example, you can set the following environment under 64-bit Linux machine by a C shell (csh) script:

```
setenv LD_LIBRARY_PATH
$VERDI_HOME/share/NPI/lib/linux64:$LD_LIBRARY_PATH
setenv LD_LIBRARY_PATH
$VERDI_HOME/platform/linux64/bin:$LD_LIBRARY_PATH
```

3. Add python library path into your python code:

```
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(rel_lib_path))
```

4. Import “npisys” to use the function of NPI initialization and exit.

Import “lang” to use the APIs of Language Model.

Import libraries as follows:

```
from pynpi import lang
from pynpi import npisys
```

Demo Case

In the demo case, three files are needed.

Design file (demo.v):

```
module top1;
  sub sub_inst1();
  sub sub_inst2();
endmodule

module top2;
  parameter P1 = 11;
  parameter P2 = 22;
endmodule

module sub;
  parameter P = 10;
endmodule
```

Python program to use NPI language model (demo.py):

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
```

```

sys.path.append(os.path.abspath(rel_lib_path))
from pynpi import lang
from pynpi import npisys

# Initialize NPI
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI")
    assert 0
# Load design
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design")
    assert 0

# Start to develop your code -----
# Function to dump handle information
def dump_hdl_info_by_l0(hdl):
    # dump string properties
    print(" npiName    = "+hdl.name())
    print(" npiType    = "+hdl.type())
    print(" npiFile     = "+hdl.file())
    # dump int properties
    print(" npiLineNo = "+str(hdl.line_no()))

# Get top instances
print("==== Get top instances =====")
inst_list = lang.get_top_inst_list()
for inst in inst_list:
    print("Top instance: "+inst.full_name())
    dump_hdl_info_by_l0(inst)

# Get specified handle by full name
print("==== Get specified handle by full name =====")
p1 = lang.handle_by_name("top2.P1", None)
print("Get P1 by \"handle_by_name\": "+p1.full_name())
dump_hdl_info_by_l0(p1)

# 1-1 relationship
print("==== 1-1 relationship =====")
scope = p1.scope_handle()
print("Get scope handle of P1: "+scope.full_name())
dump_hdl_info_by_l0(scope)

# 1-m relationship
print("==== 1-m relationship =====")
param_list = scope.parameter_handles()
for param in param_list:
    print("Get parameter handle of top2: "+param.full_name())
    dump_hdl_info_by_l0(param)
# End of your code -----

# End NPI
npisys.end()

```

C shell script to setup environment and execute Python program (run_demo):

```
#!/bin/csh -f

# You can also setup your $VERDI_HOME here
# setup NPI shared library path
setenv LD_LIBRARY_PATH $VERDI_HOME/share/NPI/lib/linux64:$LD_LIBRARY_PATH
setenv LD_LIBRARY_PATH $VERDI_HOME/platform/linux64/bin:$LD_LIBRARY_PATH
# run the python program
./demo.py -sv demo.v
```

Execution:

```
./run_demo
```

Result:

```
===== Get top instances =====
Top instance: top1
  npiname    = top1
  npitype    = npimodule
  npifile    = demo.v
  npilineNo  = 1
Top instance: top2
  npiname    = top2
  npitype    = npimodule
  npifile    = demo.v
  npilineNo  = 6
===== Get specified handle by full name =====
Get P1 by "handle_by_name": top2.P1
  npiname    = P1
  npitype    = npiparameter
  npifile    = demo.v
  npilineNo  = 7
===== 1-1 relationship =====
Get scope handle of P1: top2
  npiname    = top2
  npitype    = npimodule
  npifile    = demo.v
  npilineNo  = 6
===== 1-m relationship =====
Get parameter handle of top2: top2.P1
  npiname    = P1
  npitype    = npiparameter
  npifile    = demo.v
  npilineNo  = 7
Get parameter handle of top2: top2.P2
  npiname    = P2
  npitype    = npiparameter
  npifile    = demo.v
  npilineNo  = 8
```

4

Class Objects

This chapter contains the following topics:

- [Supported Properties and Methods](#)
 - [Mixed Design Objects](#)
 - [SystemVerilog Objects](#)
 - [VHDL Objects](#)
 - [SystemC Objects](#)
-

Supported Properties and Methods

All possible relationships and properties that an object might have are documented in the form of object diagrams, which use enclosures to represent objects and arrows to represent the type of relationship.

The 1-1 relationship is named “tag_handle()”, for example, “hdl.scope_handle()” will return the Scope handle of hdl.

The 1-m relationship is named “tag_handles()”, for example, “hdl.internal_scope_handles()” will return a list of all internal scopes of hdl.

A property might return an integer or a string. If the property supports both integer and string, the option “is_get_enum” is used to control the output format. For example, the property “Type” supports both integer and string, the property is defined by the following syntax:

```
def type(self, is_get_enum=False)
```

Default value of “is_get_enum” is False. “hdl.type()” returns object type in a string and “hdl.type(True)” returns object type in an integer.

All objects have three common properties:

1. **LangType**: It shows the language type, and the returned property type will be CommonUseLang (for every language model could be used in mixed design), SvLang (for SystemVerilog Model), VhdlLang (for VHDL Model) or ScLang (for SystemC Model).

2. **BelongToHierObj**: It shows if the object belongs to hierarchy scope. If this object is non-scope object (ex. constant and vh literal.), the return value of this property will always be 0. Please refer to Vh Design Unit and UDP.
3. **Instrumented**: If the object is instrumented by VCS with MVSIM NLP (Native Low Power) mode, the returned value will be 1. Otherwise the returned value will be 0.

Mixed Design Objects

Mixed design objects are composed by SystemVerilog Model, VHDL Model and SystemC Model. The following table describes the Object Category and their corresponding object types or tags with pyDoc link:

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Lang Interface	lang interface	npiLangInterface
Port	port	npiPort
Iterator	iterator	(container)

For more information about Mixed Design Objects Object Category, see the “Object Diagrams” chapter in the *VC Apps Native Programming Interface (NPI) Guide*.

npiterator

class lang.IteratorHandle(cps_obj)

Class for object type “npiterator”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
        print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

npilanginterface

class lang.LangInterfaceHandle(*cps_obj*)

Class for object type “npilanginterface”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

actual_handle()

Get handle lists of the 1-1 relationship “npiActual”.

Parameters: None.

Returns: handle of 1-1 relationship “npiActual”.

Return type: handle

Examples

```
>>> hdl = ref.actual_handle()
```

port_handles()

Get handle lists of the 1-m relationship “npiPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

param_map_handles()

Get handle lists of the 1-m relationship “npiParamMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_map_handles()
```

npiPort

***class* lang.PortHandle(*cps_obj*)**

Class for object type “npiPort”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

direction(is_get_enum=False)

Get property “npiDirection”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiDirection”.

Return type: integer or string

Examples

```
>>> hdl.direction()     # Get string
>>> hdl.direction(True) # Get integer
```

port_index()

Get integer property “npiPortIndex”.

Parameters: None.

Returns: integer property of “npiPortIndex”.

Return type: int

Examples

```
>>> hdl.port_index()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

port_type(is_get_enum=False)

Get property “npiPortType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiPortType”.

Return type: integer or string

Examples

```
>>> hdl.port_type()      # Get string
>>> hdl.port_type(True)  # Get integer
```

port_lang_status(is_get_enum=False)

Get property “npPortLangStatus”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npPortLangStatus”.

Return type: integer or string

Examples

```
>>> hdl.port_lang_status()      # Get string
>>> hdl.port_lang_status(True)  # Get integer
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npFile”.

Parameters: None.

Returns: string property of “npFile”.

Return type: string

Examples

```
>>> hdl.file()
```

high_conn_handle()

Get handle lists of the 1-1 relationship “npHighConn”.

Parameters: None.

Returns: handle of 1-1 relationship “npHighConn”.

Return type: handle

Examples

```
>>> hdl = ref.high_conn_handle()
```

low_conn_handle()

Get handle lists of the 1-1 relationship “npiLowConn”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLowConn”.

Return type: handle

Examples

```
>>> hdl = ref.low_conn_handle()
```

vh_map_elem_handles()

Get handle lists of the 1-m relationship “npiVhMapElem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhMapElem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_map_elem_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```


SystemVerilog Objects

The following table describes the Object Category and their corresponding object types or tags with pyDoc link:

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Instance	Instance	(class)
Module	sc scope	npiModule
Interface	sc method process	npiInterface
Program	program	npiProgram
Modport	modport	npiModport
Instance Arrays	instance array	(class)
Scope	scope	(class)
IO Declaration	io declaration	npiIODecl
Reference Objects	ref obj	npiRefObj
External Reference	ext ref	npiExtRef
Nets	nets	(class)
Net select	net select	npiNetSelect
Variables	variables	(class)
Variable Select	var select	npiVarSelect
Object Range	range	npiRange
Typespec	typespec	(class)
Named Events	named event	npiNamedEvent
Parameter, Param Assign, Def Param	parameter, param assign, def param	npiParameter , npiTypeParameter , npiParameter
Parameter Select	parameter select	npiParameterSelect
Primitive	primitive	(class)
UDP	udp	npiUdp

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Task and Function Declaration	task func	(class)
Task and Function Call	tf call	(class)
Continuous Assignment	cont assign	npiContAssign
Alias Statement	alias stmt	npiAliasStmt
Bit Select	bit select	npiBitSelect
Simple Expressions	simple expr	(class)
Use	use	(class)
Expressions	expression	(class)
Atomic Statement	atomic stmt, stmt	(class)
Event Statement	event stmt	npiEventStmt
Process	process	(class)
Assignment	assignment	npiAssignment
Event Control	event control “@”	npiEventControl
While, Repeat	while, repeat	npiWhile , npiRepeat
Wait	wait	npiWait
Delay Control	delay control “#”	npiDelayControl
Repeat Control	repeat control	npiDelayControl
Forever	forever	npiForever
If, If-else	if, if-else	npilf , npilfElse
Case	case	npiCase
For	for	npiFor
Do-while	do while	npiDoWhile
Foreach Statement	foreach statement	npiForeachStmt
Disable	disable	npiDisable
Return Statement	return stmt	npiReturnStmt

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Assign Statement, Deassign, Force, Release	assign stmt, deassign, force, release	(class)
Generates	gen scope	npiGenScope
Pattern Item	pattern item	(class)
Class Variables, Class Objects	class var, class obj	npiClassVar , npiClassObj
Class Definition	class defn	npiClassDefn
Class Typespec	class typespec	npiClassTypespec
Class Access Objects	class access obj	npiClassAccessObj
Virtual Interface Var	virtual interface var	npiVirtualInterfaceVar
Interface Typespec	interface typespec	npiInterfaceTypespec
Import Item	import item	npiImportItem
Export Item	export item	npiExportItem
Clocking Block, Clocking IO declaration	clocking block, clocking io decl	npiClockingBlock , npiClockingIODecl
Assertion	assertion	(class)
Sequence	sequence	npiSequenceInst , npiSequenceDecl , npiSeqFormalDecl
Property	property	npiPropertyDecl , npiPropertyInst , npiSequenceInst
Distribution	distribution, dist item	npiDistribution , npiDistItem
Constraint	constraint, constraint item, constraint ordering, constraint expr	npiConstraint , npiConstraintOrdering , npiConstrIf , npiConstrIfElse , npiConstrForeach
Covergroup	covergroup, coverage option, coverage spec, cover point, cover cross, bins	npiCovergroup , npiCoverageOption , npiCoverPoint , npiCoverCross , npiBins

For more information about SystemVerilog Object Category, see the “Object Diagrams” chapter in the *VC Apps Native Programming Interface (NPI) Guide*.

npirealvar

class lang.RealVarHandle(cps_obj)

Class for object type “npirealvar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string  
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npistructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabName”.

Parameters: None.

Returns: string property of “npielabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabFullName”.

Parameters: None.

Returns: string property of “npielabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiReturnStmt

class lang.ReturnStmtHandle(cps_obj)

Class for object type “npiReturnStmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python
```

```
import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npimodulearray

class lang.ModuleArrayHandle(cps_obj)

Class for object type “npiModuleArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum (bool)` – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

contain_black_box()

Get integer property “npicontainblackbox”.

Parameters: None.

Returns: integer property of “npicontainblackbox”.

Return type: int

Examples

```
>>> hdl.contain_black_box()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```


size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiMethodTaskCall

class lang.MethodTaskCallHandle(*cps_obj*)

Class for object type “npiMethodTaskCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

visibility
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

is_expr()

Get integer property “npilsExpr”.

Parameters: None.

Returns: integer property of “npilsExpr”.

Return type: int

Examples

```
>>> hdl.is_expr()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

task_handle()

Get handle lists of the 1-1 relationship “npiTask”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTask”.

Return type: handle

Examples

```
>>> hdl = ref.task_handle()
```

prefix_handle()

Get handle lists of the 1-1 relationship “npiPrefix”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPrefix”.

Return type: handle

Examples

```
>>> hdl = ref.prefix_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npilmmmediateAssert

class lang.ImmediateAssertHandle(cps_obj)

Class for object type “npilmmmediateAssert”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```



```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type()      # Get string
>>> hdl.type(True)  # Get integer
```

is_deferred()

Get integer property “npilsDeferred”.

Parameters: None.

Returns: integer property of “npilsDeferred”.

Return type: int

Examples

```
>>> hdl.is_deferred()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

else_stmt_handle()

Get handle lists of the 1-1 relationship “npiElseStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElseStmt”.

Return type: handle

Examples

```
>>> hdl = ref.else_stmt_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiTypePattern

class lang.TypePatternHandle(cps_obj)

Class for object type “npiTypePattern”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiFile`”.

Parameters: None.

Returns: string property of “`npiFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`index_handle()`

Get handle lists of the 1-1 relationship “`npiIndex`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npiIndex`”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiPropFormalDecl

class lang.PropFormalDeclHandle(*cps_obj*)

Class for object type “npiPropFormalDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`direction(is_get_enum=False)`

Get property “`npiParam`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
>>> hdl.direction(True) # Get integer
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiClassObj

class lang.ClassObjHandle(cps_obj)

Class for object type “npiClassObj”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
example.v:
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
```

```

        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

direction(is_get_enum=False)

Get property “npiDirection”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiDirection”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
>>> hdl.direction(True) # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

class_typespec_handle()

Get handle lists of the 1-1 relationship “npiClassTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClassTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.class_typespec_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

methods_handles()

Get handle lists of the 1-m relationship “npiMethods”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMethods”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.methods_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```


property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiTaskCall

class lang.TaskCallHandle(*cps_obj*)

Class for object type “npiTaskCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

const_type(is_get_enum=False)

Get property “npIConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

task_handle()

Get handle lists of the 1-1 relationship “npiTask”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTask”.

Return type: handle

Examples

```
>>> hdl = ref.task_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiClassTypespec

class lang.ClassTypespecHandle(cps_obj)

Class for object type “npiClassTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

class_defn_handle()

Get handle lists of the 1-1 relationship “npiClassDefn”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClassDefn”.

Return type: handle

Examples

```
>>> hdl = ref.class_defn_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

param_assign_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_assign_handles()
```

methods_handles()

Get handle lists of the 1-m relationship “npiMethods”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMethods”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.methods_handles()
```

constraint_handles()

Get handle lists of the 1-m relationship “npiConstraint”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraint”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npilntVar

class lang.IntVarHandle(cps_obj)

Class for object type “npilntVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiParamAutomatic”.

Parameters: None.

Returns: integer property of “npiParamAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiParamRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParamRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiParamVisibility”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiVisibility`”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

`size()`

Get integer property “`npiSize`”.

Parameters: None. –

Returns: integer property of “`npiSize`”.

Return type: int

Examples

```
>>> hdl.size()
```

`array_member()`

Get integer property “`npiArrayMember`”.

Parameters: None.

Returns: integer property of “`npiArrayMember`”.

Return type: int

Examples

```
>>> hdl.array_member()
```

`struct_union_member()`

Get integer property “`npiStructUnionMember`”.

Parameters: None.

Returns: integer property of “`npiStructUnionMember`”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```


line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npilUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npilUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npilRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npilRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npilDefaultPattern

class lang.DefaultPatternHandle(*cps_obj*)

Class for object type “npilDefaultPattern”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiNamedBegin

***class* lang.NamedBeginHandle(*cps_obj*)**

Class for object type “npiNamedBegin”.

set_persistent()

Set the handle as ‘persistent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

return_handle()

Get handle lists of the 1-1 relationship “npiReturn”.

Parameters: None.

Returns: handle of 1-1 relationship “npiReturn”.

Return type: handle

Examples

```
>>> hdl = ref.return_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npifunction

class lang.FunctionHandle(cps_obj)

Class for object type “npifunction”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

return_handle()

Get handle lists of the 1-1 relationship “npiReturn”.

Parameters: None.

Returns: handle of 1-1 relationship “npiReturn”.

Return type: handle

Examples

```
>>> hdl = ref.return_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npilODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npilODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npilInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npilInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npilntTypespec

***class* lang.IntTypespecHandle(*cps_obj*)**

Class for object type “npilntTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiFork

class lang.ForkHandle(cps_obj)

Class for object type “npiFork”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```

        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParamater”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamater”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVaribles”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVaribles”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTtypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTtypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiParamDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npisSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npisSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiTimeTypespec

class lang.TimeTypespecHandle(*cps_obj*)

Class for object type “npiTimeTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python
```

```
import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```


type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiClassAccessObj

class lang.ClassAccessObjHandle(cps_obj)

Class for object type “npiClassAccessObj”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

is_expr()

Get integer property “npilsExpr”.

Parameters: None.

Returns: integer property of “npilsExpr”.

Return type: int

Examples

```
>>> hdl.is_expr()
```

access_type(is_get_enum=False)

Get property “npiAccessType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiAccessType”.

Return type: integer or string

Examples

```
>>> hdl.access_type() # Get string  
>>> hdl.access_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

actual_handle()

Get handle lists of the 1-1 relationship “npiActual”.

Parameters: None.

Returns: handle of 1-1 relationship “npiActual”.

Return type: handle

Examples

```
>>> hdl = ref.actual_handle()
```

prefix_handle()

Get handle lists of the 1-1 relationship “npiPrefix”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPrefix”.

Return type: handle

Examples

```
>>> hdl = ref.prefix_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiEnumConst

class lang.EnumConstHandle(cps_obj)

Class for object type “npiEnumConst”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```



```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(*is_get_enum=False*)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(*is_get_enum=False*)

Get property “npiconstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiconstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

value_eval_str()

Get string property “npiValueEvalStr”.

Parameters: None.

Returns: string property of “npiValueEvalStr”.

Return type: string

Examples

```
>>> hdl.value_eval_str()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

enum_typespec_handle()

Get handle lists of the 1-1 relationship “npiEnumTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiEnumTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.enum_typespec_handle()
```

npiStructNet

class lang.StructNetHandle(*cps_obj*)

Class for object type “npiStructNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`packed_array_member()`

Get integer property “`npiParamArrayMember`”.

Parameters: None.

Returns: integer property of “`npiParamArrayMember`”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

`net_type(is_get_enum=False)`

Get property “`npiParamType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

`implicit_decl()`

Get integer property “`npiParamDecl`”.

Parameters: None.

Returns: integer property of “`npiParamDecl`”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```


member_handles()

Get handle lists of the 1-m relationship “npiMember”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMember”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.member_handles()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiTimeNet

class lang.TimeNetHandle(cps_obj)

Class for object type “npiTimeNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

net_type(is_get_enum=False)

Get property “npinetype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinetype”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npimplicitdecl”.

Parameters: None.

Returns: integer property of “npimplicitdecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```


ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiStructTypespec

class lang.StructTypespecHandle(cps_obj)

Class for object type “npiStructTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```

from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`packed()`

Get integer property “`npiPacked`”.

Parameters: None.

Returns: integer property of “`npiPacked`”.

Return type: int

Examples

```
>>> hdl.packed()
```

`signed()`

Get integer property “`npiSigned`”.

Parameters: None.

Returns: integer property of “`npiSigned`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

typespec_member_handles()

Get handle lists of the 1-m relationship “npiTypespecMember”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypespecMember”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typespec_member_handles()
```

npiArrayTypespec

***class* lang.ArrayTypespecHandle(*cps_obj*)**

Class for object type “npiArrayTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

array_type(is_get_enum=False)

Get property “npIArrayType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIArrayType”.

Return type: integer or string

Examples

```
>>> hdl.array_type() # Get string
>>> hdl.array_type(True) # Get integer
```

signed()

Get integer property “npISigned”.

Parameters: None.

Returns: integer property of “npISigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```


elem_typespec_handle()

Get handle lists of the 1-1 relationship “npiElemTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElemTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.elem_typespec_handle()
```

index_typespec_handle()

Get handle lists of the 1-1 relationship “npiIndexTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndexTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.index_typespec_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiDeassign

class lang.DeassignHandle(cps_obj)

Class for object type “npiDeassign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiParamFile`”.

Parameters: None.

Returns: string property of “`npiParamFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`lhs_handle()`

Get handle lists of the 1-1 relationship “`npiParamLhs`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npiParamLhs`”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npIntegerTypespec

***class* lang.IntegerTypespecHandle(*cps_obj*)**

Class for object type “npIntegerTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()      # Get string
>>> hdl.type(True)  # Get integer
```

signed()

Get integer property “npISigned”.

Parameters: None.

Returns: integer property of “npISigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```


scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiNamedFork

class lang.NamedForkHandle(cps_obj)

Class for object type “npiNamedFork”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

`full_name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiParamDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiParamDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiParamDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiParamDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiParamDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiParamTypespec

class lang.EnumTypespecHandle(*cps_obj*)

Class for object type “npiParamTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

signed()

Get integer property “npISigned”.

Parameters: None.

Returns: integer property of “npISigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

base_typespec_handle()

Get handle lists of the 1-1 relationship “npiBaseTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiBaseTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.base_typespec_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

enum_const_handles()

Get handle lists of the 1-m relationship “npiEnumConst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiEnumConst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.enum_const_handles()
```

npiStringVar

class lang.StringVarHandle(cps_obj)

Class for object type “npiStringVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npistructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```


expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiPackedArrayTypespec

***class* lang.PackedArrayTypespecHandle(*cps_obj*)**

Class for object type “npiPackedArrayTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`vector()`

Get integer property “`npiVector`”.

Parameters: None.

Returns: integer property of “`npiVector`”.

Return type: int

Examples

```
>>> hdl.vector()
```

`signed()`

Get integer property “`npiSigned`”.

Parameters: None.

Returns: integer property of “`npiSigned`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elem_typespec_handle()

Get handle lists of the 1-1 relationship “npiElemTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElemTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.elem_typespec_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTtypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTtypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npIScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npIScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npIRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npIRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npIGateArray

class lang.GateArrayHandle(*cps_obj*)

Class for object type “npIGateArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiConstrIfElse

class lang.ConstrIfElseHandle(cps_obj)

Class for object type “npiConstrIfElse”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

else_const_handles()

Get handle lists of the 1-m relationship “npiElseConst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiElseConst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.else_const_handles()
```

constraint_expr_handles()

Get handle lists of the 1-m relationship “npiConstraintExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_expr_handles()
```

npiConstrForeach

class lang.ConstrForeachHandle(cps_obj)

Class for object type “npiConstrForeach”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```


Result:

```
Set permanent for object npiVar => Success
Release object npiVar => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiVar”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVar”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npiVarNo”.

Parameters: None.

Returns: integer property of “npiVarNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiVar”.

Parameters: None.

Returns: string property of “npiVar”.

Return type: string

Examples

```
>>> hdl.file()
```

variables_handle()

Get handle lists of the 1-1 relationship “npiVarVariables”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVariables”.

Return type: handle

Examples

```
>>> hdl = ref.variables_handle()
```

loop_vars_handles()

Get handle lists of the 1-m relationship “npiLoopVars”.

Parameters: None.

Returns: handle list of 1-m relationship “npiLoopVars”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.loop_vars_handles()
```

constraint_expr_handles()

Get handle lists of the 1-m relationship “npiConstraintExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_expr_handles()
```

npiCHandleVar

class lang.ChandleVarHandle(*cps_obj*)

Class for object type “npiCHandleVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npivisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiExportItem

class lang.ExportItemHandle(*cps_obj*)

Class for object type “npiExportItem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

npiExtRef

class lang.ExtRefHandle(cps_obj)

Class for object type “npiExtRef”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

actual_handle()

Get handle lists of the 1-1 relationship “npiActual”.

Parameters: None.

Returns: handle of 1-1 relationship “npiActual”.

Return type: handle

Examples

```
>>> hdl = ref.actual_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npIntegerNet

class lang.IntegerNetHandle(cps_obj)

Class for object type “npIntegerNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

net_type(is_get_enum=False)

Get property “npiNetType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npINetType`”.

Return type: integer or string

Examples

```
>>> hdl.net_type()          # Get string
>>> hdl.net_type(True)     # Get integer
```

`implicit_decl()`

Get integer property “`npImplicitDecl`”.

Parameters: None.

Returns: integer property of “`npImplicitDecl`”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

`size()`

Get integer property “`npISize`”.

Parameters: None.

Returns: integer property of “`npISize`”.

Return type: int

Examples

```
>>> hdl.size()
```

`array_member()`

Get integer property “`npIArrayMember`”.

Parameters: None.

Returns: integer property of “`npIArrayMember`”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiShortIntVar

class lang.ShortIntVarHandle(*cps_obj*)

Class for object type “npiShortIntVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npIAutomatic”.

Parameters: None.

Returns: integer property of “npIAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```


rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npUdp

class lang.UdpHandle(cps_obj)

Class for object type “npUdp”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

prim_type(is_get_enum=False)

Get property “npiPrimType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiPrimType”.

Return type: integer or string

Examples

```
>>> hdl.prim_type()      # Get string
>>> hdl.prim_type(True)  # Get integer
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples


```
>>> hdl.size()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

def_name()

Get string property “npidefname”.

Parameters: None.

Returns: string property of “npidefname”.

Return type: string

Examples

```
>>> hdl.def_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

udp_defn_handle()

Get handle lists of the 1-1 relationship “npiUdpDefn”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUdpDefn”.

Return type: handle

Examples

```
>>> hdl = ref.udp_defn_handle()
```

delay_handle()

Get handle lists of the 1-1 relationship “npiDelay”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelay”.

Return type: handle

Examples

```
>>> hdl = ref.delay_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npilIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npilIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npilInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

prim_term_handles()

Get handle lists of the 1-m relationship “npiPrimTerm”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPrimTerm”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.prim_term_handles()
```

npisysTaskCall

class lang.SysTaskCallHandle(cps_obj)

Class for object type “npisysTaskCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```

    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)      # Get integer

```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()      # Get string
>>> hdl.const_type(True)  # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npAssignment

***class* lang.AssignmentHandle(*cps_obj*)**

Class for object type “npAssignment”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```



```

    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

blocking()

Get integer property “npiBlocking”.

Parameters: None.

Returns: integer property of “npiBlocking”.

Return type: int

Examples

```
>>> hdl.blocking()
```

op_type(is_get_enum=False)

Get property “npiOpType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiOpType”.

Return type: integer or string

Examples

```
>>> hdl.op_type()          # Get string
>>> hdl.op_type(True)     # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

delay_control_handle()

Get handle lists of the 1-1 relationship “npiDelayControl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelayControl”.

Return type: handle

Examples

```
>>> hdl = ref.delay_control_handle()
```

event_control_handle()

Get handle lists of the 1-1 relationship “npiEventControl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiEventControl”.

Return type: handle

Examples

```
>>> hdl = ref.event_control_handle()
```

repeat_control_handle()

Get handle lists of the 1-1 relationship “npiRepeatControl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRepeatControl”.

Return type: handle

Examples

```
>>> hdl = ref.repeat_control_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiClockingBlock

***class* lang.ClockingBlockHandle(*cps_obj*)**

Class for object type “npiClockingBlock”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

input_edge()

Get integer property “npInputEdge”.

Parameters: None.

Returns: integer property of “npInputEdge”.

Return type: int

Examples

```
>>> hdl.input_edge()
```

output_edge()

Get integer property “npOutputEdge”.

Parameters: None.

Returns: integer property of “npOutputEdge”.

Return type: int

Examples

```
>>> hdl.output_edge()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```


input_skew_handle()

Get handle lists of the 1-1 relationship “npInputSkew”.

Parameters: None.

Returns: handle of 1-1 relationship “npInputSkew”.

Return type: handle

Examples

```
>>> hdl = ref.input_skew_handle()
```

output_skew_handle()

Get handle lists of the 1-1 relationship “npOutputSkew”.

Parameters: None.

Returns: handle of 1-1 relationship “npOutputSkew”.

Return type: handle

Examples

```
>>> hdl = ref.output_skew_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npIScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npIScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

clocking_i_o_decl_handles()

Get handle lists of the 1-m relationship “npiClockingIODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiClockingIODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.clocking_i_o_decl_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParamater”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamater”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVaribles”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVaribles”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTtypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTtypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiParameterDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameterDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiRestrict

class lang.RestrictHandle(cps_obj)

Class for object type “npiRestrict”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle
```

property_handle()

Get handle lists of the 1-1 relationship “npiProperty”.

Parameters: None.

Returns: handle of 1-1 relationship “npiProperty”.

Return type: handle

Examples

```
>>> hdl = ref.property_handle()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiPackage

class lang.PackageHandle(cps_obj)

Class for object type “npiPackage”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

is_root()

Get integer property “npilsRoot”.

Parameters: None.

Returns: integer property of “npilsRoot”.

Return type: int

Examples

```
>>> hdl.is_root()
```

top()

Get integer property “npitop”.

Parameters: None.

Returns: integer property of “npitop”.

Return type: int

Examples

```
>>> hdl.top()
```

cell_instance()

Get integer property “npiCellInstance”.

Parameters: None.

Returns: integer property of “npiCellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

def_line_no()

Get integer property “npiDefLineNo”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

contain_protect()

Get integer property “npiContainProtect”.

Parameters: None.

Returns: integer property of “npiContainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

is_protected()

Get integer property “npilsProtected”.

Parameters: None.

Returns: integer property of “npilsProtected”.

Return type: int

Examples

```
>>> hdl.is_protected()
```

bind()

Get integer property “npiBind”.

Parameters: None.

Returns: integer property of “npiBind”.

Return type: int

Examples

```
>>> hdl.bind()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npiDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

def_file()

Get string property “npiDefFile”.

Parameters: None.

Returns: string property of “npiDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

library_name()

Get string property “npiLibraryName”.

Parameters: None.

Returns: string property of “npiLibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

def_library_name()

Get string property “npiDefLibraryName”.

Parameters: None.

Returns: string property of “npiDefLibraryName”.

Return type: string

Examples

```
>>> hdl.def_library_name()
```

time_scale()

Get string property “npiParamScale”.

Parameters: None.

Returns: string property of “npiParamScale”.

Return type: string

Examples

```
>>> hdl.time_scale()
```

file()

Get string property “npiParamFile”.

Parameters: None.

Returns: string property of “npiParamFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiParamName”.

Parameters: None.

Returns: string property of “npiParamName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiParamFullName”.

Parameters: None.

Returns: string property of “npiParamFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

net_handles()

Get handle lists of the 1-m relationship “npinet”.

Parameters: None.

Returns: handle list of 1-m relationship “npinet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.net_handles()
```

array_net_handles()

Get handle lists of the 1-m relationship “npisArrayNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npisArrayNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.array_net_handles()
```

task_func_handles()

Get handle lists of the 1-m relationship “npitaskFunc”.

Parameters: None.

Returns: handle list of 1-m relationship “npitaskFunc”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.task_func_handles()
```

import_handles()

Get handle lists of the 1-m relationship “npilImport”.

Parameters: None.

Returns: handle list of 1-m relationship “npilImport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.import_handles()
```


export_handles()

Get handle lists of the 1-m relationship “npiExport”.

Parameters: None.

Returns: handle list of 1-m relationship “npiExport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.export_handles()
```

def_param_handles()

Get handle lists of the 1-m relationship “npiDefParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npiDefParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.def_param_handles()
```

clocking_block_handles()

Get handle lists of the 1-m relationship “npiClockingBlock”.

Parameters: None.

Returns: handle list of 1-m relationship “npiClockingBlock”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.clocking_block_handles()
```

assertion_handles()

Get handle lists of the 1-m relationship “npiAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.assertion_handles()
```

covergroup_handles()

Get handle lists of the 1-m relationship “npiCovergroup”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCovergroup”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.covergroup_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParamater”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamater”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVaribles”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVaribles”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTtypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTtypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiParameterDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameterDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npIndexedPartSelect

***class* lang.IndexedPartSelectHandle(*cps_obj*)**

Class for object type “npIndexedPartSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
```

constant_select()

Get integer property “npiconstantSelect”.

Parameters: None.

Returns: integer property of “npiconstantSelect”.

Return type: int

Examples

```
>>> hdl.constant_select()
```

indexed_part_select_type(is_get_enum=False)

Get property “npindexedPartSelectType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npindexedPartSelectType”.

Return type: integer or string

Examples

```
>>> hdl.indexed_part_select_type() # Get string
>>> hdl.indexed_part_select_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

base_expr_handle()

Get handle lists of the 1-1 relationship “npiBaseExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiBaseExpr”.

Return type: handle

Examples

```
>>> hdl = ref.base_expr_handle()
```

width_expr_handle()

Get handle lists of the 1-1 relationship “npiWidthExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiWidthExpr”.

Return type: handle

Examples

```
>>> hdl = ref.width_expr_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiParamSelect

class lang.NetSelectHandle(cps_obj)

Class for object type “npiParamSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiRepeat

class lang.RepeatHandle(cps_obj)

Class for object type “npiRepeat”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npInterface

class lang.InterfaceHandle(*cps_obj*)

Class for object type “npInterface”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npNet => Success
Release object npNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

is_black_box()

Get integer property “npIsBlackBox”.

Parameters: None.

Returns: integer property of “npIsBlackBox”.

Return type: int

Examples

```
>>> hdl.is_black_box()
```

top()

Get integer property “npTop”.

Parameters: None.

Returns: integer property of “npTop”.

Return type: int

Examples

```
>>> hdl.top()
```

cell_instance()

Get integer property “npCellInstance”.

Parameters: None.

Returns: integer property of “npiCellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

def_line_no()

Get integer property “npiDefLineNo”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

contain_protect()

Get integer property “npiContainProtect”.

Parameters: None.

Returns: integer property of “npiContainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

is_protected()

Get integer property “npilsProtected”.

Parameters: None.

Returns: integer property of “npilsProtected”.

Return type: int

Examples

```
>>> hdl.is_protected()
```

bind()

Get integer property “npiBind”.

Parameters: None.

Returns: integer property of “npiBind”.

Return type: int

Examples

```
>>> hdl.bind()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

def_file()

Get string property “npDefFile”.

Parameters: None.

Returns: string property of “npDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

library_name()

Get string property “npLibraryName”.

Parameters: None.

Returns: string property of “npLibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

def_library_name()

Get string property “npDefLibraryName”.

Parameters: None.

Returns: string property of “npDefLibraryName”.

Return type: string

Examples

```
>>> hdl.def_library_name()
```

time_scale()

Get string property “npTimeScale”.

Parameters: None.

Returns: string property of “npiTimeScale”.

Return type: string

Examples

```
>>> hdl.time_scale()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npiInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npiInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

cont_assign_handles()

Get handle lists of the 1-m relationship “npiContAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiContAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cont_assign_handles()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

param_assign_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_assign_handles()
```

process_handles()

Get handle lists of the 1-m relationship “npiProcess”.

Parameters: None.

Returns: handle list of 1-m relationship “npiProcess”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.process_handles()
```

modport_handles()

Get handle lists of the 1-m relationship “npiModport”.

Parameters: None.

Returns: handle list of 1-m relationship “npiModport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.modport_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

alias_stmt_handles()

Get handle lists of the 1-m relationship “npiAliasStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAliasStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.alias_stmt_handles()
```

net_handles()

Get handle lists of the 1-m relationship “npiNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.net_handles()
```

array_net_handles()

Get handle lists of the 1-m relationship “npiArrayNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArrayNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.array_net_handles()
```

task_func_handles()

Get handle lists of the 1-m relationship “npiTaskFunc”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTaskFunc”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.task_func_handles()
```

import_handles()

Get handle lists of the 1-m relationship “npilImport”.

Parameters: None.

Returns: handle list of 1-m relationship “npIImport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.import_handles()
```

export_handles()

Get handle lists of the 1-m relationship “npIExport”.

Parameters: None.

Returns: handle list of 1-m relationship “npIExport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.export_handles()
```

def_param_handles()

Get handle lists of the 1-m relationship “npIDefParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npIDefParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.def_param_handles()
```

clocking_block_handles()

Get handle lists of the 1-m relationship “npIClockingBlock”.

Parameters: None.

Returns: handle list of 1-m relationship “npIClockingBlock”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.clocking_block_handles()
```

assertion_handles()

Get handle lists of the 1-m relationship “npIAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.assertion_handles()
```

covergroup_handles()

Get handle lists of the 1-m relationship “npiCovergroup”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCovergroup”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.covergroup_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npIPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiVirtualInterfaceVar

class lang.VirtualInterfaceVarHandle(*cps_obj*)

Class for object type “npiVirtualInterfaceVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```



```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

is_modport()

Get integer property “npilsModport”.

Parameters: None.

Returns: integer property of “npilsModport”.

Return type: int

Examples

```
>>> hdl.is_modport()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabName”.

Parameters: None.

Returns: string property of “npielabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabFullName”.

Parameters: None.

Returns: string property of “npielabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npitypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

npiPackedArrayVar

class lang.PackedArrayVarHandle(cps_obj)

Class for object type “npiStringVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```

        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

packed_array_member()

Get integer property “npiPackedArrayMember”.

Parameters: None.

Returns: integer property of “npiPackedArrayMember”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiPropertyInst

class lang.PropertyInstHandle(*cps_obj*)

Class for object type “npiPropertyInst”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabfullname”.

Parameters: None.

Returns: string property of “npielabfullname”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npipropertydecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npipropertydecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiBitVar

***class* lang.BitVarHandle(*cps_obj*)**

Class for object type “npiBitVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
```

automatic()

Get integer property “npiautomatic”.

Parameters: None.

Returns: integer property of “npiautomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npirandtype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npirandtype”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiRegBit

class lang.RegBitHandle(cps_obj)

Class for object type “npiRegBit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npIAutomatic”.

Parameters: None.

Returns: integer property of “npIAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npitableEntry

class lang.TableEntryHandle(cps_obj)

Class for object type “npitableEntry”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```


value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: is_get_default (int) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```

```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

npiConstraintOrdering

class lang.ConstraintOrderingHandle(cps_obj)

Class for object type “npiConstraintOrdering”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

solve_before_handles()

Get handle lists of the 1-m relationship “npiSolveBefore”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSolveBefore”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.solve_before_handles()
```

solve_after_handles()

Get handle lists of the 1-m relationship “npiSolveAfter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSolveAfter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.solve_after_handles()
```

npilfElse

class lang.IfElseHandle(cps_obj)

Class for object type “npilfElse”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

qualifier(is_get_enum=False)

Get property “npiQualifier”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npQualifier`”.

Return type: integer or string

Examples

```
>>> hdl.qualifier()          # Get string
>>> hdl.qualifier(True)     # Get integer
```

`line_no()`

Get integer property “`npLineNo`”.

Parameters: None.

Returns: integer property of “`npLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npFile`”.

Parameters: None.

Returns: string property of “`npFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`else_stmt_handle()`

Get handle lists of the 1-1 relationship “`npElseStmt`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npElseStmt`”.

Return type: handle

Examples

```
>>> hdl = ref.else_stmt_handle()
```


condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiParameterSelect

class lang.ParameterSelectHandle(cps_obj)

Class for object type “npiParameterSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiPropertySpec

class lang.PropertySpecHandle(cps_obj)

Class for object type “npiPropertySpec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

property_expr_handle()

Get handle lists of the 1-1 relationship “npiPropertyExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPropertyExpr”.

Return type: handle

Examples

```
>>> hdl = ref.property_expr_handle()
```

disable_condition_handle()

Get handle lists of the 1-1 relationship “npiDisableCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDisableCondition”.

Return type: handle

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

local_param()

Get integer property “npiLocalParam”.

Parameters: None.

Returns: integer property of “npiLocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()          # Get string
>>> hdl.const_type(True)     # Get integer
```

set_value_by_option()

Get integer property “npiSetValueByOption”.

Parameters: None.

Returns: integer property of “npisetValueByOption”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiParameter

class lang.ParameterHandle(cps_obj)

Class for object type “npiParameter”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())
```

```
# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

local_param()

Get integer property “npilocalParam”.

Parameters: None.

Returns: integer property of “npilocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()          # Get string
>>> hdl.const_type(True)     # Get integer
```

set_value_by_option()

Get integer property “npiSetValueByOption”.

Parameters: None.

Returns: integer property of “npiSetValueByOption”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npilUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npilUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npilRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npilRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npilDistItem

class lang.DistItemHandle(*cps_obj*)

Class for object type “npilDistItem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

dist_type()

Get integer property “npiDistType”.

Parameters: None.

Returns: integer property of “npiDistType”.

Return type: int

Examples

```
>>> hdl.dist_type()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

distribution_handle()

Get handle lists of the 1-1 relationship “npiDistribution”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDistribution”.

Return type: handle

Examples

```
>>> hdl = ref.distribution_handle()
```

value_range_handle()

Get handle lists of the 1-1 relationship “npiValueRange”.

Parameters: None.

Returns: handle of 1-1 relationship “npiValueRange”.

Return type: handle

Examples

```
>>> hdl = ref.value_range_handle()
```

weight_handle()

Get handle lists of the 1-1 relationship “npiWeight”.

Parameters: None.

Returns: handle of 1-1 relationship “npiWeight”.

Return type: handle

Examples

```
>>> hdl = ref.weight_handle()
```

npiPrimTerm

class lang.PrimTermHandle(cps_obj)

Class for object type “npiPrimTerm”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`direction(is_get_enum=False)`

Get property “`npiParam`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
```

`term_index()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.term_index()
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

primitive_handle()

Get handle lists of the 1-1 relationship “npiPrimitive”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPrimitive”.

Return type: handle

Examples

```
>>> hdl = ref.primitive_handle()
```

npiAlways

class lang.AlwaysHandle(*cps_obj*)

Class for object type “npiAlways”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

always_type(is_get_enum=False)

Get property “npialwaystype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npialwaystype”.

Return type: integer or string

Examples

```
>>> hdl.always_type() # Get string
>>> hdl.always_type(True) # Get integer
```


line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```

```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

local_param()

Get integer property “npiLocalParam”.

Parameters: None.

Returns: integer property of “npiLocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()      # Get string
>>> hdl.const_type(True)  # Get integer
```

set_value_by_option()

Get integer property “npiSetValueByOption”.

Parameters: None.

Returns: integer property of “npiSetValueByOption”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npisArrayParameter

class lang.ArrayParameterHandle(cps_obj)

Class for object type “npisArrayParameter”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

npIImmediateCover

class lang.ImmediateCoverHandle(cps_obj)

Class for object type “npIImmediateCover”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent

handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
```

```
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

is_deferred()

Get integer property “npilsDeferred”.

Parameters: None.

Returns: integer property of “npilsDeferred”.

Return type: int

Examples

```
>>> hdl.is_deferred()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiParamDisable

class lang.DisiParamHandle(cps_obj)

Class for object type “npiParamDisable”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiUnionTypespec

class lang.UnionTypespecHandle(cps_obj)

Class for object type “npiUnionTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```

obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

packed()

Get integer property “npiPacked”.

Parameters: None.

Returns: integer property of “npiPacked”.

Return type: int

Examples

```
>>> hdl.packed()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

typespec_member_handles()

Get handle lists of the 1-m relationship “npITypeSpecMember”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypeSpecMember”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typespec_member_handles()
```

npIInterfaceArray

class lang.InterfaceArrayHandle(*cps_obj*)

Class for object type “npIInterfaceArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

contain_black_box()

Get integer property “npicontainblackbox”.

Parameters: None.

Returns: integer property of “npicontainblackbox”.

Return type: int

Examples

```
>>> hdl.contain_black_box()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiBreak

class lang.BreakHandle(cps_obj)

Class for object type “npiBreak”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiGenScope

class lang.GenScopeHandle(cps_obj)

Class for object type “npiGenScope”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

implicit_decl()

Get integer property “npImplicitDecl”.

Parameters: None.

Returns: integer property of “npImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

net_handles()

Get handle lists of the 1-m relationship “npiNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.net_handles()
```

array_net_handles()

Get handle lists of the 1-m relationship “npiArrayNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArrayNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.array_net_handles()
```

process_handles()

Get handle lists of the 1-m relationship “npiProcess”.

Parameters: None.

Returns: handle list of 1-m relationship “npiProcess”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.process_handles()
```

cont_assign_handles()

Get handle lists of the 1-m relationship “npiContAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiContAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cont_assign_handles()
```

primitive_handles()

Get handle lists of the 1-m relationship “npiPrimitive”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPrimitive”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.primitive_handles()
```

def_param_handles()

Get handle lists of the 1-m relationship “npiDefParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npiDefParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.def_param_handles()
```

alias_stmt_handles()

Get handle lists of the 1-m relationship “npiAliasStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAliasStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.alias_stmt_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiShortRealVar

class lang.ShortRealVarHandle(*cps_obj*)

Class for object type “npiShortRealVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npiVisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```


struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

task_handle()

Get handle lists of the 1-1 relationship “npiTask”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTask”.

Return type: handle

Examples

```
>>> hdl = ref.task_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiSysFuncCall

class lang.SysFuncCallHandle(cps_obj)

Class for object type “npiSysFuncCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`value(is_get_default=False)`

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default` (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys
```

```
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

is_expr()

Get integer property “npilsExpr”.

Parameters: None.

Returns: integer property of “npilsExpr”.

Return type: int

Examples

```
>>> hdl.is_expr()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```


const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiNettypeTypespec

class lang.NettypeTypespecHandle(cps_obj)

Class for object type “npiNettypeTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

res_func_handle()

Get handle lists of the 1-1 relationship “npiResFunc”.

Parameters: None.

Returns: handle of 1-1 relationship “npiResFunc”.

Return type: handle

Examples

```
>>> hdl = ref.res_func_handle()
```

elem_typespec_handle()

Get handle lists of the 1-1 relationship “npiElemTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElemTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.elem_typespec_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiFuncCall

class lang.FuncCallHandle(cps_obj)

Class for object type “npiFuncCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```



```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npNet => Success
Release object npNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
```

```
        sub #(.P_INT(20)) sub_inst();
    endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

is_expr()

Get integer property “npilsExpr”.

Parameters: None.

Returns: integer property of “npilsExpr”.

Return type: int

Examples

```
>>> hdl.is_expr()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string  
>>> hdl.const_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

function_handle()

Get handle lists of the 1-1 relationship “npiFunction”.

Parameters: None.

Returns: handle of 1-1 relationship “npiFunction”.

Return type: handle

Examples

```
>>> hdl = ref.function_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiByteTypespec

class lang.ByteTypespecHandle(*cps_obj*)

Class for object type “npiByteTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```


scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiDoWhile

class lang.DoWhileHandle(cps_obj)

Class for object type “npiDoWhile”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiRepeatControl

class lang.RepeatControlHandle(cps_obj)

Class for object type “npiRepeatControl.”

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

event_control_handle()

Get handle lists of the 1-1 relationship “npiEventControl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiEventControl”.

Return type: handle

Examples

```
>>> hdl = ref.event_control_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npinullstmt

class lang.NullStmtHandle(cps_obj)

Class for object type “npinullstmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npilmmmediateAssume

class lang.ImmediateAssumeHandle(cps_obj)

Class for object type “npilmmmediateAssume”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

is_deferred()

Get integer property “npilsDeferred”.

Parameters: None.

Returns: integer property of “npilsDeferred”.

Return type: int

Examples

```
>>> hdl.is_deferred()
```

line_no()

Get integer property “npilLineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npilName”.

Parameters: None.

Returns: string property of “npilName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npilFile”.

Parameters: None.

Returns: string property of “npilFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npilElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

else_stmt_handle()

Get handle lists of the 1-1 relationship “npiElseStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElseStmt”.

Return type: handle

Examples

```
>>> hdl = ref.else_stmt_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiSequenceInst

class lang.SequenceInstHandle(*cps_obj*)

Class for object type “npiSequenceInst”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```


Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npiParamNo”.

Parameters: None.

Returns: integer property of “npiParamNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiParam”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiNet

class lang.NetHandle(cps_obj)

Class for object type “npiNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```

        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

net_type(is_get_enum=False)

Get property “npiNetType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiNetType”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string  
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npiImplicitDecl”.

Parameters: None.

Returns: integer property of “npiImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabName”.

Parameters: None.

Returns: string property of “npielabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabFullName”.

Parameters: None.

Returns: string property of “npielabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npitypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiCase

class lang.CaseHandle(*cps_obj*)

Class for object type “npiCase”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

case_type(is_get_enum=False)

Get property “npicaseType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npicaseType”.

Return type: integer or string

Examples

```
>>> hdl.case_type() # Get string
>>> hdl.case_type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

case_item_handles()

Get handle lists of the 1-m relationship “npiCaseItem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCaseItem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.case_item_handles()
```

npiUdpDefn

class lang.UdpDefnHandle(*cps_obj*)

Class for object type “npiUdpDefn”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

size()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.size()
```

prim_type(is_get_enum=False)

Get property “npiParamPrimType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParamPrimType”.

Return type: integer or string

Examples

```
>>> hdl.prim_type() # Get string
>>> hdl.prim_type(True) # Get integer
```

line_no()

Get integer property “npiParamLineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_name()

Get string property “npilDefName”.

Parameters: None.

Returns: string property of “npilDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

file()

Get string property “npilFile”.

Parameters: None.

Returns: string property of “npilFile”.

Return type: string

Examples

```
>>> hdl.file()
```

initial_handle()

Get handle lists of the 1-1 relationship “npilInitial”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInitial”.

Return type: handle

Examples

```
>>> hdl = ref.initial_handle()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npilIODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npilODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

table_entry_handles()

Get handle lists of the 1-m relationship “npitableEntry”.

Parameters: None.

Returns: handle list of 1-m relationship “npitableEntry”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.table_entry_handles()
```

npicaseItem

class lang.CaseItemHandle(cps_obj)

Class for object type “npicaseItem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiParamNo”.

Parameters: None.

Returns: integer property of “npiParamNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiParam”.

Return type: string

Examples

```
>>> hdl.file()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiParam”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

expr_handles()

Get handle lists of the 1-m relationship “npiExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.expr_handles()
```

npiNetBit

class lang.NetBitHandle(*cps_obj*)

Class for object type “npiNetBit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

net_type(is_get_enum=False)

Get property “npinNetType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinNetType”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npilImplicitDecl”.

Parameters: None.

Returns: integer property of “npilImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiConstant

class lang.ConstantHandle(*cps_obj*)

Class for object type “npiConstant”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: `is_get_enum` (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

`signed()`

Get integer property “`npiParamSigned`”.

Parameters: None.

Returns: integer property of “`npiParamSigned`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`npiParamDecl`

`class lang.IOPDeclHandle(cps_obj)`

Class for object type “`npiParamDecl`”.

`set_permanent()`

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```



```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

direction(is_get_enum=False)

Get property “npIDirection”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIDirection”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
>>> hdl.direction(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

npiBitSelect

class lang.BitSelectHandle(cps_obj)

Class for object type “npiBitSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`constant_select()`

Get integer property “`npiParamConstantSelect`”.

Parameters: None.

Returns: integer property of “`npiParamConstantSelect`”.

Return type: int

Examples

```
>>> hdl.constant_select()
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParamName`”.

Parameters: None.

Returns: string property of “`npiParamName`”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParam”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParam”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npImplication

class lang.ImplicationHandle(*cps_obj*)

Class for object type “npImplication”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

constraint_expr_handles()

Get handle lists of the 1-m relationship “npiConstraintExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_expr_handles()
```

npiStructVar

class lang.StructVarHandle(*cps_obj*)

Class for object type “npiStructVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

packed_array_member()

Get integer property “npIPackedArrayMember”.

Parameters: None.

Returns: integer property of “npIPackedArrayMember”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string  
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npistructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabfullname”.

Parameters: None.

Returns: string property of “npielabfullname”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiepr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiepr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

member_handles()

Get handle lists of the 1-m relationship “npiMember”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMember”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.member_handles()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiStringTypespec

class lang.StringTypespecHandle(cps_obj)

Class for object type “npiStringTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python
```

```
import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiRelease

class lang.ReleaseHandle(cps_obj)

Class for object type “npiRelease”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```


upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiShortRealTypespec

class lang.ShortRealTypespecHandle(*cps_obj*)

Class for object type “npiShortRealTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`signed()`

Get integer property “`npiParamSigned`”.

Parameters: None.

Returns: integer property of “`npiParamSigned`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiParamFile`”.

Parameters: None.

Returns: string property of “`npiParamFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiFor

class lang.ForHandle(*cps_obj*)

Class for object type “npiFor”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
example.v:
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

local_var_decls()

Get integer property “npiParamDecls”.

Parameters: None.

Returns: integer property of “npiParamDecls”.

Return type: int

Examples

```
>>> hdl.local_var_decls()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```


scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

for_init_stmt_handles()

Get handle lists of the 1-m relationship “npiForInitStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiForInitStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.for_init_stmt_handles()
```

for_inc_stmt_handles()

Get handle lists of the 1-m relationship “npiForIncStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiForIncStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.for_inc_stmt_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npINamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiMpPortBit

class lang.MpPortBitHandle(*cps_obj*)

Class for object type “npiMpPortBit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```

`elab_name()`

Get string property “`npiElabName`”.

Parameters: None.

Returns: string property of “`npiElabName`”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParam”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParam”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

index_handles()

Get handle lists of the 1-m relationship “npIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npAliasStmt

class lang.AliasStmtHandle(*cps_obj*)

Class for object type “npAliasStmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

net_alias_handles()

Get handle lists of the 1-m relationship “npiNetAlias”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNetAlias”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.net_alias_handles()
```

npiProgram

class lang.ProgramHandle(*cps_obj*)

Class for object type “npiProgram”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

contain_specify_block()

Get integer property “npIContainSpecifyBlock”.

Parameters: None.

Returns: integer property of “npIContainSpecifyBlock”.

Return type: int

Examples

```
>>> hdl.contain_specify_block()
```

cell_compilation()

Get integer property “npiCellCompilation”.

Parameters: None.

Returns: integer property of “npiCellCompilation”.

Return type: int

Examples

```
>>> hdl.cell_compilation()
```

is_black_box()

Get integer property “npilsBlackBox”.

Parameters: None.

Returns: integer property of “npilsBlackBox”.

Return type: int

Examples

```
>>> hdl.is_black_box()
```

top()

Get integer property “npiTop”.

Parameters: None.

Returns: integer property of “npiTop”.

Return type: int

Examples

```
>>> hdl.top()
```

cell_instance()

Get integer property “npiCellInstance”.

Parameters: None.

Returns: integer property of “npiCellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

def_line_no()

Get integer property “npiDefLineNo”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

contain_protect()

Get integer property “npiContainProtect”.

Parameters: None.

Returns: integer property of “npiContainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

is_protected()

Get integer property “npilsProtected”.

Parameters: None.

Returns: integer property of “npilsProtected”.

Return type: int

Examples

```
>>> hdl.is_protected()
```

bind()

Get integer property “npiBind”.

Parameters: None.

Returns: integer property of “npiBind”.

Return type: int

Examples

```
>>> hdl.bind()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_file()

Get string property “npiDefFile”.

Parameters: None.

Returns: string property of “npiDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

library_name()

Get string property “npilibraryName”.

Parameters: None.

Returns: string property of “npilibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

def_library_name()

Get string property “npidefLibraryName”.

Parameters: None.

Returns: string property of “npidefLibraryName”.

Return type: string

Examples

```
>>> hdl.def_library_name()
```

time_scale()

Get string property “npitimeScale”.

Parameters: None.

Returns: string property of “npitimeScale”.

Return type: string

Examples

```
>>> hdl.time_scale()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```


name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

index_handle()

Get handle lists of the 1-1 relationship “npilIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npilIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npilInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

primitive_handles()

Get handle lists of the 1-m relationship “npiPrimitive”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPrimitive”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.primitive_handles()
```

alias_stmt_handles()

Get handle lists of the 1-m relationship “npiAliasStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAliasStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.alias_stmt_handles()
```

cont_assign_handles()

Get handle lists of the 1-m relationship “npiContAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiContAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cont_assign_handles()
```

port_handles()

Get handle lists of the 1-m relationship “npiPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npilODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npilODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

param_assign_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_assign_handles()
```

process_handles()

Get handle lists of the 1-m relationship “npiProcess”.

Parameters: None.

Returns: handle list of 1-m relationship “npiProcess”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.process_handles()
```

array_net_handles()

Get handle lists of the 1-m relationship “npiArrayNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArrayNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.array_net_handles()
```

task_func_handles()

Get handle lists of the 1-m relationship “npITaskFunc”.

Parameters: None.

Returns: handle list of 1-m relationship “npITaskFunc”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.task_func_handles()
```

import_handles()

Get handle lists of the 1-m relationship “npIImport”.

Parameters: None.

Returns: handle list of 1-m relationship “npIImport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.import_handles()
```

export_handles()

Get handle lists of the 1-m relationship “npIExport”.

Parameters: None.

Returns: handle list of 1-m relationship “npIExport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.export_handles()
```

def_param_handles()

Get handle lists of the 1-m relationship “npIDefParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npIDefParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.def_param_handles()
```

assertion_handles()

Get handle lists of the 1-m relationship “npiAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.assertion_handles()
```

covergroup_handles()

Get handle lists of the 1-m relationship “npiCovergroup”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCovergroup”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.covergroup_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiForever

class lang.ForeverHandle(*cps_obj*)

Class for object type “npiForever”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
hdl_list = ref.stmt_handle
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiParamPort

class lang.MiParamHandle(cps_obj)

Class for object type “npiParamPort”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of "npiType".

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

direction(is_get_enum=False)

Get property "npiDirection".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npDirection`”.

Return type: integer or string

Examples

```
>>> hdl.direction()      # Get string
>>> hdl.direction(True)  # Get integer
```

`line_no()`

Get integer property “`npLineNo`”.

Parameters: None.

Returns: integer property of “`npLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npName`”.

Parameters: None.

Returns: string property of “`npName`”.

Return type: string

Examples

```
>>> hdl.name()
```

`full_name()`

Get string property “`npFullName`”.

Parameters: None.

Returns: string property of “`npFullName`”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

modport_handle()

Get handle lists of the 1-1 relationship “npiModport”.

Parameters: None.

Returns: handle of 1-1 relationship “npiModport”.

Return type: handle

Examples

```
>>> hdl = ref.modport_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiAssume

class lang.AssumeHandle(*cps_obj*)

Class for object type “npiAssume”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

property_handle()

Get handle lists of the 1-1 relationship “npiProperty”.

Parameters: None.

Returns: handle of 1-1 relationship “npiProperty”.

Return type: handle

Examples

```
>>> hdl = ref.property_handle()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiPropertyDecl

class lang.PropertyDeclHandle(*cps_obj*)

Class for object type “npiPropertyDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

property_spec_handle()

Get handle lists of the 1-1 relationship “npiPropertySpec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPropertySpec”.

Return type: handle

Examples

```
>>> hdl = ref.property_spec_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```


variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

prop_formal_decl_handles()

Get handle lists of the 1-m relationship “npiPropFormalDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropFormalDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.prop_formal_decl_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npIPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npIPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npByteVar

class lang.ByteVarHandle(*cps_obj*)

Class for object type “npByteVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiautomatic”.

Parameters: None.

Returns: integer property of “npiautomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npirandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npirandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npiVisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples


```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiWait

class lang.WaitHandle(*cps_obj*)

Class for object type “npiWait”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiRange

class lang.RangeHandle(cps_obj)

Class for object type “npiRange”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`left_range_handle()`

Get handle lists of the 1-1 relationship “`npiLeftRange`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npiLeftRange`”.

Return type: handle

Examples

```
>>> hdl = ref.left_range_handle()
```

`right_range_handle()`

Get handle lists of the 1-1 relationship “`npiRightRange`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npiRightRange`”.

Return type: handle

Examples

```
>>> hdl = ref.right_range_handle()
```

`npiClassDefn`

`class lang.ClassDefnHandle(cps_obj)`

Class for object type “`npiClassDefn`”.

`set_permanent()`

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent

handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
```

```
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

virtual()

Get integer property “npivirtual”.

Parameters: None.

Returns: integer property of “npivirtual”.

Return type: int

Examples

```
>>> hdl.virtual()
```

interface_class()

Get integer property “npiinterfaceclass”.

Parameters: None.

Returns: integer property of “npInterfaceClass”.

Return type: int

Examples

```
>>> hdl.interface_class()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

extends_handle()

Get handle lists of the 1-1 relationship “npiExtends”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExtends”.

Return type: handle

Examples

```
>>> hdl = ref.extends_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

methods_handles()

Get handle lists of the 1-m relationship “npiMethods”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMethods”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.methods_handles()
```

implements_handles()

Get handle lists of the 1-m relationship “npiImplements”.

Parameters: None.

Returns: handle list of 1-m relationship “npiImplements”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.implements_handles()
```

derived_classes_handles()

Get handle lists of the 1-m relationship “npiDerivedClasses”.

Parameters: None.

Returns: handle list of 1-m relationship “npiDerivedClasses”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.derived_classes_handles()
```

constraint_handles()

Get handle lists of the 1-m relationship “npiConstraint”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraint”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_handles()
```

covergroup_handles()

Get handle lists of the 1-m relationship “npiCovergroup”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCovergroup”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.covergroup_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiforeachstmt

class lang.ForeachStmtHandle(cps_obj)

Class for object type “npiforeachstmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

function_handle()

Get handle lists of the 1-1 relationship “npiFunction”.

Parameters: None.

Returns: handle of 1-1 relationship “npiFunction”.

Return type: handle

Examples

```
>>> hdl = ref.function_handle()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

loop_vars_handles()

Get handle lists of the 1-m relationship “npiLoopVars”.

Parameters: None.

Returns: handle list of 1-m relationship “npiLoopVars”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.loop_vars_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npiParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiProgramArray

class lang.ProgramArrayHandle(cps_obj)

Class for object type “npiProgramArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiAssert

class lang.AssertHandle(cps_obj)

Class for object type “npiAssert”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```

obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabName”.

Parameters: None.

Returns: string property of “npielabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

else_stmt_handle()

Get handle lists of the 1-1 relationship “npiElseStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElseStmt”.

Return type: handle

Examples

```
>>> hdl = ref.else_stmt_handle()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

property_handle()

Get handle lists of the 1-1 relationship “npiProperty”.

Parameters: None.

Returns: handle of 1-1 relationship “npiProperty”.

Return type: handle

Examples

```
>>> hdl = ref.property_handle()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiAssignStmt

class lang.AssignStmtHandle(*cps_obj*)

Class for object type “npiAssignStmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```


example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiParamNo”.

Parameters: None.

Returns: integer property of “npiParamNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiParam”.

Return type: string

Examples

```
>>> hdl.file()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiParam”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

is_modport()

Get integer property “npilsModport”.

Parameters: None.

Returns: integer property of “npilsModport”.

Return type: int

Examples

```
>>> hdl.is_modport()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

def_name()

Get string property “npidefname”.

Parameters: None.

Returns: string property of “npidefname”.

Return type: string

Examples

```
>>> hdl.def_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npilElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

parent_handle()

Get handle lists of the 1-1 relationship “npilParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npilParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

param_assign_handles()

Get handle lists of the 1-m relationship “npilParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npilParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_assign_handles()
```

npilInterfaceTypespec

class lang.InterfaceTypespecHandle(*cps_obj*)

Class for object type “npilInterfaceTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

`type(is_get_enum=False)`

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

npitypeParameter

`class lang.TypeParameterHandle(cps_obj)`

Class for object type “npitypeParameter”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```



```

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

local_param()

Get integer property “npiLocalParam”.

Parameters: None.

Returns: integer property of “npiLocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiArrayVar

class lang.ArrayVarHandle(*cps_obj*)

Class for object type “npiArrayVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

array_type(is_get_enum=False)

Get property “npiArrayType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiArrayType”.

Return type: integer or string

Examples

```
>>> hdl.array_type() # Get string
>>> hdl.array_type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string  
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npistructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabName”.

Parameters: None.

Returns: string property of “npielabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npielabFullName”.

Parameters: None.

Returns: string property of “npielabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npEnumVar

class lang.EnumVarHandle(cps_obj)

Class for object type “npEnumVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

packed_array_member()

Get integer property “npnPackedArrayMember”.

Parameters: None.

Returns: integer property of “npnPackedArrayMember”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

automatic()

Get integer property “npIAutomatic”.

Parameters: None.

Returns: integer property of “npIAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npIRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string  
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npIVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiContinue

class lang.ContinueHandle(cps_obj)

Class for object type “npiContinue”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiSwitchArray

***class* lang.SwitchArrayHandle(*cps_obj*)**

Class for object type “npiSwitchArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type()      # Get string
>>> hdl.type(True)  # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```


name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiArrayNet

class lang.ArrayNetHandle(*cps_obj*)

Class for object type “npiArrayNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinNet => Success
Release object npinNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

net_type(is_get_enum=False)

Get property “npinNetType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinNetType”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npImplicitDecl”.

Parameters: None.

Returns: integer property of “npImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```


use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiEnumNet

class lang.EnumNetHandle(cps_obj)

Class for object type “npiEnumNet”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

packed_array_member()

Get integer property “npipackedArrayMember”.

Parameters: None.

Returns: integer property of “npipackedArrayMember”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

net_type(is_get_enum=False)

Get property “npinNetType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinNetType”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npImplicitDecl”.

Parameters: None.

Returns: integer property of “npImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

implicit_decl()

Get integer property “npImplicitDecl”.

Parameters: None.

Returns: integer property of “npImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```


use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiShortIntTypespec

***class* lang.ShortIntTypespecHandle(*cps_obj*)**

Class for object type “npiShortIntTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiUnionVar

class lang.UnionVarHandle(cps_obj)

Class for object type “npiUnionVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npIType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`packed_array_member()`

Get integer property “`npiPackedArrayMember`”.

Parameters: None.

Returns: integer property of “`npiPackedArrayMember`”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

`automatic()`

Get integer property “`npiAutomatic`”.

Parameters: None.

Returns: integer property of “`npiAutomatic`”.

Return type: int

Examples

```
>>> hdl.automatic()
```

`rand_type(is_get_enum=False)`

Get property “`npiRandType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiRandType`”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npiVisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples


```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

member_handles()

Get handle lists of the 1-m relationship “npiMember”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMember”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.member_handles()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiBegin

class lang.BeginHandle(cps_obj)

Class for object type “npiBegin”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handles()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```


named_event_handles()

Get handle lists of the 1-m relationship “npINamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

clocking_block_handles()

Get handle lists of the 1-m relationship “npIClockingBlock”.

Parameters: None.

Returns: handle list of 1-m relationship “npIClockingBlock”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.clocking_block_handles()
```

assertion_handles()

Get handle lists of the 1-m relationship “npIAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npIAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.assertion_handles()
```

covergroup_handles()

Get handle lists of the 1-m relationship “npICovergroup”.

Parameters: None.

Returns: handle list of 1-m relationship “npICovergroup”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.covergroup_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npIInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npIInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npINamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiGenVar

class lang.GenVarHandle(cps_obj)

Class for object type “npiGenVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())
```



```
# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiconstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiconstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()      # Get string
>>> hdl.const_type(True)  # Get integer
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

npiModule

class lang.ModuleHandle(cps_obj)

Class for object type “npiModule”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

contain_specify_block()

Get integer property “npicontainSpecifyBlock”.

Parameters: None.

Returns: integer property of “npiContainSpecifyBlock”.

Return type: int

Examples

```
>>> hdl.contain_specify_block()
```

cell_compilation()

Get integer property “npiCellCompilation”.

Parameters: None.

Returns: integer property of “npiCellCompilation”.

Return type: int

Examples

```
>>> hdl.cell_compilation()
```

is_black_box()

Get integer property “npilsBlackBox”.

Parameters: None.

Returns: integer property of “npilsBlackBox”.

Return type: int

Examples

```
>>> hdl.is_black_box()
```

top()

Get integer property “npiTop”.

Parameters: None.

Returns: integer property of “npiTop”.

Return type: int

Examples

```
>>> hdl.top()
```

cell_instance()

Get integer property “npiCellInstance”.

Parameters: None.

Returns: integer property of “npiCellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

def_line_no()

Get integer property “npiDefLineNo”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

contain_protect()

Get integer property “npiContainProtect”.

Parameters: None.

Returns: integer property of “npiContainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

is_protected()

Get integer property “npilsProtected”.

Parameters: None.

Returns: integer property of “npilsProtected”.

Return type: int

Examples

```
>>> hdl.is_protected()
```

bind()

Get integer property “npiBind”.

Parameters: None.

Returns: integer property of “npiBind”.

Return type: int

Examples

```
>>> hdl.bind()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

def_file()

Get string property “npDefFile”.

Parameters: None.

Returns: string property of “npDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

library_name()

Get string property “npLibraryName”.

Parameters: None.

Returns: string property of “npLibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

def_library_name()

Get string property “npDefLibraryName”.

Parameters: None.

Returns: string property of “npDefLibraryName”.

Return type: string

Examples

```
>>> hdl.def_library_name()
```

time_scale()

Get string property “npTimeScale”.

Parameters: None.

Returns: string property of “npiTimeScale”.

Return type: string

Examples

```
>>> hdl.time_scale()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npilElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npilElabFullName”.

Parameters: None.

Returns: string property of “npilElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

index_handle()

Get handle lists of the 1-1 relationship “npilIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npilIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npilInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npilScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

cont_assign_handles()

Get handle lists of the 1-m relationship “npiContAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiContAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cont_assign_handles()
```

port_handles()

Get handle lists of the 1-m relationship “npiPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npiIODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npilODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

param_assign_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.param_assign_handles()
```

primitive_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.primitive_handles()
```

process_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParamAssign”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.process_handles()
```

alias_stmt_handles()

Get handle lists of the 1-m relationship “npiParamAssign”.

Parameters: None.

Returns: handle list of 1-m relationship “npiAliasStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.alias_stmt_handles()
```

array_net_handles()

Get handle lists of the 1-m relationship “npiArrayNet”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArrayNet”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.array_net_handles()
```

task_func_handles()

Get handle lists of the 1-m relationship “npiTaskFunc”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTaskFunc”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.task_func_handles()
```

import_handles()

Get handle lists of the 1-m relationship “npilImport”.

Parameters: None.

Returns: handle list of 1-m relationship “npilImport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.import_handles()
```

export_handles()

Get handle lists of the 1-m relationship “npiExport”.

Parameters: None.

Returns: handle list of 1-m relationship “npiExport”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.export_handles()
```

def_param_handles()

Get handle lists of the 1-m relationship “npiDefParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npiDefParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.def_param_handles()
```

npilf

class lang.IfHandle(cps_obj)

Class for object type “npilf”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

Result:
```

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

qualifier(is_get_enum=False)

Get property “npiParamQualifier”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParamQualifier”.

Return type: integer or string

Examples

```
>>> hdl.qualifier()     # Get string
>>> hdl.qualifier(True) # Get integer
```

line_no()

Get integer property “npiParamLineNo”.

Parameters: None.

Returns: integer property of “npiParamLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```


file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-1 relationship “npiStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiStmt”.

Return type: handle

Examples

```
>>> hdl = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiUdpArray

class lang.UdpArrayHandle(cps_obj)

Class for object type “npiUdpArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`size()`

Get integer property “`npiParamSize`”.

Parameters: None.

Returns: integer property of “`npiParamSize`”.

Return type: int

Examples

```
>>> hdl.size()
```

`name()`

Get string property “`npiParamName`”.

Parameters: None.

Returns: string property of “`npiParamName`”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiDelayControl

class lang.DelayControlHandle(cps_obj)

Class for object type “npiDelayControl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```


stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

delay_handle()

Get handle lists of the 1-1 relationship “npiDelay”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelay”.

Return type: handle

Examples

```
>>> hdl = ref.delay_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiFinal

class lang.FinalHandle(cps_obj)

Class for object type “npiFinal”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiClassVar

class lang.ClassVarHandle(cps_obj)

Class for object type “npiClassVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

rand_type(is_get_enum=False)

Get property “npRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```


class_obj_handle()

Get handle lists of the 1-1 relationship “npiClassObj”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClassObj”.

Return type: handle

Examples

```
>>> hdl = ref.class_obj_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

npiOperation

***class* lang.OperationHandle(*cps_obj*)**

Class for object type “npiOperation”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

op_type(is_get_enum=False)

Get property “npiOpType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiOpType”.

Return type: integer or string

Examples

```
>>> hdl.op_type() # Get string
>>> hdl.op_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default (int)** – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npitypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npitypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

operand_handles()

Get handle lists of the 1-m relationship “npioperand”.

Parameters: None.

Returns: handle list of 1-m relationship “npioperand”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.operand_handles()
```

npiGate

class lang.GateHandle(cps_obj)

Class for object type “npiGate”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```



```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

prim_type(is_get_enum=False)

Get property “npiPrimType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiPrimType”.

Return type: integer or string

Examples

```
>>> hdl.prim_type()      # Get string
>>> hdl.prim_type(True)  # Get integer
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npiDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

delay_handle()

Get handle lists of the 1-1 relationship “npiDelay”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelay”.

Return type: handle

Examples

```
>>> hdl = ref.delay_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npilInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

prim_term_handles()

Get handle lists of the 1-m relationship “npiPrimTerm”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPrimTerm”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.prim_term_handles()
```

npilNamedEventArray

class lang.NamedEventArrayHandle(*cps_obj*)

Class for object type “npilNamedEventArray”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

automatic()

Get integer property “npAutomatic”.

Parameters: None.

Returns: integer property of “npAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiIntegerVar

class lang.IntegerVarHandle(*cps_obj*)

Class for object type “npiIntegerVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npivisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npIDistribution

class lang.DistributionHandle(*cps_obj*)

Class for object type “npIDistribution”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

dist_item_handles()

Get handle lists of the 1-m relationship “npiDistItem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiDistItem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.dist_item_handles()
```

npilnitial

class lang.InitialHandle(cps_obj)

Class for object type “npilnitial”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiParamAssign

class lang.ParamAssignHandle(cps_obj)

Class for object type “npiParamAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiLongIntVar

class lang.LongIntVarHandle(cps_obj)

Class for object type “npiLongIntVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```

obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiAutomatic”.

Parameters: None.

Returns: integer property of “npiAutomatic”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiRandType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiRandType”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(is_get_enum=False)

Get property “npiVisibility”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npSize”.

Parameters: None.

Returns: integer property of “npSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npArrayMember”.

Parameters: None.

Returns: integer property of “npArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npStructUnionMember”.

Parameters: None.

Returns: integer property of “npStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

function_handle()

Get handle lists of the 1-1 relationship “npiFunction”.

Parameters: None.

Returns: handle of 1-1 relationship “npiFunction”.

Return type: handle

Examples

```
>>> hdl = ref.function_handle()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```


index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npilUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npilUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npilRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npilRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npilTypespecMember

class lang.TypespecMemberHandle(*cps_obj*)

Class for object type “npilTypespecMember”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

npiParameterBit

class lang.ParameterBitHandle(cps_obj)

Class for object type “npiParameterBit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())
```

```
# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

local_param()

Get integer property “npilocalParam”.

Parameters: None.

Returns: integer property of “npilocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type()          # Get string
>>> hdl.const_type(True)     # Get integer
```

set_value_by_option()

Get integer property “npiSetValueByOption”.

Parameters: None.

Returns: integer property of “npiSetValueByOption”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

constraint_item_handles()

Get handle lists of the 1-m relationship “npiConstraintItem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintItem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_item_handles()
```

npiNamedEvent

class lang.NamedEventHandle(*cps_obj*)

Class for object type “npiNamedEvent”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`automatic()`

Get integer property “`npiAutomatic`”.

Parameters: None.

Returns: integer property of “`npiAutomatic`”.

Return type: int

Examples

```
>>> hdl.automatic()
```

`array_member()`

Get integer property “`npiArrayMember`”.

Parameters: None.

Returns: integer property of “`npiArrayMember`”.

Return type: int

Examples

```
>>> hdl.array_member()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```


elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

npiConstraint

class lang.ConstraintHandle(cps_obj)

Class for object type “npiConstraint”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`virtual()`

Get integer property “`npiVirtual`”.

Parameters: None.

Returns: integer property of “`npiVirtual`”.

Return type: int

Examples

```
>>> hdl.virtual()
```

`automatic()`

Get integer property “`npiAutomatic`”.

Parameters: None.

Returns: integer property of “`npiAutomatic`”.

Return type: int

Examples

```
>>> hdl.automatic()
```

extern_line_no()

Get integer property “npExternLineNo”.

Parameters: None.

Returns: integer property of “npExternLineNo”.

Return type: int

Examples

```
>>> hdl.extern_line_no()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npFile”.

Parameters: None.

Returns: string property of “npFile”.

Return type: string

Examples

```
>>> hdl.file()
```

extern_file()

Get string property “npiExternFile”.

Parameters: None.

Returns: string property of “npiExternFile”.

Return type: string

Examples

```
>>> hdl.extern_file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

constraint_item_handles()

Get handle lists of the 1-m relationship “npiConstraintItem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintItem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_item_handles()
```

npiSwitch

class lang.SwitchHandle(*cps_obj*)

Class for object type “npiSwitch”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npIType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npIType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`prim_type(is_get_enum=False)`

Get property “`npIPrimType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npIPrimType`”.

Return type: integer or string

Examples

```
>>> hdl.prim_type()      # Get string
>>> hdl.prim_type(True)  # Get integer
```

`array_member()`

Get integer property “`npIArrayMember`”.

Parameters: None.

Returns: integer property of “`npIArrayMember`”.

Return type: int

Examples

```
>>> hdl.array_member()
```

`line_no()`

Get integer property “`npILineNo`”.

Parameters: None.

Returns: integer property of “`npILineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```


size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npiDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

delay_handle()

Get handle lists of the 1-1 relationship “npiDelay”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelay”.

Return type: handle

Examples

```
>>> hdl = ref.delay_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npilIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npilIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

instance_array_handle()

Get handle lists of the 1-1 relationship “npilInstanceArray”.

Parameters: None.

Returns: handle of 1-1 relationship “npilInstanceArray”.

Return type: handle

Examples

```
>>> hdl = ref.instance_array_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

prim_term_handles()

Get handle lists of the 1-m relationship “npiPrimTerm”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPrimTerm”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.prim_term_handles()
```

npiParamPattern

class lang.MemberPatternHandle(cps_obj)

Class for object type “npiParamPattern”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiforce

class lang.ForceHandle(cps_obj)

Class for object type “npiforce”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiRefObj

class lang.RefObjHandle(cps_obj)

Class for object type “npiRefObj”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success  
Release object npiParam => Failure  
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

generic()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.generic()
```

line_no()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

def_name()

Get string property “npidefname”.

Parameters: None.

Returns: string property of “npidefname”.

Return type: string

Examples

```
>>> hdl.def_name()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

actual_handle()

Get handle lists of the 1-1 relationship “npiActual”.

Parameters: None.

Returns: handle of 1-1 relationship “npiActual”.

Return type: handle

Examples

```
>>> hdl = ref.actual_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

use_handle()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handle()
```

npiReg

class lang.RegHandle(cps_obj)

Class for object type “npiReg”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```



```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

automatic()

Get integer property “npiParam”.

Parameters: None.

Returns: integer property of “npiParam”.

Return type: int

Examples

```
>>> hdl.automatic()
```

rand_type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

visibility(*is_get_enum=False*)

Get property “npivisibility”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivisibility”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npisArrayMember”.

Parameters: None.

Returns: integer property of “npisArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npistructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiWhile

class lang.WhileHandle(cps_obj)

Class for object type “npiWhile”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

`example.v`:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

`example.py`:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
hdl_list = ref.stmt_handle
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiSequenceDecl

class lang.SequenceDeclHandle(cps_obj)

Class for object type “npiSequenceDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
example.v:  
module top;  
    wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiParamNo”.

Parameters: None.

Returns: integer property of “npiParamNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiParam”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiParamFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

seq_formal_decl_handles()

Get handle lists of the 1-m relationship “npiSeqFormalDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSeqFormalDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.seq_formal_decl_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npiInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npiNamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiNamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npiNamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npIParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npIParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npIVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npIVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npITypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npITypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npIPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiConstrIf

class lang.ConstrIfHandle(*cps_obj*)

Class for object type “npiConstrIf”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)      # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

constraint_expr_handles()

Get handle lists of the 1-m relationship “npiConstraintExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConstraintExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.constraint_expr_handles()
```

npiLongIntTypespec

class lang.LongIntTypespecHandle(*cps_obj*)

Class for object type “npiLongIntTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npilImportItem

class lang.ImportItemHandle(cps_obj)

Class for object type “npilImportItem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```



```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

`file()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

npiVarSelect

class lang.VarSelectHandle(cps_obj)

Class for object type “npiVarSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`size()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.size()
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiModport

class lang.ModportHandle(*cps_obj*)

Class for object type “npiModport”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```



```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

`file()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npiIODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

mp_port_handles()

Get handle lists of the 1-m relationship “npiMpPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiMpPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.mp_port_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiTimeVar

class lang.TimeVarHandle(*cps_obj*)

Class for object type “npiTimeVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`automatic()`

Get integer property “`npiAutomatic`”.

Parameters: None.

Returns: integer property of “`npiAutomatic`”.

Return type: int

Examples

```
>>> hdl.automatic()
```

`rand_type(is_get_enum=False)`

Get property “`npiRandType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiRandType`”.

Return type: integer or string

Examples

```
>>> hdl.rand_type() # Get string
>>> hdl.rand_type(True) # Get integer
```

`visibility(is_get_enum=False)`

Get property “`npiVisibility`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiVisibility`”.

Return type: integer or string

Examples

```
>>> hdl.visibility() # Get string  
>>> hdl.visibility(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples


```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npiIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiLogicTypespec

class lang.LogicTypespecHandle(cps_obj)

Class for object type “npiLogicTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vector()

Get integer property “npivector”.

Parameters: None.

Returns: integer property of “npivector”.

Return type: int

Examples

```
>>> hdl.vector()
```

signed()

Get integer property “npisigned”.

Parameters: None.

Returns: integer property of “npisigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elem_typespec_handle()

Get handle lists of the 1-1 relationship “npiElemTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElemTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.elem_typespec_handle()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npiRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npiPackedArrayParameter

class lang.PackedArrayParameterHandle(cps_obj)

Class for object type “npiPackedArrayParameter”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

local_param()

Get integer property “npiLocalParam”.

Parameters: None.

Returns: integer property of “npiLocalParam”.

Return type: int

Examples

```
>>> hdl.local_param()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npConstType`”.

Return type: integer or string

Examples

```
>>> hdl.const_type()          # Get string
>>> hdl.const_type(True)     # Get integer
```

`set_value_by_option()`

Get integer property “`npSetValueByOption`”.

Parameters: None.

Returns: integer property of “`npSetValueByOption`”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

`name()`

Get string property “`npName`”.

Parameters: None.

Returns: string property of “`npName`”.

Return type: string

Examples

```
>>> hdl.name()
```

`full_name()`

Get string property “`npFullName`”.

Parameters: None.

Returns: string property of “`npFullName`”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

index_handle()

Get handle lists of the 1-1 relationship “npiIndex”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIndex”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiSeqFormalDecl

class lang.SeqFormalDeclHandle(*cps_obj*)

Class for object type “npiSeqFormalDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```

from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`direction(is_get_enum=False)`

Get property “`npiDirection`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiDirection`”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
>>> hdl.direction(True) # Get integer
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```


file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiEventControl

class lang.EventControlHandle(cps_obj)

Class for object type “npiEventControl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

condition_handle()

Get handle lists of the 1-1 relationship “npiCondition”.

Parameters: None.

Returns: handle of 1-1 relationship “npiCondition”.

Return type: handle

Examples

```
>>> hdl = ref.condition_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiEventStmt

class lang.EventStmtHandle(cps_obj)

Class for object type “npiEventStmt”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```

```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiParamNo”.

Parameters: None.

Returns: integer property of “npiParamNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiParam”.

Parameters: None.

Returns: string property of “npiParam”.

Return type: string

Examples

```
>>> hdl.file()
```

named_event_handle()

Get handle lists of the 1-1 relationship “npiParamNamedEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiNamedEvent”.

Return type: handle

Examples

```
>>> hdl = ref.named_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

npiPartSelect

class lang.PartSelectHandle(cps_obj)

Class for object type “npiPartSelect”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

left_range_handle()

Get handle lists of the 1-1 relationship “npiLeftRange”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLeftRange”.

Return type: handle

Examples

```
>>> hdl = ref.left_range_handle()
```

right_range_handle()

Get handle lists of the 1-1 relationship “npiRightRange”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRightRange”.

Return type: handle

Examples

```
>>> hdl = ref.right_range_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npiClockingIODecl

class lang.ClockingIODeclHandle(cps_obj)

Class for object type “npiClockingIODecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

direction(is_get_enum=False)

Get property “npidirection”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npidirection”.

Return type: integer or string

Examples

```
>>> hdl.direction()      # Get string  
>>> hdl.direction(True)  # Get integer
```

input_edge()

Get integer property “npInputEdge”.

Parameters: None.

Returns: integer property of “npInputEdge”.

Return type: int

Examples

```
>>> hdl.input_edge()
```

output_edge()

Get integer property “npOutputEdge”.

Parameters: None.

Returns: integer property of “npOutputEdge”.

Return type: int

Examples

```
>>> hdl.output_edge()
```

name()

Get string property “npName”.

Parameters: None.

Returns: string property of “npName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npFullName”.

Parameters: None.

Returns: string property of “npFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

input_skew_handle()

Get handle lists of the 1-1 relationship “npiInputSkew”.

Parameters: None.

Returns: handle of 1-1 relationship “npiInputSkew”.

Return type: handle

Examples

```
>>> hdl = ref.input_skew_handle()
```

output_skew_handle()

Get handle lists of the 1-1 relationship “npiOutputSkew”.

Parameters: None.

Returns: handle of 1-1 relationship “npiOutputSkew”.

Return type: handle

Examples


```
>>> hdl = ref.output_skew_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiCHandleTypespec

class lang.ChandleTypespecHandle(*cps_obj*)

Class for object type “npiCHandleTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
```

```

sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParam`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`signed()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`line_no()`

Get integer property “`npiParam`”.

Parameters: None.

Returns: integer property of “`npiParam`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiParam`”.

Parameters: None.

Returns: string property of “`npiParam`”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiParamFuncCall

class lang.MethodFuncCallHandle(cps_obj)

Class for object type “npiParamFuncCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())
```

```
# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

is_expr()

Get integer property “npilsExpr”.

Parameters: None.

Returns: integer property of “npilsExpr”.

Return type: int

Examples

```
>>> hdl.is_expr()
```

const_type(is_get_enum=False)

Get property “npConstType”.

Parameters: **is_get_enum** (bool) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```


name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

class_obj_handle()

Get handle lists of the 1-1 relationship “npiClassObj”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClassObj”.

Return type: handle

Examples

```
>>> hdl = ref.class_obj_handle()
```

function_handle()

Get handle lists of the 1-1 relationship “npiFunction”.

Parameters: None.

Returns: handle of 1-1 relationship “npiFunction”.

Return type: handle

Examples

```
>>> hdl = ref.function_handle()
```

prefix_handle()

Get handle lists of the 1-1 relationship “npiPrefix”.

Parameters: None.

Returns: handle of 1-1 relationship “npiPrefix”.

Return type: handle

Examples

```
>>> hdl = ref.prefix_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

argument_handles()

Get handle lists of the 1-m relationship “npiArgument”.

Parameters: None.

Returns: handle list of 1-m relationship “npiArgument”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.argument_handles()
```

npiPositionPattern

class lang.PositionPatternHandle(*cps_obj*)

Class for object type “npiPositionPattern”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```

from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiParamFile`”.

Parameters: None.

Returns: string property of “`npiParamFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`index_handle()`

Get handle lists of the 1-1 relationship “`npiParamIndex`”.

Parameters: None.

Returns: handle of 1-1 relationship “`npiParamIndex`”.

Return type: handle

Examples

```
>>> hdl = ref.index_handle()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiTask

class lang.TaskHandle(*cps_obj*)

Class for object type “npiTask”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npIType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`automatic()`

Get integer property “`npiAutomatic`”.

Parameters: None.

Returns: integer property of “`npiAutomatic`”.

Return type: int

Examples

```
>>> hdl.automatic()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```


full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.upper_stmt_handle()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npiIODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiIODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

internal_scope_handles()

Get handle lists of the 1-m relationship “npInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.internal_scope_handles()
```

named_event_handles()

Get handle lists of the 1-m relationship “npINamedEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_handles()
```

named_event_array_handles()

Get handle lists of the 1-m relationship “npINamedEventArray”.

Parameters: None.

Returns: handle list of 1-m relationship “npINamedEventArray”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.named_event_array_handles()
```

parameter_handles()

Get handle lists of the 1-m relationship “npParameter”.

Parameters: None.

Returns: handle list of 1-m relationship “npParameter”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.parameter_handles()
```

variables_handles()

Get handle lists of the 1-m relationship “npiVariables”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVariables”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.variables_handles()
```

typedef_handles()

Get handle lists of the 1-m relationship “npiTypedef”.

Parameters: None.

Returns: handle list of 1-m relationship “npiTypedef”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.typedef_handles()
```

property_decl_handles()

Get handle lists of the 1-m relationship “npiPropertyDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPropertyDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.property_decl_handles()
```

sequence_decl_handles()

Get handle lists of the 1-m relationship “npiSequenceDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiSequenceDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sequence_decl_handles()
```

concurrent_assertion_handles()

Get handle lists of the 1-m relationship “npiConcurrentAssertion”.

Parameters: None.

Returns: handle list of 1-m relationship “npiConcurrentAssertion”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.concurrent_assertion_handles()
```

npiBitTypespec

class lang.BitTypespecHandle(*cps_obj*)

Class for object type “npiBitTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vector()`

Get integer property “`npiVector`”.

Parameters: None.

Returns: integer property of “`npiVector`”.

Return type: int

Examples

```
>>> hdl.vector()
```

`signed()`

Get integer property “`npiSigned`”.

Parameters: None.

Returns: integer property of “`npiSigned`”.

Return type: int

Examples

```
>>> hdl.signed()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elem_typespec_handle()

Get handle lists of the 1-1 relationship “npiElemTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiElemTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.elem_typespec_handle()
```


typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTtypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTtypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npIScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npIScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

range_handles()

Get handle lists of the 1-m relationship “npIRange”.

Parameters: None.

Returns: handle list of 1-m relationship “npIRange”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.range_handles()
```

npIPackedArrayNet

class lang.PackedArrayNetHandle(*cps_obj*)

Class for object type “npIPackedArrayNet”.

set_persistent()

Set the handle as ‘persistent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

packed_array_member()

Get integer property “npIPackedArrayMember”.

Parameters: None.

Returns: integer property of “npIPackedArrayMember”.

Return type: int

Examples

```
>>> hdl.packed_array_member()
```

net_type(is_get_enum=False)

Get property “npiNetType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiNetType”.

Return type: integer or string

Examples

```
>>> hdl.net_type() # Get string
>>> hdl.net_type(True) # Get integer
```

implicit_decl()

Get integer property “npiImplicitDecl”.

Parameters: None.

Returns: integer property of “npiImplicitDecl”.

Return type: int

Examples

```
>>> hdl.implicit_decl()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

array_member()

Get integer property “npiArrayMember”.

Parameters: None.

Returns: integer property of “npiArrayMember”.

Return type: int

Examples

```
>>> hdl.array_member()
```

struct_union_member()

Get integer property “npiStructUnionMember”.

Parameters: None.

Returns: integer property of “npiStructUnionMember”.

Return type: int

Examples

```
>>> hdl.struct_union_member()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

elab_full_name()

Get string property “npiElabFullName”.

Parameters: None.

Returns: string property of “npiElabFullName”.

Return type: string

Examples

```
>>> hdl.elab_full_name()
```

typespec_handle()

Get handle lists of the 1-1 relationship “npiTypespec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypespec”.

Return type: handle

Examples

```
>>> hdl = ref.typespec_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

ports_handles()

Get handle lists of the 1-m relationship “npiPorts”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPorts”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ports_handles()
```

port_inst_handles()

Get handle lists of the 1-m relationship “npiPortInst”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPortInst”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_inst_handles()
```

index_handles()

Get handle lists of the 1-m relationship “npilIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npilIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.index_handles()
```

use_handles()

Get handle lists of the 1-m relationship “npiUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.use_handles()
```

ref_obj_handles()

Get handle lists of the 1-m relationship “npiRefObj”.

Parameters: None.

Returns: handle list of 1-m relationship “npiRefObj”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.ref_obj_handles()
```

npirealTypespec

class lang.RealTypespecHandle(*cps_obj*)

Class for object type “npirealTypespec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

signed()

Get integer property “npISigned”.

Parameters: None.

Returns: integer property of “npISigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

signed()

Get integer property “npiSigned”.

Parameters: None.

Returns: integer property of “npiSigned”.

Return type: int

Examples

```
>>> hdl.signed()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

typedef_alias_handle()

Get handle lists of the 1-1 relationship “npiTypedefAlias”.

Parameters: None.

Returns: handle of 1-1 relationship “npiTypedefAlias”.

Return type: handle

Examples

```
>>> hdl = ref.typedef_alias_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiCover

class lang.CoverHandle(*cps_obj*)

Class for object type “npiCover”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

is_cover_sequence()

Get integer property “npIlsCoverSequence”.

Parameters: None.

Returns: integer property of “npIlsCoverSequence”.

Return type: int

Examples

```
>>> hdl.is_cover_sequence()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

stmt_handle()

Get handle lists of the 1-m relationship “npiStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.stmt_handle()
```

property_handle()

Get handle lists of the 1-1 relationship “npiProperty”.

Parameters: None.

Returns: handle of 1-1 relationship “npiProperty”.

Return type: handle

Examples

```
>>> hdl = ref.property_handle()
```

clocking_event_handle()

Get handle lists of the 1-1 relationship “npiClockingEvent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiClockingEvent”.

Return type: handle

Examples

```
>>> hdl = ref.clocking_event_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiDefParam

class lang.DefParamHandle(cps_obj)

Class for object type “npiDefParam”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer

```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

npiContAssign

class lang.ContAssignHandle(cps_obj)

Class for object type “npiContAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```

obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()      # Get string  
>>> hdl.type(True)  # Get integer
```

net_decl_assign()

Get integer property “npiNetDeclAssign”.

Parameters: None.

Returns: integer property of “npiNetDeclAssign”.

Return type: int

Examples

```
>>> hdl.net_decl_assign()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

delay_handle()

Get handle lists of the 1-1 relationship “npiDelay”.

Parameters: None.

Returns: handle of 1-1 relationship “npiDelay”.

Return type: handle

Examples

```
>>> hdl = ref.delay_handle()
```

lhs_handle()

Get handle lists of the 1-1 relationship “npiLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiLhs”.

Return type: handle

Examples

```
>>> hdl = ref.lhs_handle()
```

rhs_handle()

Get handle lists of the 1-1 relationship “npiRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiRhs”.

Return type: handle

Examples

```
>>> hdl = ref.rhs_handle()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiScope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

npiCovergroup

class lang.CovergroupHandle(cps_obj)

Class for object type “npiCovergroup”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```



```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npielabname”.

Parameters: None.

Returns: string property of “npielabname”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiscope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiscope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

i_o_decl_handles()

Get handle lists of the 1-m relationship “npilODecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npilODecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.i_o_decl_handles()
```

coverage_event_handles()

Get handle lists of the 1-m relationship “npiCoverageEvent”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCoverageEvent”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.coverage_event_handles()
```

cover_point_handles()

Get handle lists of the 1-m relationship “npiCoverPoint”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCoverPoint”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cover_point_handles()
```

cover_cross_handles()

Get handle lists of the 1-m relationship “npiCoverCross”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCoverCross”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cover_cross_handles()
```

coverage_option_handles()

Get handle lists of the 1-m relationship “npiCoverageOption”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCoverageOption”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.coverage_option_handles()
```

npiCoverageOption

class lang.CoverageOptionHandle(cps_obj)

Class for object type “npiCoverageOption”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
```

```

sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npiType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`covopt_type(is_get_enum=False)`

Get property “`npiCovoptType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiCovoptType`”.

Return type: integer or string

Examples

```
>>> hdl.covopt_type()      # Get string
>>> hdl.covopt_type(True)  # Get integer
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

npiCoverPoint

class lang.CoverPointHandle(*cps_obj*)

Class for object type “npiCoverPoint”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

expr_handle()

Get handle lists of the 1-1 relationship “npiExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiExpr”.

Return type: handle

Examples

```
>>> hdl = ref.expr_handle()
```

iff_handle()

Get handle lists of the 1-1 relationship “npilff”.

Parameters: None.

Returns: handle of 1-1 relationship “npilff”.

Return type: handle

Examples

```
>>> hdl = ref.iff_handle()
```

bins_handles()

Get handle lists of the 1-m relationship “npIBins”.

Parameters: None.

Returns: handle list of 1-m relationship “npIBins”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.bins_handles()
```

coverage_option_handles()

Get handle lists of the 1-m relationship “npICoverageOption”.

Parameters: None.

Returns: handle list of 1-m relationship “npICoverageOption”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.coverage_option_handles()
```

npICoverCross

class lang.CoverCrossHandle(*cps_obj*)

Class for object type “npICoverCross”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

iff_handle()

Get handle lists of the 1-1 relationship “npiIff”.

Parameters: None.

Returns: handle of 1-1 relationship “npiIff”.

Return type: handle

Examples

```
>>> hdl = ref.iff_handle()
```

cross_item_handles()

Get handle lists of the 1-m relationship “npiCrossItem”.

Parameters: None.

Returns: handle list of 1-m relationship “npiCrossItem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.cross_item_handles()
```

npiBins

class lang.BinsHandle(*cps_obj*)

Class for object type “npiBins”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```

from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npIType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type()          # Get string
>>> hdl.type(True)     # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`bins_type(is_get_enum=False)`

Get property “`npiBinsType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiBinsType`”.

Return type: integer or string

Examples

```
>>> hdl.bins_type()      # Get string
>>> hdl.bins_type(True)  # Get integer
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```

elab_name()

Get string property “npiElabName”.

Parameters: None.

Returns: string property of “npiElabName”.

Return type: string

Examples

```
>>> hdl.elab_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

bins_spec_handle()

Get handle lists of the 1-1 relationship “npiBinsSpec”.

Parameters: None.

Returns: handle of 1-1 relationship “npiBinsSpec”.

Return type: handle

Examples

```
>>> hdl = ref.bins_spec_handle()
```

VHDL Objects

The following table describes the Object Category and their corresponding object types or tags with pyDoc link:

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Vh Instance	(container)	(class)

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Vh Root	vh root	npiVhRoot
Vh Component, Vh Component Declaration	vh component, vh comp decl	npiVhComponent , npiVhCompDecl
Vh Package	vh package	npiVhPackage
Vh Design Unit	vh design unit	(class)
Vh Map Element	vh map elem	npiVhMapElem
Vh Scope	vh scope	(class)
Vh Block	vh block	npiVhBlock
Vh Generate, Vh Index Var	vh generate, vh index var	npiVhGenerate , npiVhIndexVar
Vh Subprogram, Vh Subprogram Declaration	vh subprogram, vh subp decl	(class)
Vh Name	vh name	(class)
Vh Object Declaration	Vh obj decl	(class)
Vh Statement	vh stmt	(class)
Vh Signal Assignment	vh sig assign	(class)
Vh Variable Assignment	vh var assign	npiVhVarAssign
Vh Waveform	vh waveform	(class)
Vh Subprogram Call	vh subp call	(class)
Vh Assertion	vh assert	(class)
Vh Process	vh process	npiVhProcess
Vh Loop	vh loop	(class)
Vh Wait	vh wait	npiVhWait
Vh If, Vh Case	vh if, vh case	npiVhIf , npiVhCase
Vh Branch	vh branch	npiVhBranch
Vh Return	vh return	npiVhReturn
Vh Exit, Vh Next	vh exit, vh next	npiVhExit , npiVhNext

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Vh Literal	vh literal	(class)
Vh Physical Unit	vh physical unit	npiVhPhysUnit
Vh Parameter	vh parameter	(class)
Vh Range	vh range	npiVhRange
Vh Type	vh type	(class)
Vh Scalar Type Declaration, Vh Constraint	vh scalar type decl, vh constraint	(class)
Vh Composite Type Declaration	Vh composite type decl	(class)
Vh Attribute Name, Vh Entity Designator	vh attr name, vh entity designator	(class)
Vh Attribute Declaration, Vh Attribute Specification	vh attr decl, vh attr spec	npiVhAttrDecl , npiVhAttrSpec
Vh Library Declaration	vh library decl npiVhLibraryDecl	npiVhLibraryDecl
Vh Use Clause	vh use clause	npiVhUseClause
Vh Expression	vh expr	(class)
Vh Declaration	vh decl	(class)
Vh Aggregate, Vh Elem Assoc	vh aggregate, vh elem assoc	npiVhAggregate , npiVhElemAssoc
Vh Type Conv, Vh Qualified Expr	vh type conv, vh qualified expr	npiVhTypeConv , npiVhQualifiedExpr
Vh Use	vh use	(class)

For more information about VHDL Object Category, see the “Object Diagrams” chapter in the *VC Apps Native Programming Interface (NPI) Guide*.

npiVhConst

class lang.VhConstHandle(cps_obj)

Class for object type “npiVhConst”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```

```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and

expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npivhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhProcedure

class lang.VhProcedureHandle(*cps_obj*)

Class for object type “npivhProcedure”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_overload()

Get integer property “npiVarOverload”.

Parameters: None.

Returns: integer property of “npiVarOverload”.

Return type: int

Examples

```
>>> hdl.vh_overload()
```

line_no()

Get integer property “npiVarLineNo”.

Parameters: None.

Returns: integer property of “npiVarLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiVarName”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_subp_decl_handle()

Get handle lists of the 1-1 relationship “npivhsubpdecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhsubpdecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_decl_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhscope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npivhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npivhOperation

class lang.VhOperationHandle(cps_obj)

Class for object type “npivhOperation”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```


Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_op_type(is_get_enum=False)

Get property “npivhOpType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivhOpType”.

Return type: integer or string

Examples

```
>>> hdl.vh_op_type() # Get string
>>> hdl.vh_op_type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property "npiFile".

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_operand_handles()

Get handle lists of the 1-m relationship “npiVhOperand”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhOperand”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_operand_handles()
```

npiVhGeneric

class lang.VhGenericHandle(cps_obj)

Class for object type “npiVhGeneric”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(*is_get_enum=False*)

Get property “npiType”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

set_value_by_option()

Get integer property “npiSetValueByOption”.

Parameters: None.

Returns: integer property of “npiSetValueByOption”.

Return type: int

Examples

```
>>> hdl.set_value_by_option()
```

value(*is_get_default=False*)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npivhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhSimpAttrName

class lang.VhSimpAttrNameHandle(cps_obj)

Class for object type “npivhSimpAttrName”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_predef_attr_type(is_get_enum=False)

Get property “npivhPredefAttrType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivhPredefAttrType”.

Return type: integer or string

Examples

```
>>> hdl.vh_predef_attr_type() # Get string
>>> hdl.vh_predef_attr_type(True) # Get integer
```

vh_attr_kind_type(is_get_enum=False)

Get property “npivhAttrKindType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npivhAttrKindType”.

Return type: integer or string

Examples

```
>>> hdl.vh_attr_kind_type() # Get string
>>> hdl.vh_attr_kind_type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

npiVhRoot

class lang.VhRootHandle(*cps_obj*)

Class for object type “npiVhRoot”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_entity_line_no()

Get integer property “npIVhEntityLineNo”.

Parameters: None.

Returns: integer property of “npIVhEntityLineNo”.

Return type: int

Examples

```
>>> hdl.vh_entity_line_no()
```

vh_arch_line_no()

Get integer property “npivhArchLineNo”.

Parameters: None.

Returns: integer property of “npivhArchLineNo”.

Return type: int

Examples

```
>>> hdl.vh_arch_line_no()
```

cell_instance()

Get integer property “npicellInstance”.

Parameters: None.

Returns: integer property of “npicellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

contain_protect()

Get integer property “npicontainProtect”.

Parameters: None.

Returns: integer property of “npicontainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_entity_name()

Get string property “npivhEntityName”.

Parameters: None.

Returns: string property of “npivhEntityName”.

Return type: string

Examples

```
>>> hdl.vh_entity_name()
```

vh_arch_name()

Get string property “npivhArchName”.

Parameters: None.

Returns: string property of “npivhArchName”.

Return type: string

Examples

```
>>> hdl.vh_arch_name()
```

vh_entity_file()

Get string property “npivhEntityFile”.

Parameters: None.

Returns: string property of “npivhEntityFile”.

Return type: string

Examples

```
>>> hdl.vh_entity_file()
```

vh_arch_file()

Get string property “npivhArchFile”.

Parameters: None.

Returns: string property of “npivhArchFile”.

Return type: string

Examples

```
>>> hdl.vh_arch_file()
```


vh_entity_file()

Get string property “npivhEntityFile”.

Parameters: None.

Returns: string property of “npivhEntityFile”.

Return type: string

Examples

```
>>> hdl.vh_entity_file()
```

vh_arch_file()

Get string property “npivhArchFile”.

Parameters: None.

Returns: string property of “npivhArchFile”.

Return type: string

Examples

```
>>> hdl.vh_arch_file()
```

library_name()

Get string property “npilibraryName”.

Parameters: None.

Returns: string property of “npilibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_entity_handle()

Get handle lists of the 1-1 relationship “npiVhEntity”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhEntity”.

Return type: handle

Examples

```
>>> hdl = ref.vh_entity_handle()
```

vh_arch_body_handle()

Get handle lists of the 1-1 relationship “npiVhArchBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhArchBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_arch_body_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

port_handles()

Get handle lists of the 1-m relationship “npipPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npipPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_generic_map_handles()

Get handle lists of the 1-m relationship “npivhGenericMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGenericMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_map_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npivhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npivhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhMapElem

class lang.VhMapElemHandle(cps_obj)

Class for object type “npiVhMapElem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_formal_handle()

Get handle lists of the 1-1 relationship “npiVhFormal”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhFormal”.

Return type: handle

Examples

```
>>> hdl = ref.vh_formal_handle()
```

vh_actual_handle()

Get handle lists of the 1-1 relationship “npiVhActual”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhActual”.

Return type: handle

Examples

```
>>> hdl = ref.vh_actual_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

npivhPackDecl

class lang.VhPackDeclHandle(cps_obj)

Class for object type “npivhPackDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```



```
obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilLineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_library_name()

Get string property “npivhLibraryName”.

Parameters: None.

Returns: string property of “npivhLibraryName”.

Return type: string

Examples

```
>>> hdl.vh_library_name()
```

library_name()

Get string property “npilibraryName”.

Parameters: None.

Returns: string property of “npilibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_library_decl_handles()

Get handle lists of the 1-m relationship “npivhLibraryDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhLibraryDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_library_decl_handles()
```

vh_use_clause_handles()

Get handle lists of the 1-m relationship “npivhUseClause”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUseClause”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_clause_handles()
```

vh_generic_map_handles()

Get handle lists of the 1-m relationship “npivhGenericMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGenericMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_map_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npivhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

npivhForeverLoop

class lang.VhForeverLoopHandle(*cps_obj*)

Class for object type “npivhForeverLoop”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
```

```
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npivhEnumLiteral

class lang.VhEnumLiteralHandle(cps_obj)

Class for object type “npivhEnumLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_position()

Get integer property “npIVhPosition”.

Parameters: None.

Returns: integer property of “npIVhPosition”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```

```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

npiVhNullLiteral

class lang.VhNullLiteralHandle(cps_obj)

Class for object type “npiVhNullLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npNet => Success
Release object npNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
```

```
        sub #(.P_INT(20)) sub_inst();
    endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

npivhbitstrliteral

***class* lang.VhBitStrLiteralHandle(*cps_obj*)**

Class for object type “npivhbitstrliteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent

handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
```



```
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

const_type(is_get_enum=False)

Get property “npiConstType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiConstType”.

Return type: integer or string

Examples

```
>>> hdl.const_type() # Get string
>>> hdl.const_type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```

```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

npiVarUserAttrName

class lang.VhUserAttrNameHandle(cps_obj)

Class for object type “npiVarUserAttrName”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
```

```

from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_attr_spec_handle()

Get handle lists of the 1-1 relationship “npiVhAttrSpec”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhAttrSpec”.

Return type: handle

Examples

```
>>> hdl = ref.vh_attr_spec_handle()
```

npivhReturn

class lang.VhReturnHandle(cps_obj)

Class for object type “npivhReturn”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_return_expr_handle()

Get handle lists of the 1-1 relationship “npVhReturnExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhReturnExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_return_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npVhRange

class lang.VhRangeHandle(cps_obj)

Class for object type “npVhRange”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```

```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_dir_type(is_get_enum=False)

Get property “npiVhDirType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVhDirType”.

Return type: integer or string

Examples

```
>>> hdl.vh_dir_type()      # Get string
>>> hdl.vh_dir_type(True)  # Get integer
```

vh_left_range_handle()

Get handle lists of the 1-1 relationship “npivhLeftRange”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhLeftRange”.

Return type: handle

Examples

```
>>> hdl = ref.vh_left_range_handle()
```

vh_right_range_handle()

Get handle lists of the 1-1 relationship “npivhRightRange”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhRightRange”.

Return type: handle

Examples

```
>>> hdl = ref.vh_right_range_handle()
```

npivhPackage

class lang.VhPackageHandle(cps_obj)

Class for object type “npivhPackage”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()
```

```
# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

cell_instance()

Get integer property “npicellinstance”.

Parameters: None.

Returns: integer property of “npicellinstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

contain_protect()

Get integer property “npicontainprotect”.

Parameters: None.

Returns: integer property of “npicontainprotect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_library_name()

Get string property “npiVhLibraryName”.

Parameters: None.

Returns: string property of “npiVhLibraryName”.

Return type: string

Examples

```
>>> hdl.vh_library_name()
```

library_name()

Get string property “npiLibraryName”.

Parameters: None.

Returns: string property of “npiLibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_pack_decl_handle()

Get handle lists of the 1-1 relationship “npiVhPackDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhPackDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_pack_decl_handle()
```

vh_pack_body_handle()

Get handle lists of the 1-1 relationship “npiVhPackBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhPackBody”.

Return type: handle

Examples


```
>>> hdl = ref.vh_pack_body_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npivhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_generic_map_handles()

Get handle lists of the 1-m relationship “npivhGenericMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGenericMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_map_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npivhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

npivhProcDecl

class lang.VhProcDeclHandle(cps_obj)

Class for object type “npivhProcDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_num_params()

Get integer property “npivhNumParams”.

Parameters: None.

Returns: integer property of “npivhNumParams”.

Return type: int

Examples

```
>>> hdl.vh_num_params()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_subp_body_handle()

Get handle lists of the 1-1 relationship “npivhSubpBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhSubpBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_body_handle()
```

vh_param_handles()

Get handle lists of the 1-m relationship “npivhParam”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_param_handles()
```

npivhWait

class lang.VhWaitHandle(*cps_obj*)

Class for object type “npivhWait”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhseqstmt”.

Parameters: None.

Returns: integer property of “npivhseqstmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```


file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npiVhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_time_expr_handle()

Get handle lists of the 1-1 relationship “npiVhTimeExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhTimeExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_time_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_sensitivities_handles()

Get handle lists of the 1-m relationship “npivhSensitivities”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSensitivities”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_sensitivities_handles()
```

npivhConcAssert

class lang.VhConcAssertHandle(*cps_obj*)

Class for object type “npivhConcAssert”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npivhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npivhTypeConv

class lang.VhTypeConvHandle(*cps_obj*)

Class for object type “npivhTypeConv”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property "npiFile".

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_expr_handle()

Get handle lists of the 1-1 relationship “npiVhExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_expr_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

npiVhPhysLiteral

class lang.VhPhysLiteralHandle(cps_obj)

Class for object type “npiVhPhysLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

vh_abstract_literal_handle()

Get handle lists of the 1-1 relationship “npiVhAbstractLiteral”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhAbstractLiteral”.

Return type: handle

Examples

```
>>> hdl = ref.vh_abstract_literal_handle()
```

vh_phys_unit_handle()

Get handle lists of the 1-1 relationship “npiVhPhysUnit”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhPhysUnit”.

Return type: handle

Examples

```
>>> hdl = ref.vh_phys_unit_handle()
```

npiVhParamAttrName

class lang.VhParamAttrNameHandle(*cps_obj*)

Class for object type “npiVhParamAttrName”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_predef_attr_type(is_get_enum=False)

Get property “npiVarPredefAttrType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVarPredefAttrType”.

Return type: integer or string

Examples

```
>>> hdl.vh_predef_attr_type() # Get string
>>> hdl.vh_predef_attr_type(True) # Get integer
```

vh_attr_kind_type(is_get_enum=False)

Get property “npiVarAttrKindType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiVarAttrKindType”.

Return type: integer or string

Examples

```
>>> hdl.vh_attr_kind_type() # Get string
>>> hdl.vh_attr_kind_type(True) # Get integer
```


line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

vh_param_expr_handle()

Get handle lists of the 1-1 relationship “npiVhParamExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhParamExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_param_expr_handle()
```

npiVhForLoop

class lang.VhForLoopHandle(cps_obj)

Class for object type “npiVhForLoop”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vh_seq_stmt()`

Get integer property “`npiVarSeqStmt`”.

Parameters: None.

Returns: integer property of “`npiVarSeqStmt`”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

`line_no()`

Get integer property “`npiVarLineNo`”.

Parameters: None.

Returns: integer property of “`npiVarLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiVarFile`”.

Parameters: None.

Returns: string property of “`npiVarFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_param_handle()

Get handle lists of the 1-1 relationship “npivhParam”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhParam”.

Return type: handle

Examples

```
>>> hdl = ref.vh_param_handle()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npivhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npiVhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhCase

class lang.VhCaseHandle(cps_obj)

Class for object type “npiVhCase”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhseqstmt”.

Parameters: None.

Returns: integer property of “npivhseqstmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```


file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_case_expr_handle()

Get handle lists of the 1-1 relationship “npiVhCaseExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhCaseExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_case_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npivhSig

class lang.VhSigHandle(cps_obj)

Class for object type “npivhSig”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npisize”.

Parameters: None.

Returns: integer property of “npisize”.

Return type: int

Examples

```
>>> hdl.size()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npivhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npiVhSimpleSigAssign

class lang.VhSimpleSigAssignHandle(cps_obj)

Class for object type “npiVhSimpleSigAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_guarded()

Get integer property “npivhguarded”.

Parameters: None.

Returns: integer property of “npivhGuarded”.

Return type: int

Examples

```
>>> hdl.vh_guarded()
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_lhs_handle()

Get handle lists of the 1-1 relationship “npivhLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhLhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_lhs_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_waveform_handles()

Get handle lists of the 1-m relationship “npivhWaveform”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhWaveform”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_waveform_handles()
```

npiVhProcess

class lang.VhProcessHandle(cps_obj)

Class for object type “npiVhProcess”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_sensitivities_handles()

Get handle lists of the 1-m relationship “npivhSensitivities”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSensitivities”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_sensitivities_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhIndexVar

class lang.VhIndexVarHandle(cps_obj)

Class for object type “npiVhIndexVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```
obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilLineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

npiVhWhileLoop

class lang.VhWhileLoopHandle(cps_obj)

Class for object type “npiVhWhileLoop”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npVhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npVhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhSelectSigAssign

class lang.VhSelectSigAssignHandle(cps_obj)

Class for object type “npiVhSelectSigAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```


vh_guarded()

Get integer property “npivhGuarded”.

Parameters: None.

Returns: integer property of “npivhGuarded”.

Return type: int

Examples

```
>>> hdl.vh_guarded()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_select_expr_handle()

Get handle lists of the 1-1 relationship “npivhSelectExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhSelectExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_select_expr_handle()
```

vh_lhs_handle()

Get handle lists of the 1-1 relationship “npivhLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhLhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_lhs_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_waveform_handles()

Get handle lists of the 1-m relationship “npivhWaveform”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhWaveform”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_waveform_handles()
```

npivhlf

class lang.VhlfHandle(cps_obj)

Class for object type “npivhlf”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhseqstmt”.

Parameters: None.

Returns: integer property of “npivhseqstmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_branch_handles()

Get handle lists of the 1-m relationship “npivhBranch”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhBranch”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_branch_handles()
```

npivhIntTypeDecl

class lang.VhIntTypeDeclHandle(*cps_obj*)

Class for object type “npivhIntTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiFile`”.

Parameters: None.

Returns: string property of “`npiFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVarConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVarConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

npiVarEntity

class lang.VhEntityHandle(*cps_obj*)

Class for object type “npiVarEntity”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiParamLineNo`”.

Parameters: None.

Returns: integer property of “`npiParamLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`library_name()`

Get string property “`npiParamLibraryName`”.

Parameters: None.

Returns: string property of “`npiParamLibraryName`”.

Return type: string

Examples

```
>>> hdl.library_name()
```

`file()`

Get string property “`npiParamFile`”.

Parameters: None.

Returns: string property of “`npiParamFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

port_handles()

Get handle lists of the 1-m relationship “npiPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npiVhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

vh_library_decl_handles()

Get handle lists of the 1-m relationship “npiVhLibraryDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhLibraryDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_library_decl_handles()
```

vh_use_clause_handles()

Get handle lists of the 1-m relationship “npivhUseClause”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUseClause”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_clause_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

npiVhCompDecl

class lang.VhCompDeclHandle(cps_obj)

Class for object type “npiVhCompDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

port_handles()

Get handle lists of the 1-m relationship “npiPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npiVhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

npiVhSliceName

class lang.VhSliceNameHandle(cps_obj)

Class for object type “npiVhSliceName”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```
obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property "npiFile".

Parameters: None.

Returns: string property of "npiFile".

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npiVhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhConstParam

class lang.VhConstParamHandle(*cps_obj*)

Class for object type “npivhConstParam”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
```

```
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples


```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

direction(is_get_enum=False)

Get property “npiDirection”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiDirection”.

Return type: integer or string

Examples

```
>>> hdl.direction() # Get string
>>> hdl.direction(True) # Get integer
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_position()

Get integer property “npiVhPosition”.

Parameters: None.

Returns: integer property of “npiVhPosition”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhSelectWaveform

class lang.VhSelectWaveformHandle(*cps_obj*)

Class for object type “npivhSelectWaveform”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()
```

```
# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_waveform_elem_handle()

Get handle lists of the 1-1 relationship “npivhwaveformelem”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhwaveformelem”.

Return type: handle

Examples

```
>>> hdl = ref.vh_waveform_elem_handle()
```

vh_choices_handles()

Get handle lists of the 1-m relationship “npivhchoices”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhchoices”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_choices_handles()
```

npivhIntLiteral

class lang.VhIntLiteralHandle(cps_obj)

Class for object type “npivhIntLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```



```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

npiVarCondWaveform

class lang.VhCondWaveformHandle(cps_obj)

Class for object type “npiVarCondWaveform”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
```

```

from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_waveform_elem_handle()

Get handle lists of the 1-1 relationship “npiVhWaveformElem”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhWaveformElem”.

Return type: handle

Examples

```
>>> hdl = ref.vh_waveform_elem_handle()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npiVhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

npiVhLibraryDecl

class lang.VhLibraryDeclHandle(*cps_obj*)

Class for object type “npiVhLibraryDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(*is_get_enum=False*)

Get property “npiType”.

Parameters: *is_get_enum* (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

npiVhRecordTypeDecl

class lang.VhRecordTypeDeclHandle(*cps_obj*)

Class for object type “npiVhRecordTypeDecl”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```

```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_num_fields()

Get integer property “npivhnumfields”.

Parameters: None.

Returns: integer property of “npivhnumfields”.

Return type: int

Examples

```
>>> hdl.vh_num_fields()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_elem_decl_handles()

Get handle lists of the 1-m relationship “npiVhElemDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhElemDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_elem_decl_handles()
```

npiVhAttrSpec

class lang.VhAttrSpecHandle(cps_obj)

Class for object type “npiVhAttrSpec”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_attr_decl_handle()

Get handle lists of the 1-1 relationship “npiVhAttrDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhAttrDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_attr_decl_handle()
```

vh_expr_handle()

Get handle lists of the 1-1 relationship “npiVhExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_expr_handle()
```

vh_entity_designator_handles()

Get handle lists of the 1-m relationship “npiVhEntityDesignator”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhEntityDesignator”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_entity_designator_handles()
```

npiVhConcProcCall

class lang.VhConcProcCallHandle(cps_obj)

Class for object type “npiVhConcProcCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npitype".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of "npitype".

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property "npivhseqstmt".

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_subp_body_handle()

Get handle lists of the 1-1 relationship “npiVhSubpBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhSubpBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_body_handle()
```

vh_subp_decl_handle()

Get handle lists of the 1-1 relationship “npiVhSubpDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhSubpDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_decl_handle()
```

vh_map_elem_handles()

Get handle lists of the 1-m relationship “npivhMapElem”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhMapElem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_map_elem_handles()
```

npivhBlock

class lang.VhBlockHandle(cps_obj)

Class for object type “npivhBlock”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
```



```
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhseqstmt”.

Parameters: None.

Returns: integer property of “npivhseqstmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_label_name()

Get string property “npivhlabelname”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_guard_expr_handle()

Get handle lists of the 1-1 relationship “npivhGuardExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhGuardExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_guard_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

port_handles()

Get handle lists of the 1-m relationship “npipPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npiParam”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

vh_generic_map_handles()

Get handle lists of the 1-m relationship “npiVarGenericMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVarGenericMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_map_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npiVarGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVarGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npiVarInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVarInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npiVarStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhIndexedName

class lang.VhIndexedNameHandle(cps_obj)

Class for object type “npiVhIndexedName”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

value(*is_get_default=False*)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: *is_get_default (int)* – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property "npiFile".

Parameters: None.

Returns: string property of "npiFile".

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

parent_handle()

Get handle lists of the 1-1 relationship “npiParent”.

Parameters: None.

Returns: handle of 1-1 relationship “npiParent”.

Return type: handle

Examples

```
>>> hdl = ref.parent_handle()
```

vh_index_handles()

Get handle lists of the 1-m relationship “npiVhIndex”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhIndex”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_index_handles()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npiVhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhseqsigassign

class lang.VhSeqSigAssignHandle(*cps_obj*)

Class for object type “npivhseqsigassign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
```

```
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

vh_guarded()

Get integer property “npivhGuarded”.

Parameters: None.

Returns: integer property of “npivhGuarded”.

Return type: int

Examples

```
>>> hdl.vh_guarded()
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_lhs_handle()

Get handle lists of the 1-1 relationship “npivhLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhLhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_lhs_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_waveform_handles()

Get handle lists of the 1-m relationship “npivhWaveform”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhWaveform”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_waveform_handles()
```

npivhFuncCall

class lang.VhFuncCallHandle(*cps_obj*)

Class for object type “npivhFuncCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
```

Chapter 4: Class Objects

VHDL Objects

```

sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(*is_get_default=False*)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default` (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys
```

```
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_subp_body_handle()

Get handle lists of the 1-1 relationship “npiVhSubpBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhSubpBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_body_handle()
```

vh_subp_decl_handle()

Get handle lists of the 1-1 relationship “npiVhSubpDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhSubpDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_decl_handle()
```

vh_map_elem_handles()

Get handle lists of the 1-m relationship “npivhMapElem”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhMapElem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_map_elem_handles()
```

npivhBranch

class lang.VhBranchHandle(*cps_obj*)

Class for object type “npivhBranch”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`vh_branch_type(is_get_enum=False)`

Get property “`npiVhBranchType`”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiVhBranchType`”.

Return type: integer or string

Examples

```
>>> hdl.vh_branch_type() # Get string
>>> hdl.vh_branch_type(True) # Get integer
```

`file()`

Get string property “`npiFile`”.

Parameters: None.

Returns: string property of “`npiFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npiVhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

vh_cond_expr_handles()

Get handle lists of the 1-m relationship “npiVhCondExpr”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhCondExpr”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_cond_expr_handles()
```

npiVhNull

class lang.VhNullHandle(*cps_obj*)

Class for object type “npiVhNull”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```



```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npIVhSeqStmt”.

Parameters: None.

Returns: integer property of “npIVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npiVhArrayTypeDecl

class lang.VhArrayTypeDeclHandle(cps_obj)

Class for object type “npiVhArrayTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiParamType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vh_num_dims()`

Get integer property “`npiVarNumDims`”.

Parameters: None.

Returns: integer property of “`npiVarNumDims`”.

Return type: int

Examples

```
>>> hdl.vh_num_dims()
```

`line_no()`

Get integer property “`npiVarLineNo`”.

Parameters: None.

Returns: integer property of “`npiVarLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiVarFile`”.

Parameters: None.

Returns: string property of “`npiVarFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_elem_type_handle()

Get handle lists of the 1-1 relationship “npiVhElemType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhElemType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_elem_type_handle()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

npiVhFloatTypeDecl

class lang.VhFloatTypeDeclHandle(*cps_obj*)

Class for object type “npiVhFloatTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```


file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

npiVhRealLiteral

class lang.VhRealLiteralHandle(*cps_obj*)

Class for object type “npiVhRealLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiParam => Success
Release object npiParam => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiParam”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiParam”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

npiVhSigParam

class lang.VhSigParamHandle(cps_obj)

Class for object type “npiVhSigParam”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of "npiType".

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

direction(is_get_enum=False)

Get property "npiDirection".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npDirection`”.

Return type: integer or string

Examples

```
>>> hdl.direction()      # Get string
>>> hdl.direction(True)  # Get integer
```

`line_no()`

Get integer property “`npLineNo`”.

Parameters: None.

Returns: integer property of “`npLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`vh_position()`

Get integer property “`npVhPosition`”.

Parameters: None.

Returns: integer property of “`npVhPosition`”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

`size()`

Get integer property “`npSize`”.

Parameters: None.

Returns: integer property of “`npSize`”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```


vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhElemDecl

class lang.VhElemDeclHandle(cps_obj)

Class for object type “npivhElemDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npinType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npinType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_position()

Get integer property “npivhPosition”.

Parameters: None.

Returns: integer property of “npivhPosition”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

npiVhCharLiteral

class lang.VhCharLiteralHandle(*cps_obj*)

Class for object type “npiVhCharLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_position()

Get integer property “npIVhPosition”.

Parameters: None.

Returns: integer property of “npIVhPosition”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine value() will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If isGetDefaultVal is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
```

```
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

npiVhSeqProcCall

class lang.VhSeqProcCallHandle(cps_obj)

Class for object type “npiVhSeqProcCall”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npINet => Success
Release object npINet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npIVhSeqStmt”.

Parameters: None.

Returns: integer property of “npIVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npILineNo”.

Parameters: None.

Returns: integer property of “npILineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_subp_body_handle()

Get handle lists of the 1-1 relationship “npivhSubpBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhSubpBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_body_handle()
```

vh_subp_decl_handle()

Get handle lists of the 1-1 relationship “npivhSubpDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhSubpDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_decl_handle()
```

vh_map_elem_handles()

Get handle lists of the 1-m relationship “npivhMapElem”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhMapElem”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_map_elem_handles()
```

npiVhStringLiteral

class lang.VhStringLiteralHandle(cps_obj)

Class for object type “npiVhStringLiteral”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: `is_get_default (int)` – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())
```

```
# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

npiVarAttrDecl

class lang.VhAttrDeclHandle(cps_obj)

Class for object type “npiVarAttrDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```



```
obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilLineNo”.

Parameters: None.

Returns: integer property of “npilLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullName”.

Parameters: None.

Returns: string property of “npifullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

npiVhAggregate

class lang.VhAggregateHandle(*cps_obj*)

Class for object type “npiVhAggregate”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npNet => Success
Release object npNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npLineNo”.

Parameters: None.

Returns: integer property of “npLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npFile”.

Parameters: None.

Returns: string property of “npFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_elem_assoc_handles()

Get handle lists of the 1-m relationship “npVhElemAssoc”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhElemAssoc”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_elem_assoc_handles()
```

npiVhSeqAssert

class lang.VhSeqAssertHandle(cps_obj)

Class for object type “npiVhSeqAssert”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
```

```
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string  
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npVhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npVhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npiVhEnumTypeDecl

class lang.VhEnumTypeDeclHandle(cps_obj)

Class for object type “npiVhEnumTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")
```

```
def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npType”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_num_literals()

Get integer property “npivhNumLiterals”.

Parameters: None.

Returns: integer property of “npivhNumLiterals”.

Return type: int

Examples

```
>>> hdl.vh_num_literals()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_enum_literal_handles()

Get handle lists of the 1-m relationship “npivhEnumLiteral”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhEnumLiteral”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_enum_literal_handles()
```

npivhArchBody

class lang.VhArchBodyHandle(*cps_obj*)

Class for object type “npivhArchBody”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`file()`

Get string property “`npiFile`”.

Parameters: None.

Returns: string property of “`npiFile`”.

Return type: string

Examples

```
>>> hdl.file()
```

`name()`

Get string property “`npiName`”.

Parameters: None.

Returns: string property of “`npiName`”.

Return type: string

Examples

```
>>> hdl.name()
```

library_name()

Get string property “npilibraryName”.

Parameters: None.

Returns: string property of “npilibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_library_decl_handles()

Get handle lists of the 1-m relationship “npivhLibraryDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhLibraryDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_library_decl_handles()
```


vh_use_clause_handles()

Get handle lists of the 1-m relationship “npivhUseClause”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUseClause”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_clause_handles()
```

npivhGenerate

class lang.VhGenerateHandle(*cps_obj*)

Class for object type “npivhGenerate”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiVarType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vh_seq_stmt()`

Get integer property “`npiVarSeqStmt`”.

Parameters: None.

Returns: integer property of “`npiVarSeqStmt`”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

`line_no()`

Get integer property “`npiVarLineNo`”.

Parameters: None.

Returns: integer property of “`npiVarLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`vh_label_name()`

Get string property “`npiVarLabelName`”.

Parameters: None.

Returns: string property of “`npiVarLabelName`”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npiVhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npiVhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

npiVhWaveformElem

class lang.VhWaveformElemHandle(*cps_obj*)

Class for object type “npiVhWaveformElem”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_unaffected()

Get integer property “npIVhUnaffected”.

Parameters: None.

Returns: integer property of “npIVhUnaffected”.

Return type: int

Examples

```
>>> hdl.vh_unaffected()
```

npiVhVarParam

class lang.VhVarParamHandle(cps_obj)

Class for object type “npiVhVarParam”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of "npiType".

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

direction(is_get_enum=False)

Get property "npiDirection".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npDirection`”.

Return type: integer or string

Examples

```
>>> hdl.direction()      # Get string
>>> hdl.direction(True)  # Get integer
```

`line_no()`

Get integer property “`npLineNo`”.

Parameters: None.

Returns: integer property of “`npLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`vh_position()`

Get integer property “`npVhPosition`”.

Parameters: None.

Returns: integer property of “`npVhPosition`”.

Return type: int

Examples

```
>>> hdl.vh_position()
```

`size()`

Get integer property “`npSize`”.

Parameters: None.

Returns: integer property of “`npSize`”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npivhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npivhVarAssign

class lang.VhVarAssignHandle(cps_obj)

Class for object type “npivhVarAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhseqstmt”.

Parameters: None.

Returns: integer property of “npivhseqstmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_lhs_handle()

Get handle lists of the 1-1 relationship “npiVhLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhLhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_lhs_handle()
```

vh_rhs_handle()

Get handle lists of the 1-1 relationship “npiVhRhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhRhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_rhs_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npiVhQualifiedExpr

class lang.VhQualifiedExprHandle(cps_obj)

Class for object type “npiVhQualifiedExpr”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

value(is_get_default=False)

Retrieve the value of the parameter object and literal.

The NPI routine `value()` will adjust value type and return the value of handle reference in string or integer. Currently, NPI only supports getting the value for parameter and literal/constant objects.

Parameters: **is_get_default** (*int*) – Flag to determine to retrieve default value or final value. NPI only supports this option for parameter objects, constant objects, and expression objects that are composed by parameters or constants in SystemVerilog. If `isGetDefaultVal` is true, then the argument named “value” is referenced to default value. Otherwise, value is referenced to final value.

Returns: value of handle.

Return type: string or integer

Examples

example.v:

```
module sub;
    parameter P_INT = 10;
    parameter P_STR = "hello";
endmodule

module top;
    sub #(.P_INT(20)) sub_inst();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

p_int = lang.handle_by_name("top.sub_inst.P_INT", None)
p_str = lang.handle_by_name("top.sub_inst.P_STR", None)
print("Get value of "+p_int.full_name()+" -> "+str(p_int.value()))
print("Get default value of "+p_int.full_name()+" -> "+str(p_int.value(True)))
print("Get value of "+p_str.full_name()+" -> "+p_str.value())

# end NPI
npisys.end()
```

Output:

```
Get value of top.sub_inst.P_INT -> 20
Get default value of top.sub_inst.P_INT -> 10
Get value of top.sub_inst.P_STR -> hello
```

file()

Get string property "npiFile".

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_expr_handle()

Get handle lists of the 1-1 relationship “npiVhExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_expr_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

npiVhPhysUnit

class lang.VhPhysUnitHandle(cps_obj)

Class for object type “npiVhPhysUnit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_phys_literal_handle()

Get handle lists of the 1-1 relationship “npivhPhysLiteral”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhPhysLiteral”.

Return type: handle

Examples

```
>>> hdl = ref.vh_phys_literal_handle()
```

npivhElemAssoc

class lang.VhElemAssocHandle(cps_obj)

Class for object type “npivhElemAssoc”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```


Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

vh_is_named()

Get integer property “npivhIsNamed”.

Parameters: None.

Returns: integer property of “npivhIsNamed”.

Return type: int

Examples

```
>>> hdl.vh_is_named()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_expr_handle()

Get handle lists of the 1-1 relationship “npiVhExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_expr_handle()
```

vh_choices_handles()

Get handle lists of the 1-m relationship “npiVhChoices”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhChoices”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_choices_handles()
```

npiVhComponent

class lang.VhComponentHandle(cps_obj)

Class for object type “npiVhComponent”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
```

```
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_entity_line_no()

Get integer property “npivhEntityLineNo”.

Parameters: None.

Returns: integer property of “npivhEntityLineNo”.

Return type: int

Examples

```
>>> hdl.vh_entity_line_no()
```

vh_arch_line_no()

Get integer property “npivhArchLineNo”.

Parameters: None.

Returns: integer property of “npivhArchLineNo”.

Return type: int

Examples

```
>>> hdl.vh_arch_line_no()
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

bind()

Get integer property “npiBind”.

Parameters: None.

Returns: integer property of “npiBind”.

Return type: int

Examples

```
>>> hdl.bind()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

cell_instance()

Get integer property “npiCellInstance”.

Parameters: None.

Returns: integer property of “npiCellInstance”.

Return type: int

Examples

```
>>> hdl.cell_instance()
```

contain_protect()

Get integer property “npiContainProtect”.

Parameters: None.

Returns: integer property of “npiContainProtect”.

Return type: int

Examples

```
>>> hdl.contain_protect()
```

vh_entity_name()

Get string property “npiVhEntityName”.

Parameters: None.

Returns: string property of “npiVhEntityName”.

Return type: string

Examples

```
>>> hdl.vh_entity_name()
```

vh_arch_name()

Get string property “npiVhArchName”.

Parameters: None.

Returns: string property of “npiVhArchName”.

Return type: string

Examples

```
>>> hdl.vh_arch_name()
```

vh_entity_file()

Get string property “npiVhEntityFile”.

Parameters: None.

Returns: string property of “npiVhEntityFile”.

Return type: string

Examples

```
>>> hdl.vh_entity_file()
```

vh_arch_file()

Get string property “npivhArchFile”.

Parameters: None.

Returns: string property of “npivhArchFile”.

Return type: string

Examples

```
>>> hdl.vh_arch_file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

library_name()

Get string property “npilibraryName”.

Parameters: None.

Returns: string property of “npilibraryName”.

Return type: string

Examples

```
>>> hdl.library_name()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

vh_comp_decl_handle()

Get handle lists of the 1-1 relationship “npiVhCompDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhCompDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_comp_decl_handle()
```

vh_entity_handle()

Get handle lists of the 1-1 relationship “npiVhEntity”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhEntity”.

Return type: handle

Examples

```
>>> hdl = ref.vh_entity_handle()
```

vh_arch_body_handle()

Get handle lists of the 1-1 relationship “npivhArchBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhArchBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_arch_body_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

port_handles()

Get handle lists of the 1-m relationship “npipPort”.

Parameters: None.

Returns: handle list of 1-m relationship “npipPort”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.port_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_generic_map_handles()

Get handle lists of the 1-m relationship “npivhGenericMap”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGenericMap”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_map_handles()
```

vh_generic_handles()

Get handle lists of the 1-m relationship “npivhGeneric”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhGeneric”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_generic_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npivhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

npivhVar

class lang.VhVarHandle(*cps_obj*)

Class for object type “npivhVar”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
```

```
print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_shared()

Get integer property “npivhshared”.

Parameters: None.

Returns: integer property of “npivhshared”.

Return type: int

Examples

```
>>> hdl.vh_shared()
```

line_no()

Get integer property “npiline_no”.

Parameters: None.

Returns: integer property of “npiline_no”.

Return type: int

Examples

```
>>> hdl.line_no()
```

size()

Get integer property “npiSize”.

Parameters: None.

Returns: integer property of “npiSize”.

Return type: int

Examples

```
>>> hdl.size()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npivhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npivhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_init_expr_handle()

Get handle lists of the 1-1 relationship “npivhInitExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhInitExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_init_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_use_handles()

Get handle lists of the 1-m relationship “npiVhUse”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhUse”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_handles()
```

npiVhExit

class lang.VhExitHandle(*cps_obj*)

Class for object type “npiVhExit”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```



```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiVarType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vh_seq_stmt()`

Get integer property “`npiVarSeqStmt`”.

Parameters: None.

Returns: integer property of “`npiVarSeqStmt`”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

`line_no()`

Get integer property “`npiVarLineNo`”.

Parameters: None.

Returns: integer property of “`npiVarLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

`vh_loop_label_name()`

Get string property “`npiVarVhLoopLabelName`”.

Parameters: None.

Returns: string property of “`npiVarVhLoopLabelName`”.

Return type: string

Examples

```
>>> hdl.vh_loop_label_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npiVhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npiVhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npiVhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npiVhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npiVhFuncDecl

class lang.VhFuncDeclHandle(cps_obj)

Class for object type “npiVhFuncDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “`npiType`”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

`vh_pure()`

Get integer property “`npiVhPure`”.

Parameters: None.

Returns: integer property of “`npiVhPure`”.

Return type: int

Examples

```
>>> hdl.vh_pure()
```

`vh_num_params()`

Get integer property “`npiVhNumParams`”.

Parameters: None.

Returns: integer property of “`npiVhNumParams`”.

Return type: int

Examples

```
>>> hdl.vh_num_params()
```

`line_no()`

Get integer property “`npiLineNo`”.

Parameters: None.

Returns: integer property of “`npiLineNo`”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_return_type_handle()

Get handle lists of the 1-1 relationship “npiVhReturnType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhReturnType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_return_type_handle()
```

vh_subp_body_handle()

Get handle lists of the 1-1 relationship “npiVhSubpBody”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhSubpBody”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_body_handle()
```

npiVhCondSigAssign

class lang.VhCondSigAssignHandle(cps_obj)

Class for object type “npiVhCondSigAssign”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_guarded()

Get integer property “npivhguarded”.

Parameters: None.

Returns: integer property of “npiVarGuarded”.

Return type: int

Examples

```
>>> hdl.vh_guarded()
```

line_no()

Get integer property “npiVarLineNo”.

Parameters: None.

Returns: integer property of “npiVarLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiVarFile”.

Parameters: None.

Returns: string property of “npiVarFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_seq_stmt()

Get integer property “npiVarVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVarVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

vh_label_name()

Get string property “npiVarVhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_lhs_handle()

Get handle lists of the 1-1 relationship “npivhLhs”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhLhs”.

Return type: handle

Examples

```
>>> hdl = ref.vh_lhs_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

vh_waveform_handles()

Get handle lists of the 1-m relationship “npivhWaveform”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhWaveform”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_waveform_handles()
```

npivhPhysTypeDecl

class lang.VhPhysTypeDeclHandle(cps_obj)

Class for object type “npivhPhysTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVhConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

vh_phys_unit_handles()

Get handle lists of the 1-m relationship “npiVhPhysUnit”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhPhysUnit”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_phys_unit_handles()
```

npiVhUseClause

class lang.VhUseClauseHandle(*cps_obj*)

Class for object type “npiVhUseClause”.

set_permanent()

Set the handle as 'permanent'. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as "permanent". To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```



```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npiType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

npivhReport

class lang.VhReportHandle(cps_obj)

Class for object type “npivhReport”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
```

```
if hdl.set_permanent():
    print("Set permanent for object "+obj_type+" => Success")
else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npivhSeqStmt”.

Parameters: None.

Returns: integer property of “npivhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npivhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npivhSubTypeDecl

class lang.VhSubTypeDeclHandle(cps_obj)

Class for object type “npivhSubTypeDecl”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")
```

```
# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_unconstraint()

Get integer property “npivhUnconstraint”.

Parameters: None.

Returns: integer property of “npivhUnconstraint”.

Return type: int

Examples

```
>>> hdl.vh_unconstraint()
```

vh_anonymous()

Get integer property “npivhAnonymous”.

Parameters: None.

Returns: integer property of “npivhAnonymous”.

Return type: int

Examples

```
>>> hdl.vh_anonymous()
```

vh_resolved()

Get integer property “npivhResolved”.

Parameters: None.

Returns: integer property of “npivhResolved”.

Return type: int

Examples

```
>>> hdl.vh_resolved()
```

vh_predefine()

Get integer property “npivhPredefine”.

Parameters: None.

Returns: integer property of “npivhPredefine”.

Return type: int

Examples

```
>>> hdl.vh_predefine()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```


name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_base_type_handle()

Get handle lists of the 1-1 relationship “npiVhBaseType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhBaseType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_base_type_handle()
```

vh_type_handle()

Get handle lists of the 1-1 relationship “npiVhType”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhType”.

Return type: handle

Examples

```
>>> hdl = ref.vh_type_handle()
```

vh_resol_func_handle()

Get handle lists of the 1-1 relationship “npiVhResolFunc”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhResolFunc”.

Return type: handle

Examples

```
>>> hdl = ref.vh_resol_func_handle()
```

vh_constraint_handle()

Get handle lists of the 1-1 relationship “npiVarConstraint”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVarConstraint”.

Return type: handle

Examples

```
>>> hdl = ref.vh_constraint_handle()
```

npiVarOthers

class lang.VhOthersHandle(*cps_obj*)

Class for object type “npiVarOthers”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npiNet => Success
Release object npiNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npiType".

Parameters: `is_get_enum` (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

npiVhNext

class lang.VhNextHandle(*cps_obj*)

Class for object type “npiVhNext”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys
```

```
def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property "npIType".

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npiType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_seq_stmt()

Get integer property “npiVhSeqStmt”.

Parameters: None.

Returns: integer property of “npiVhSeqStmt”.

Return type: int

Examples

```
>>> hdl.vh_seq_stmt()
```

vh_loop_label_name()

Get string property “npiVhLoopLabelName”.

Parameters: None.

Returns: string property of “npiVhLoopLabelName”.

Return type: string

Examples

```
>>> hdl.vh_loop_label_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_label_name()

Get string property “npiVhLabelName”.

Parameters: None.

Returns: string property of “npivhLabelName”.

Return type: string

Examples

```
>>> hdl.vh_label_name()
```

vh_cond_expr_handle()

Get handle lists of the 1-1 relationship “npivhCondExpr”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhCondExpr”.

Return type: handle

Examples

```
>>> hdl = ref.vh_cond_expr_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

npiVhPackBody

class lang.VhPackBodyHandle(cps_obj)

Class for object type “npiVhPackBody”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```



```
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

vh_library_decl_handles()

Get handle lists of the 1-m relationship “npiVhLibraryDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npiVhLibraryDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_library_decl_handles()
```

vh_use_clause_handles()

Get handle lists of the 1-m relationship “npiVhUseClause”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhUseClause”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_use_clause_handles()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

npivhFunction

class lang.VhFunctionHandle(cps_obj)

Class for object type “npivhFunction”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
    wire a, b;
    assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)
```

```
# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

vh_overload()

Get integer property “npivhOverload”.

Parameters: None.

Returns: integer property of “npivhOverload”.

Return type: int

Examples

```
>>> hdl.vh_overload()
```

line_no()

Get integer property “npilineNo”.

Parameters: None.

Returns: integer property of “npilineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiName”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npiFullName”.

Parameters: None.

Returns: string property of “npiFullName”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npiFile”.

Parameters: None.

Returns: string property of “npiFile”.

Return type: string

Examples

```
>>> hdl.file()
```

vh_subp_decl_handle()

Get handle lists of the 1-1 relationship “npiVhSubpDecl”.

Parameters: None.

Returns: handle of 1-1 relationship “npiVhSubpDecl”.

Return type: handle

Examples

```
>>> hdl = ref.vh_subp_decl_handle()
```

vh_scope_handle()

Get handle lists of the 1-1 relationship “npivhScope”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhScope”.

Return type: handle

Examples

```
>>> hdl = ref.vh_scope_handle()
```

vh_upper_stmt_handle()

Get handle lists of the 1-1 relationship “npivhUpperStmt”.

Parameters: None.

Returns: handle of 1-1 relationship “npivhUpperStmt”.

Return type: handle

Examples

```
>>> hdl = ref.vh_upper_stmt_handle()
```

vh_decl_handles()

Get handle lists of the 1-m relationship “npivhDecl”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhDecl”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_decl_handles()
```

vh_spec_handles()

Get handle lists of the 1-m relationship “npivhSpec”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhSpec”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_spec_handles()
```

vh_internal_scope_handles()

Get handle lists of the 1-m relationship “npivhInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_internal_scope_handles()
```

vh_stmt_handles()

Get handle lists of the 1-m relationship “npivhStmt”.

Parameters: None.

Returns: handle list of 1-m relationship “npivhStmt”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.vh_stmt_handles()
```

SystemC Objects

The following table describes the Object Category and their corresponding object types or tags with pyDoc link:

Object Category	Names in the Object Diagram	Object types or tags with pyDoc link
Sc Instance	Instance	npivhScInstance
Sc Scope	sc scope	(class)
Sc Method Process	sc method process	npivhScMethodProcess

For more information about SystemC Object Category, see the “Object Diagrams” chapter in the *VC Apps Native Programming Interface (NPI) Guide*.

npiScMethodProcess

class lang.ScMethodProcessHandle(cps_obj)

Class for object type “npiScMethodProcess”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
```

```

else:
    print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()

```

Result:

```

Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!

```

type(is_get_enum=False)

Get property “npitype”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npitype”.

Return type: integer or string

Examples

```

>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer

```

def_line_no()

Get integer property “npideflineno”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_name()

Get string property “npiDefName”.

Parameters: None.

Returns: string property of “npiDefName”.

Return type: string

Examples

```
>>> hdl.def_name()
```

def_file()

Get string property “npiDefFile”.

Parameters: None.

Returns: string property of “npiDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

name()

Get string property “npiName”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiscope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiscope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

sc_internal_scope_handles()

Get handle lists of the 1-m relationship “npiscinternalscope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiScInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sc_internal_scope_handles()
```

npiScInstance

class lang.ScInstanceHandle(cps_obj)

Class for object type “npiScInstance”.

set_permanent()

Set the handle as ‘permanent’. The handle cannot be released until `release_all_handles` is called.

The NPI Language API `set_permanent` will set a handle as “permanent”. To release such a handle, `release_handle` will not work anymore. The only way to release permanent handles is to call `release_all_handles`. The routine will return 1 (true) for all valid handles except iterators and will return 0 (false) for all invalid handles.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
```

```
obj_type = hdl.type()
if lang.release_handle(hdl):
    print("Release object "+obj_type+" => Success")
else:
    print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Result:

```
Set permanent for object nppNet => Success
Release object nppNet => Failure
All handles are released!
```

type(is_get_enum=False)

Get property “npIType”.

Parameters: **is_get_enum** (*bool*) – option to select the data type of return value. True: return integer value; False: return string value. Default is False.

Returns: property of “npIType”.

Return type: integer or string

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get integer
```

top()

Get integer property “npiTop”.

Parameters: None.

Returns: integer property of “npiTop”.

Return type: int

Examples

```
>>> hdl.top()
```

def_line_no()

Get integer property “npiDefLineNo”.

Parameters: None.

Returns: integer property of “npiDefLineNo”.

Return type: int

Examples

```
>>> hdl.def_line_no()
```

line_no()

Get integer property “npiLineNo”.

Parameters: None.

Returns: integer property of “npiLineNo”.

Return type: int

Examples

```
>>> hdl.line_no()
```

def_file()

Get string property “npiDefFile”.

Parameters: None.

Returns: string property of “npiDefFile”.

Return type: string

Examples

```
>>> hdl.def_file()
```

name()

Get string property “npiname”.

Parameters: None.

Returns: string property of “npiname”.

Return type: string

Examples

```
>>> hdl.name()
```

full_name()

Get string property “npifullname”.

Parameters: None.

Returns: string property of “npifullname”.

Return type: string

Examples

```
>>> hdl.full_name()
```

file()

Get string property “npifile”.

Parameters: None.

Returns: string property of “npifile”.

Return type: string

Examples

```
>>> hdl.file()
```

scope_handle()

Get handle lists of the 1-1 relationship “npiscope”.

Parameters: None.

Returns: handle of 1-1 relationship “npiscope”.

Return type: handle

Examples

```
>>> hdl = ref.scope_handle()
```

sc_internal_scope_handles()

Get handle lists of the 1-m relationship “npiScInternalScope”.

Parameters: None.

Returns: handle list of 1-m relationship “npiScInternalScope”.

Return type: list of handle

Examples

```
>>> hdl_list = ref.sc_internal_scope_handles()
```

5

L0 APIs

L0 APIs available in the NPI Language Model.

API	Description
handle_by_name	Get a handle to an object with a specific name.
handle_by_index	Get a member handle to an object by its parent handle and a specified index number.
handle_by_range	Get a part-select handle to an object by the handle that covers it and a specified range.
get_top_inst_list	Get a list of all top instances.
release_handle	Release a handle and its associated resources allocated by NPI Language routines.
release_all_handles	Release all the handles and their associated resources allocated by NPI Language routines.
compare_objects	Compare two handles to determine whether they refer to the same object.
objects_overlap	Check if two handles overlap physically.

handle_by_name

lang.handle_by_name(name, hdl)

Get a handle to an object with a specific name.

The NPI routine `handle_by_name` will return a handle to an object with a specific name. This function can be applied to all objects with a `fullname` property. The name can be hierarchical or simple. If scope is `NULL` (C/C++), empty string `""` (Tcl), or `None` (Python) then the name will be searched for from the top level of hierarchy. If a scope object is provided, then search within that scope only.

Parameters:

- **name** (*string*) – A character string or pointer to a string containing the name of an object.
- **scope** (*handle*) – Handle to a SystemVerilog scope.

Returns: the handle to an object.

Return type: handle

Examples

example.v:

```
module Counter (input wire  clock, resetN,
                 output logic [15:0]  count);

    always @(posedge clock, negedge resetN) begin : B1
        if( !resetN ) count <= 0;
        else  count <= count + 1;
    end

endmodule

module TEST;
    wire [15:0] count;
    bit clock;
    bit resetN = 1;

    Counter dut (clock, resetN, count);

    always #10 clock = ~clock;

    initial begin
        resetN = 0;
        #2  resetN = 1;
        #1  $finish;
    end
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
```

```
print("Error: Fail to initialize NPI.")
assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# get the module instance handle for "TEST.dut"
mod_hdl = lang.handle_by_name("TEST.dut", None)
if mod_hdl is not None:
    print("The fullname of the handle is: "+mod_hdl.full_name());
    print("The type of the handle is: "+mod_hdl.type()+"\n");

# get the variable handle with simple name "count" in the scope of
mod_hdl
var_hdl = lang.handle_by_name("count", mod_hdl)
if var_hdl is not None:
    print("The fullname of the handle is: "+var_hdl.full_name());
    print("The type of the handle is: "+var_hdl.type()+"\n");

# end NPI
npisys.end()
```

Output:

```
The fullname of the handle is: TEST.dut
The type of the handle is: npiParam

The fullname of the handle is: TEST.dut.count
The type of the handle is: npiParam
```

handle_by_index

lang.handle_by_index(hdl, index)

Get a member handle to an object by its parent handle and a specified index number.

The NPI routine `handle_by_index` will return a member handle to an object by its parent handle and a specified index number. This function can be applied to all objects that which can be selected by index, e.g. getting an array member.

Parameters:

- **hdl** (*handle*) – Handle that can be selected by index. For VHDL, it is not support the object whose type is multi dimensions. For example: type `d2_array` is array (31 downto 0, 4 downto 0) of `std_ulogic`;
- **index** (*int*) – Index number.

Returns: the handle to an object.

Return type: handle

Examples

example.v:

```
module mm;
    reg mr;
endmodule

module top;
    bit [3:0] r;
    mm mArr [1:0] ();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# get the var bit handle for "top.r[1]"
var_hdl = lang.handle_by_name("top.r", None)
if var_hdl is not None:
    idx = 1
    select_hdl = lang.handle_by_index(var_hdl, idx)
    if select_hdl is not None:
        print("Get member "+select_hdl.full_name()+" by object "+var_hdl.full_name()+" and index "+str(idx))

# get the module handle for "top.mArr[0]"
arr_hdl = lang.handle_by_name("top.mArr", None)
if arr_hdl is not None:
    idx = 0
    select_hdl = lang.handle_by_index(arr_hdl, idx)
    if select_hdl is not None:
        print("Get member "+select_hdl.full_name()+" by object "+arr_hdl.full_name()+" and index "+str(idx))
```

```
# end NPI
npisys.end()
```

Output:

```
Get member top.r[1] by object top.r and index 1
Get member top.mArr[0] by object top.mArr and index 0
```

handle_by_range

lang.handle_by_range(hdl, left, right)

Get a part-select handle to an object by the handle that covers it and a specified range.

The NPI routine `handle_by_range` will return a part-select or slice name handle to an object by the handle that covers it and a specified range. This function can be applied to all objects that can be sliced by left and right range, e.g. getting a part-select from a bit-var. If the input left range is equal to the right range, the functionality of `handle_by_range` will be equivalent to the functionality of `handle_by_index`.

Parameters:

- **hdl** (*handle*) – Handle that can be sliced by range. For VHDL, it is not support the object whose type is multi dimensions. For example: type `d2_array` is array (31 down to 0, 4 downto 0) of `std_ulogic`;
- **left** (*int*) – Left range number.
- **right** (*int*) – Right range number.

Returns: the handle to an object.

Return type: handle

Examples

example.v:

```
module top;
    bit [5:0] r;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

```
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

var_hdl = lang.handle_by_name("top.r", None)
if var_hdl is not None:
    # get the bit var handle for "top.r[4:3]"
    left = 4
    right = 3
    slice_hdl = lang.handle_by_range(var_hdl, left, right)
    if slice_hdl is not None:
        parent_hdl = slice_hdl.parent_handle()
        left_hdl = slice_hdl.left_range_handle()
        right_hdl = slice_hdl.right_range_handle()
        print("Get slice handle type "+slice_hdl.type()+" and its range
["+str(left_hdl.value())+": "+str(right_hdl.value())+"] and its parent is
"+parent_hdl.full_name())
        lang.release_handle(left_hdl)
        lang.release_handle(right_hdl)
        lang.release_handle(parent_hdl)
        lang.release_handle(slice_hdl)
        lang.release_handle(var_hdl)

# end NPI
npisys.end()
```

Output:

```
Get slice handle type npiParamSelect and its range [4:3] and its parent is
top.r
```

get_top_inst_list

lang.get_top_inst_list()

Get top instances.

NPI function “get_top_inst_list” read the input design and return a list of all top instances.

Parameters: None.

Returns: list of top handle objects.

Return type: list of handle

Examples

example.v:

```
package pkg;
endpackage

module mod;
  sub sub_inst();
endmodule

interface itf;
endinterface

module sub;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# get top instances
top_inst_list = lang.get_top_inst_list()
for inst_hdl in top_inst_list:
    print("Top instance: "+inst_hdl.full_name())

# end NPI
npisys.end()
```

Output:

```
Top instance: mod
Top instance: itf
Top instance: pkg
```

release_handle

lang.release_handle(hdl)

Release a handle and its associated resources allocated by NPI routines.

The NPI routine `release_handle` will free memory allocated for NPI handles. NPI may allocate memory when a handle to an object is obtained. One may safely ignore calling `release_handle` when a handle is no longer needed, but it is always recommended to do so, provided the handle is valid and will not automatically become invalid in the future. A guideline for better memory usage is to release the created handles whenever they are no longer needed. This avoids logical memory leaks.

`release_handle` may not be called on an invalid handle.

Parameters: `hdl` (*handle*) – Handle to an object.

Returns: Returns 1 (true) if the NPI memory management system releases the memory used by the object. Returns 0 (false) if NPI does not free the memory of the object because there might be another application program using it.

Return type: int

Examples

example.v:

```
module Counter (input wire  clock, resetN,
                output logic [15:0] count);

    always @(posedge clock, negedge resetN) begin : B1
        if( !resetN ) count <= 0;
        else count <= count + 1;
    end

endmodule

module TEST;
    wire [15:0] count;
    bit clock;
    bit resetN = 1;

    Counter dut (clock, resetN, count);

    always #10 clock = ~clock;

    initial begin
        resetN = 0;
        #2 resetN = 1;
        #1 $finish;
    end
endmodule
```

```
    end
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# Get the module instance handle for "TEST.dut"
mod_hdl = lang.handle_by_name("TEST.dut", None)
if mod_hdl is not None:
    print("Module: "+mod_hdl.full_name())
    # get the scope of the module handle "TEST.dut"
    scope_hdl = mod_hdl.scope_handle()
    print("  The module is in scope: "+scope_hdl.full_name())
    # iterate the port of "TEST.dut"
    port_hdl_list = mod_hdl.port_handles()
    i = 0
    for port_hdl in port_hdl_list:
        # get the int/bool property and string property of port_hdl
        print("    "+str(i)+" Port = "+port_hdl.name()+" ("
            +port_hdl.direction()+" Size="+str(port_hdl.size())+")")
        # release port_hdl since it is useless below
        lang.release_handle(port_hdl)
        i = i+1

# end NPI
npisys.end()
```

Output:

```
Module: TEST.dut
The module is in scope: TEST
0) Port = clock (npiInput Size=1)
1) Port = resetN (npiInput Size=1)
2) Port = count (npiOutput Size=16)
```

release_all_handles

lang.release_all_handles()

Release all the handles and their associated resources allocated by NPI Language routines.

The NPI Language API `release_all_handles` will release all the handles currently created in the Language model, including the handles that are set as “permanent” (refer to `set_permanent`). The routine will return 1 (true) on success and 0 (false) on failure.

Parameters: None.

Returns: Return 1 (true) on success; otherwise, return 0 (false) on failure.

Return type: int

Examples

example.v:

```
module top;
  wire a, b;
  assign a = b;
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

def release_test(hdl):
    obj_type = hdl.type()
    if lang.release_handle(hdl):
        print("Release object "+obj_type+" => Success")
    else:
        print("Release object "+obj_type+" => Failure")

def permanent_test(hdl):
    obj_type = hdl.type()
    if hdl.set_permanent():
        print("Set permanent for object "+obj_type+" => Success")
    else:
        print("Set permanent for object "+obj_type+" => Failure")

# initialize NPI and load design example.v
```

```
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("top.a", None)
# set net object as permanent => success
permanent_test(hdl)
# release net object as permanent => failure
release_test(hdl)

# release all handles, including permanent handles
lang.release_all_handles()

# net object is invalid
if hdl.type() is None:
    print("All handles are released!")

# end NPI
npisys.end()
```

Output:

```
Set permanent for object npinet => Success
Release object npinet => Failure
All handles are released!
```

compare_objects

lang.compare_objects(hdl1, hdl2)

Compare two handles to determine whether they refer to the same object.

The NPI routine `compare_objects()` will return 1 (TRUE) if the two handles refer to the same object. Otherwise, 0 (FALSE) will be returned. The only thing NPI can compare for variable is whether the variable is a constant. For example:

```
reg arr [1:0];
parameter p = 0;
integer i = 0;

lang.compare_objects (arr[0], arr[p]) # return 1
lang.compare_objects (arr[0], arr[i]) # return 0; NPI cannot get i's
value (defined in simulation time)
```

The following is another example:

```
struct packed {  
    int a;  
    reg [0:7] b;  
} ps;  
  
lang.compare_objects (ps[0], ps.b[7]) # return 1
```

Parameters:

- **hdl1** (*handle*) – Handle to an object.
- **hdl2** (*handle*) – Handle to an object.

Returns: 1 (TRUE) if the two handles refer to the same object; otherwise, 0 (FALSE).

Return type: int

Examples

example.v:

```
module top;  
  
    typedef struct packed {  
        bit    [2:0] sa;  
        integer sb;  
        logic [1:0] sc;  
    } ps;  
  
    typedef struct packed {  
        bit    [1:0] a;  
        logic    b;  
        ps        c;  
    } st;  
  
    reg [1:0] regArr [1:0];  
    const integer idx0 = 0;  
    parameter p0 = 0;  
    integer itg;  
    st st1;  
  
    initial begin: init_blk  
        parameter p1 = 1;  
        // left [1]; right [1]  
        regArr[1] = regArr[1];  
        regArr[idx0+1] = regArr[2'b01];  
        regArr[p0+p1] = regArr[p1];  
        regArr[p0+1] = regArr[p0+p1];  
        // left [0][0]; right [0][0]  
        regArr[0][0] = regArr[0][0];  
        regArr[0][idx0] = regArr[0][idx0];  
    end
```

```

regArr[idx0][0] = regArr[idx0][1-1];
regArr[p0][p1*0] = regArr[p0][idx0];
regArr[1'b0][0] = regArr[0][idx0+p0];
// left == right
st1.a[0] = st1[38];
st1.b = st1[37];
st1.c.sc[0] = st1[0];
st1.c.sc[0] = st1.c[0];
st1.c.sb = st1.c[33:2];
st1.c.sb = st1.c[33-:32];
st1.c.sb[0] = st1.c[2];
// compare constant index for bit-select
itg[1] = itg[2'b01];
itg[p1] = itg['h1];
end

```

endmodule

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python
```

```

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

```

```

#-----
def check_bit_sel_idx(lhs_hdl, rhs_hdl):
    if lhs_hdl is None or rhs_hdl is None:
        return 0

    if lhs_hdl.type() != "npiBitSelect" or rhs_hdl.type() !=
    "npiBitSelect":
        return 0

    lhs_index_list = lhs_hdl.index_handles()
    rhs_index_list = rhs_hdl.index_handles()
    for index, (lhs_idx_hdl, rhs_idx_hdl) in
    enumerate(zip(lhs_index_list, rhs_index_list)):
        ret = lang.compare_objects(lhs_idx_hdl, rhs_idx_hdl)
        lang.release_handle(lhs_idx_hdl)
        lang.release_handle(rhs_idx_hdl)
        return ret

    return 0

#-----
def check_stmt(hdl):
    if hdl is None or hdl.type() != "npiAssignment":

```

```

        return
    lhs_hdl = hdl.lhs_handle()
    rhs_hdl = hdl.rhs_handle()

    if check_bit_sel_idx(lhs_hdl, rhs_hdl) > 0:
        print("Lhs-index and rhs-index for line "+str(hdl.line_no())+"
are the same")
        return

    if lhs_hdl.type() == "npiBitSelect" and rhs_hdl.type() ==
    "npiBitSelect":
        idx_l_hdl = lhs_hdl.index_handle()
        idx_r_hdl = rhs_hdl.index_handle()
        if lang.compare_objects(idx_l_hdl, idx_r_hdl):
            print("Lhs and rhs for line "+str(hdl.line_no())+" are the
same")
        else:
            print("Lhs and rhs for line "+str(hdl.line_no())+" are
different")
        lang.release_handle(idx_l_hdl)
        lang.release_handle(idx_r_hdl)
        lang.release_handle(lhs_hdl)
        lang.release_handle(rhs_hdl)
        return

    if lang.compare_objects(lhs_hdl, rhs_hdl) == 1:
        print("Lhs and rhs for line "+str(hdl.line_no())+" are the same")
    else:
        print("Lhs and rhs for line "+str(hdl.line_no())+" are
different")
        lang.release_handle(lhs_hdl)
        lang.release_handle(rhs_hdl)

#-----
def trv_scope(hdl):
    if hdl is None:
        return

    if hdl.type() in ['npiTask', 'npiFunction', 'npiNamedBegin',
'npiBegin', 'npiNamedFork', 'npiFork']:
        stmt_list = hdl.stmt_handles()
        for stmt in stmt_list:
            check_stmt(stmt)
            lang.release_handle(stmt)

    internal_scope_list = hdl.internal_scope_handles()
    for sub_scp in internal_scope_list:
        trv_scope(sub_scp)
        lang.release_handle(sub_scp)

#-----

```

```
def compare_both_side_for_assignment():
    top_inst_list = lang.get_top_inst_list()
    for top_inst in top_inst_list:
        trv_scope(top_inst)
        lang.release_handle(top_inst)

#-----
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

compare_both_side_for_assignment()

# end NPI
npisys.end()
```

Output:

```
Lhs and rhs for line 24 are the same
Lhs and rhs for line 25 are the same
Lhs and rhs for line 26 are the same
Lhs and rhs for line 27 are the same
Lhs and rhs for line 29 are the same
Lhs and rhs for line 30 are the same
Lhs and rhs for line 31 are the same
Lhs and rhs for line 32 are the same
Lhs and rhs for line 33 are the same
Lhs and rhs for line 35 are the same
Lhs and rhs for line 36 are the same
Lhs and rhs for line 37 are the same
Lhs and rhs for line 38 are the same
Lhs and rhs for line 39 are the same
Lhs and rhs for line 40 are the same
Lhs and rhs for line 41 are the same
Lhs-index and rhs-index for line 43 are the same
Lhs-index and rhs-index for line 44 are the same
```

objects_overlap

lang.objects_overlap(hdl1, hdl2)

Check if two handles overlap physically.

The NPI routine `objects_overlap()` will return 1 (TRUE) if the two handles overlap physically. Otherwise, 0 (FALSE) will be returned. The valid input handle for `objects_overlap` is the handle that belongs to vh name, simple expression, part-select or indexed part-select. the valid input handle belongs to If any input handle doesn't belong to vh name, simple expression, part-select or indexed part-select, `objects_overlap` will always return 0 (FALSE). VIA will treat the variable as arbitrary value whether the variable is not a constant. For example,

```
reg [2:0] pr;
parameter p = 0;
integer i = 0;

lang.objects_overlap (pr[1:0], pr[p]) # return 1
lang.objects_overlap (pr[1:0], pr[i]) # return 1; It is possible to be
    overlap because VIA cannot get i's value (defined in simulation time),
    treat "i" as arbitrary value.
lang.objects_overlap (pr[i:1], pr[p]) # return 0; It is impossible to be
    overlap even VIA treat "i" as arbitrary value.
```

The following is another example:

```
struct packed {
    int a;
    reg [0:7] b;
} ps;

lang.compare_objects (ps[8:4], ps.b[0:1]) # return 1
```

Parameters:

- **hdl1** (*handle*) – Handle to an object.
- **hdl2** (*handle*) – Handle to an object.

Returns: 1 (TRUE) if the two handles overlap physically; otherwise, 0 (FALSE).

Return type: int

Examples

example.v:

```
module top;

    typedef struct packed {
        bit    [2:0] sa;
        integer sb;
        logic  [1:0] sc;
    } ps;

    typedef struct packed {
        bit    [1:0] a;
        logic          b;
    }
```

```

    ps          c;
} st;

reg [3:0] regArr [1:0];
const integer idx0 = 0;
parameter p0 = 0;
st st1;

initial begin: init_blk
    parameter pl = 1;
    // -- left and right overlap -----
    // left contains right
    regArr[0][1:0] = regArr[0][0];
    regArr[0][1:p0] = regArr[0][idx0];
    // partial left is equal to partial right
    regArr[0][3:1] = regArr[0][2:0];
    st1.a = st1[38];
    st1.b = st1[38:35];
    // -- left and right do not overlap -----
    regArr[1][1:0] = regArr[0][1:0];
    st1[20:10] = st1[9:0];
end

endmodule

```

example.py:

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

#-----
def check_stmt(hdl):
    if hdl is None or hdl.type() != "npiAssignment":
        return
    lhs_hdl = hdl.lhs_handle()
    rhs_hdl = hdl.rhs_handle()

    if lang.objects_overlap(lhs_hdl, rhs_hdl) == 1:
        print("Lhs and rhs for line "+str(hdl.line_no())+" overlap")
    else:
        print("Lhs and rhs for line "+str(hdl.line_no())+" do not overlap")

    lang.release_handle(lhs_hdl)
    lang.release_handle(rhs_hdl)

```

```
#-----
def trv_scope(hdl):
    if hdl is None:
        return

    if hdl.type() in ['npiTask', 'npiFunction', 'npiNamedBegin',
'npiBegin', 'npiNamedFork', 'npiFork']:
        stmt_list = hdl.stmt_handles()
        for stmt in stmt_list:
            check_stmt(stmt)
            lang.release_handle(stmt)

    internal_scope_list = hdl.internal_scope_handles()
    for sub_scp in internal_scope_list:
        trv_scope(sub_scp)
        lang.release_handle(sub_scp)

#-----
def compare_both_side_for_assignment():
    top_inst_list = lang.get_top_inst_list()
    for top_inst in top_inst_list:
        trv_scope(top_inst)
        lang.release_handle(top_inst)

#-----
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

compare_both_side_for_assignment()

# end NPI
npisys.end()
```

Output:

```
Lhs and rhs for line 24 overlap
Lhs and rhs for line 25 overlap
Lhs and rhs for line 27 overlap
Lhs and rhs for line 28 overlap
Lhs and rhs for line 29 overlap
Lhs and rhs for line 31 do not overlap
Lhs and rhs for line 32 do not overlap
```

6

L1 APIs

L1 APIs available in the NPI Language Model.

Hierarchy Tree

API	Description
hier_tree_dump_txt	Dump the design hierarchy tree in plain text format.
hier_tree_dump_csv	Dump the design hierarchy tree in csv format.
hier_tree_trv_inst	Get a list of instance npiParam while traversing the design hierarchy tree.
hier_tree_trv_begin	Get a list of unnamed begin block npiParam while traversing the design hierarchy tree.
hier_tree_trv	Traverse the hierarchy tree from the specified scope (language model).
hier_tree_trv_register_cb	Register callback function for hier_tree_trv.
hier_tree_trv_reset_cb	Reset callback function for hier_tree_trv.
scope_explore	Explore scope to retrieve handles from the handle type specified (language model).
scope_explore_register_cb	Register callback function for npiParam_scope_explore.
scope_explore_reset_cb	Reset callback function for npiParam_scope_explore.

hier_tree_dump_txt

lang.hier_tree_dump_txt(root_scope, file)

Dump the hierarchical tree view of a scope.

The hierarchical tree view of a scope, specified by its name `root_scope`, is dumped into a text file. If `root_scope` is NULL, the hierarchy tree from the SV top module(s), VHDL root or SC top instance in the design will be dumped.

Parameters:

- **root_scope** (*string*) – Specify the name of the target scope.
- **file** (*file()*) – Specify the pointer to the output file.

Returns: Return 0 if the corresponding scope cannot be found; otherwise, return 1.

Return type: int

Examples

example.v:

```
module MOD;
    task tsk(output integer y, input integer x);
        endtask
    endmodule

module TOP;
    wire a, b;

    and and1[1:0] (a, b);

    generate
        genvar i;
        for(i=0; i<2; i=i+1) begin: gen
            MOD inst();
        end
    endgenerate

    MOD inst2 [1:0] ();
    MOD inst1();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```

```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# dump hier tree
fp = open("hier_tree.txt", 'w')
lang.hier_tree_dump_txt("TOP", fp)
fp.close()

# end NPI
npisys.end()
```

Output (file “hier_tree.txt”):

```
TOP (TOP)
  inst2[1] (MOD)
    tsk
  inst2[0] (MOD)
    tsk
  inst1 (MOD)
    tsk
  gen[0]
    inst (MOD)
      tsk
  gen[1]
    inst (MOD)
      tsk
```

hier_tree_dump_csv

lang.hier_tree_dump_csv(root_scope, file, is_skip_lib_cell=False)

Dump the hierarchical tree view of a scope.

The hierarchical tree from a scope, specified by its name `root_scope`, is dumped into a file in cspecify the pointer to the output file.

`is_skip_lib_cv` format. If `root_scope` is NULL, the hierarchy tree from the SV top module(s), VHDL root, and SC top instance in the design will be dumped.

Parameters:

- **root_scope** (*string*) – Specify the name of the target scope.
- **file** (*file()*) – Sell (*bool*) – Option to skip library cells. Default value is false.

Returns: Return 0 if the corresponding scope cannot be found; otherwise, return 1.

Return type: int

Examples

example.v:

```
module MOD;
  task tsk(output integer y, input integer x);
    endtask
endmodule

module TOP;
  wire a, b;

  and and1[1:0] (a, b);

  generate
    genvar i;
    for(i=0; i<2; i=i+1) begin: gen
      MOD inst();
    end
  endgenerate

  MOD inst2 [1:0] ();
  MOD inst1();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# dump hier tree
fp = open("hier_tree.csv", 'w')
lang.hier_tree_dump_csv("TOP", fp)
fp.close()

# end NPI
npisys.end()
```

Output (file “hier_tree.csv”):

```
TOP (TOP)
,inst2[1] (MOD)
,,tsk
,inst2[0] (MOD)
,,tsk
,inst1 (MOD)
,,tsk
,gen[0]
,,inst (MOD)
,,,tsk
,gen[1]
,,inst (MOD)
,,,tsk
```

hier_tree_trv_inst

lang.hier_tree_trv_inst(root_scope)

Collect all the handles of the instances under the hierarchy tree of the scope.

The handles of all instances including SV, VHDL and SC instances under the scope, specified by its name `root_scope`, are dumped into a list.

Parameters: `root_scope` (*string*) – Specify the name of the target scope.

Returns: A list of type handle. The instance handles will be put into the list.

Return type: list of handle

Examples

example.v:

```
module MOD;
  task tsk(output integer y, input integer x);
    begin
      integer tsk_var;
      for (int fvar = 0; fvar < 2; fvar++)
        $display("For scope");
      fork
        integer fork_var;
        $display("fork");
        fork: nf
          $display("named fork");
      join
    join
  end
endtask

function add_func(input int x, input int y);
```



```
        add_func = x + y;
    endfunction
endmodule

module TOP;
    wire a, b;

    and and1[1:0] (a, b);

    generate
        genvar i;
        for(i=0; i<2; i=i+1) begin: gen
            MOD inst();
        end
    endgenerate

    MOD inst2 [1:0] ();
    MOD inst1();

    initial begin: init_blk
        end

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("inst_info.log", 'w')
hdl_list = lang.hier_tree_trv_inst("TOP")
lang.dump_hdl_vec_info(hdl_list, fp)
fp.close()

# end NPI
npisys.end()
```

Output (file “inst_info.log”):

```
npiModule, TOP, {example.v : 22}  
npiModule, TOP.inst2[1], {example.v : 34}  
npiModule, TOP.inst2[0], {example.v : 34}  
npiModule, TOP.inst1, {example.v : 35}  
npiModule, TOP.gen[0].inst, {example.v : 30}  
npiModule, TOP.gen[1].inst, {example.v : 30}
```

hier_tree_trv_begin

lang.hier_tree_trv_begin(root_scope)

Collect all the handles of the unnamed begin blocks under the hierarchy tree of the scope.

The handles of all the unnamed begin blocks under the hierarchy tree of the scope, specified by its name `root_scope`, are dumped into a list.

Parameters: `root_scope` (*string*) – Specify the name of the target scope.

Returns: A list of type handle. The unnamed begin block handles will be put into the list.

Return type: list of handle

Examples

example.v:

```
module MOD;  
  task tsk(output integer y, input integer x);  
    begin  
      integer tsk_var;  
      for (int fvar = 0; fvar < 2; fvar++)  
        $display("For scope");  
      fork  
        integer fork_var;  
        $display("fork");  
        fork: nf  
          $display("named fork");  
      join  
    join  
  end  
endtask  
  
  function add_func(input int x, input int y);  
    add_func = x + y;  
  endfunction  
endmodule  
  
module TOP;  
  wire a, b;
```

```
and and1[1:0] (a, b);

generate
  genvar i;
  for(i=0; i<2; i=i+1) begin: gen
    MOD inst();
  end
endgenerate

MOD inst2 [1:0] ();
MOD inst1();

initial begin: init_blk
end

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("begin_info.log", 'w')
hdl_list = lang.hier_tree_trv_begin("TOP")
lang.dump_hdl_vec_info(hdl_list, fp)
fp.close()

# end NPI
npisys.end()
```

Output (file "begin_info.log"):

```
npiBegin, TOP.inst2[1].tsk.Unnamed_$example_v_3, {example.v : 3}
npiBegin, TOP.inst2[0].tsk.Unnamed_$example_v_3, {example.v : 3}
npiBegin, TOP.inst1.tsk.Unnamed_$example_v_3, {example.v : 3}
npiBegin, TOP.gen[0].inst.tsk.Unnamed_$example_v_3, {example.v : 3}
npiBegin, TOP.gen[1].inst.tsk.Unnamed_$example_v_3, {example.v : 3}
```

hier_tree_trv_register_cb

lang.hier_tree_trv_register_cb(obj_type, cb_func, cb_data)

Register callback function for hier_tree_trv.

The API hier_tree_trv_register_cb will create the association between user-defined callback function/data and hier_tree_trv. Object types that can be iterated from scope are allowed to register callback functions. The registered callback function will be called by hier_tree_trv when the corresponding object type is traversed.

Also refer to hier_tree_trv and hier_tree_trv_reset_cb.

Note: the object (handle) will be passed as an argument to the callback function and be released when the execution of callback function is done. If users want to preserve the object, set_permanent can be used to set the object as “permanent”.

SV object types recognized by hier_tree_trv_register_cb:

npAlways	npAssignStmt	npBitVar	npCase
npDelayControl	npEnumTypespec	npFor	npFuncCall
npIODecl	npIntTypespec	npIntegerVar	npLongIntVar
npNamedEvent	npNullStmt	npPackedArrayVar	npRealTypespec
npRepeat	npShortRealTypespec	npStructTypespec	npSysTaskCall
npTimeTypespec	npUnionTypespec	npLangInterface	npClassTypespec
npClockingBlock	npArrayNet	npAssignment	npBreak
npContAssign	npDisable	npEnumVar	npForce
npFunction	npIf	npIntVar	npInterface
npModport	npNamedEventArray	npPackage	npParamAssign
npRealVar	npReturnStmt	npShortRealVar	npStructVar
npTask	npTimeVar	npUnionVar	npProgram
npVirtualInterfaceVar	npClockingIODecl	npArrayTypespec	npBegin
npByteTypespec	npContinue	npDoWhile	npEventControl
npForever	npGate	npIfElse	npIntegerNet

npILogicTypespec	npiParamModule	npiParamNamedFork	npiParamPackedArrayNet
npiParamParameter	npiParamReg	npiParamShortIntTypespec	npiParamStringTypespec
npiParamSwitch	npiParamTaskCall	npiParamTypeParameter	npiParamWait
npiParamClassDefn	npiParamClassVar	npiParamNettypeTypespec	npiParamArrayVar
npiParamBitTypespec	npiParamByteVar	npiParamDeassign	npiParamEnumNet
npiParamEventStmt	npiParamFork	npiParamGenScope	npiParamInitial
npiParamIntegerTypespec	npiParamLongIntTypespec	npiParamNamedBegin	npiParamNet
npiParamPackedArrayTypespec	npiParamPort	npiParamRelease	npiParamShortIntVar
npiParamStructNet	npiParamSysFuncCall	npiParamTimeNet	npiParamUdp
npiParamWhile	npiParamMethodFuncCall	npiParamClassObj	npiParamForeachStmt

VHDL object types recognized by hier_tree_trv_register_cb:

npiVarHRoot	npiVarHGenerate	npiVarHSelectSigAssign	npiVarHSeqAssert
npiVarHSeqSigAssign	npiVarHReport	npiVarHExit	npiVarHNull
npiVarHArrayTypeDecl	npiVarHProcDecl	npiVarHVar	npiVarHVarParam
npiVarHComponent	npiVarHProcedure	npiVarHConcProcCall	npiVarHForLoop
npiVarHSeqProcCall	npiVarHIf npiVarHNext	npiVarHEnumTypeDecl	npiVarHPhysTypeDecl
npiVarHFuncDecl	npiVarHConst	npiVarHConstParam	npiVarHPackage
npiVarHFunction	npiVarHSimpleSigAssign	npiVarHConcAssert	npiVarHWhileLoop
npiVarHVarAssign	npiVarHCase	npiVarHIntTypeDecl	npiVarHFloatTypeDecl
npiVarHPort npiVarHGeneric	npiVarHAttrDecl	npiVarHBlock	npiVarHLangInterface
npiVarHCondSigAssign	npiVarHProcess	npiVarHForeverLoop	npiVarHWait
npiVarHReturn	npiVarHSubTypeDecl	npiVarHRecordTypeDecl	npiVarHSig
npiVarHSigParam	npiVarHCompDecl		

SC object types recognized by `hier_tree_trv_register_cb`:

`npiScInstance`

`npiScInstance`

Parameters:

- **obj_type** (int or enum) – Specify the object type that should be associated with the callback function.
- **cb_func** (pointer) – Specify the pointer to the callback function.
- **cb_data** – Specify the pointer to the callback data.

Returns: Returns 1 if the input object type is valid to be registered; otherwise, returns 0.

Return type: int

Examples

example.v:

```
module top;
  wire CLK, RESET, SEL, DIN, DOUT;
  flipflop i_ff
    (.clk(CLK), .reset(RESET), .sel(SEL), .din(DIN), .dout(DOUT));
endmodule

module flipflop (clk, reset, sel, din, dout);
  input clk, reset, sel;
  input din;
  output reg dout;

  always @(posedge clk or negedge reset) begin
    if (!reset)
      dout <= 1'b0;
    else
      dout <= sel ? din : 1'b1;
    end
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
```

```
hdl.set_permanent()
cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback funcation and data
lang.hier_tree_trv_register_cb('npiModule', cb_collect_hdl, hdl_list)
lang.hier_tree_trv_register_cb('npiPort', cb_collect_hdl, hdl_list)
# traverse the design from scope "top"
lang.hier_tree_trv("top")
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " modules or ports")

# reset the callback function
lang.hier_tree_trv_reset_cb()

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()
```

Output:

```
npiModule, top, {example.v : 1}
npiModule, top.i_ff, {example.v : 3}
npiPort, clk, {example.v : 6}
npiPort, reset, {example.v : 6}
npiPort, sel, {example.v : 6}
npiPort, din, {example.v : 6}
npiPort, dout, {example.v : 6}
Found 7 modules or ports
```

hier_tree_trv_reset_cb

lang.hier_tree_trv_reset_cb()

Reset callback function for hier_tree_trv.

The API `hier_tree_trv_reset_cb` will remove the association between user-registered callback function/data and `hier_hier_tree_trv`. When `hier_tree_trv_reset_cb` is executed, `npi_hier_hier_tree_trv` will not execute the callback function registered before.

Also refer to `hier_tree_trv` and `hier_tree_trv_register_cb`.

Parameters: None.

Returns: None.

Examples

example.v:

```
module top;
  wire CLK, RESET, SEL, DIN, DOUT;
  flipflop i_ff
  (.clk(CLK), .reset(RESET), .sel(SEL), .din(DIN), .dout(DOUT));
endmodule

module flipflop (clk, reset, sel, din, dout);
  input clk, reset, sel;
  input din;
  output reg dout;

  always @(posedge clk or negedge reset) begin
    if (!reset)
      dout <= 1'b0;
    else
      dout <= sel ? din : 1'b1;
    end
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
    hdl.set_permanent()
    cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0
```



```
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback funcation and data
lang.hier_tree_trv_register_cb('npiModule', cb_collect_hdl, hdl_list)
lang.hier_tree_trv_register_cb('npiPort', cb_collect_hdl, hdl_list)
# traverse the design from scope "top"
lang.hier_tree_trv("top")
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " modules or ports")

# reset the callback function and register new type again
hdl_list = []
lang.hier_tree_trv_reset_cb()
lang.hier_tree_trv_register_cb('npiPort', cb_collect_hdl, hdl_list)
# traverse the design from scope "top"
lang.hier_tree_trv("top")
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " ports")

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()
```

Output:

```
npiModule, top, {example.v : 1}
npiModule, top.i_ff, {example.v : 3}
npiPort, clk, {example.v : 6}
npiPort, reset, {example.v : 6}
npiPort, sel, {example.v : 6}
npiPort, din, {example.v : 6}
npiPort, dout, {example.v : 6}
Found 7 modules or ports
npiPort, clk, {example.v : 6}
npiPort, reset, {example.v : 6}
npiPort, sel, {example.v : 6}
npiPort, din, {example.v : 6}
npiPort, dout, {example.v : 6}
Found 5 ports
```

hier_tree_trv

lang.hier_tree_trv(scope_hier_name=None, depth=0)

Traverse the hierarchy tree from the specified scope.

The API `hier_tree_trv` will traverse the design hierarchy and execute the callback functions registered by the user.

Also refer to `hier_tree_trv_register_cb` and `hier_tree_trv_reset_cb`.

Parameters:

- **scope_hier_name** (*string*) – Specify the name of the target scope.
- **depth** (*int*) – Specify the traverse level. If not specified or specifying 0, traverse all levels.

Returns: Return 0 if the corresponding scope cannot be found; otherwise, return 1.

Return type: int

Examples

example.v:

```
module top;
  wire CLK, RESET, SEL, DIN, DOUT;
  flipflop i_ff
    (.clk(CLK), .reset(RESET), .sel(SEL), .din(DIN), .dout(DOUT));
endmodule

module flipflop (clk, reset, sel, din, dout);
  input clk, reset, sel;
  input din;
  output reg dout;

  always @(posedge clk or negedge reset) begin
    if (!reset)
      dout <= 1'b0;
    else
      dout <= sel ? din : 1'b1;
    end
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
```

Chapter 6: L1 APIs

Hierarchy Tree

```

from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
    hdl.set_permanent()
    cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback function and data
lang.hier_tree_trv_register_cb('npiModule', cb_collect_hdl, hdl_list)
lang.hier_tree_trv_register_cb('npiPort', cb_collect_hdl, hdl_list)
# traverse the design from scope "top"
lang.hier_tree_trv("top")
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+" modules or ports")

# reset the callback function
lang.hier_tree_trv_reset_cb()

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()

```

Output:

```

npiModule, top, {example.v : 1}
npiModule, top.i_ff, {example.v : 3}
npiPort, clk, {example.v : 6}
npiPort, reset, {example.v : 6}
npiPort, sel, {example.v : 6}
npiPort, din, {example.v : 6}
npiPort, dout, {example.v : 6}
Found 7 modules or ports

```

Module

The Module Library contains the dumping and gathering routines for the elements in a Module. The following table outlines the APIs in this category.

API	Description
mod_define_get_inst	Get the instance npiParam instantiated from the given module definition.

mod_define_get_inst

lang.mod_define_get_inst(mod_name)

Get the instance npiParam instantiated from the given module definition.

Get the instance handles instantiated from the given module definition, mod_name.

Parameters: **mod_name** (*string*) – Specify the definition name of the target instances.

Returns: A list of type npiParam. The iterated instance handles will be put into the list.

Return type: list of handle

Examples

example.v:

```
module MOD(a, b);
    input a;
    output b;
endmodule

module TOP;
    wire w1, w2;
    MOD inst(w1, w2);
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
```

```

        print("Error: Fail to initialize NPI.")
        assert 0

    if not npisys.load_design(sys.argv):
        print("Error: Fail to load design.")
        assert 0

    fp = open("mod_define_info.txt", 'w')
    hdl_list = lang.mod_define_get_inst("MOD")
    lang.dump_hdl_vec_info(hdl_list, fp)
    fp.close()

# end NPI
npisys.end()

```

Output (file “mod_define_info.txt”):

```

npiModule, TOP.inst, {example.v : 8}

```

Component

The component library contains the dumping and gathering routines for the elements in a root or component. The following table outlines the APIs in this category.

API	Description
arch_body_get_inst	Get the instance npiHandle instantiated by the given architecture body.
entity_get_inst	Get the instance npiHandle instantiated by the given entity.

arch_body_get_inst

lang.arch_body_get_inst(arch_body_name)

Get the instance npiHandle instantiated from the given module definition.

Get the instance handles instantiated by the given architecture body name, `arch_body_name`.

Parameters: `arch_body_name` (*string*) – Specify the architecture body name of the target instances.

Returns: A list of type npiHandle. The iterated instance handles will be put into the list.

Return type: list of handle

Examples

example.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity add32 is
  port(ea : in std_logic_vector(31 downto 0);
        eb : in std_logic_vector(31 downto 0);
        esum : out std_logic_vector(31 downto 0));
end add32;

architecture circuits of add32 is
begin
  esum <= ea xor eb after 1 ns;
end circuits;

library IEEE;
use IEEE.std_logic_1164.all;
use work.all;

entity top is
end top;

architecture top_body of top is
  signal a: std_logic_vector(31 downto 0) := x"00000000";
  signal b: std_logic_vector(31 downto 0) := x"FFFFFFFF";
  signal sum: std_logic_vector(31 downto 0);
  constant cv: integer := 31;

  component add32
    port(ea : in std_logic_vector(31 downto 0);
          eb : in std_logic_vector(31 downto 0);
          esum : out std_logic_vector(31 downto 0));
  end component;

begin
  adder0: add32 port map(ea(cv downto 16) => a(cv downto 16),
                        ea(15 downto 3) => a(15 downto 3),
                        ea(2 downto 2) => a(2 downto 2),
                        ea(1) => '1',
                        ea(0) => a(0),
                        eb => b,
                        esum => sum);
  adder1: add32 port map(a, b, sum);
end top_body;
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
```

```
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("inst_info.txt", 'w')
hdl_list = lang.arch_body_get_inst("circuits")
lang.dump_hdl_vec_info(hdl_list, fp)
fp.close()

# end NPI
npisys.end()
```

Output (file "inst_info.txt"):

```
npiVhComponent, top.adder0, {example.vhd : 35}
npiVhComponent, top.adder1, {example.vhd : 42}
```

entity_get_inst

lang.entity_get_inst(entity_name)

Get the instance npiHandle instantiated from the given entity declaration.

Get the instance handles instantiated from the given entity declaration, entity_name.

Parameters: **entity_name** (*string*) – Specify the entity name of the target instances.

Returns: A list of type npiHandle. The iterated instance handles will be put into the list.

Return type: list of handle

Examples

example.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity add32 is
    port (ea      : in  std_logic_vector(31 downto 0);
          eb      : in  std_logic_vector(31 downto 0);
          esum    : out std_logic_vector(31 downto 0));
```

```

end add32;

architecture circuits of add32 is
begin
    esum <= ea xor eb after 1 ns;
end circuits;

library IEEE;
use IEEE.std_logic_1164.all;
use work.all;

entity top is
end top;

architecture top_body of top is
    signal a:      std_logic_vector(31 downto 0) := x"00000000";
    signal b:      std_logic_vector(31 downto 0) := x"FFFFFFFF";
    signal sum:    std_logic_vector(31 downto 0);
    constant cv: integer := 31;

    component add32
        port(ea      : in  std_logic_vector(31 downto 0);
             eb      : in  std_logic_vector(31 downto 0);
             esum    : out std_logic_vector(31 downto 0));
    end component;

begin
    adder0:  add32 port map(ea(cv downto 16) => a(cv downto 16),
                           ea(15 downto 3)  => a(15 downto 3),
                           ea(2 downto 2)   => a(2 downto 2),
                           ea(1)            => '1',
                           ea(0)            => a(0),
                           eb               => b,
                           esum             => sum);
    adder1:  add32 port map(a, b, sum);
end top_body;

```

example.py:

```

#!/usr/bin/env python3
import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):

```



```
print("Error: Fail to load design.")
assert 0

fp = open("inst_info.txt", 'w')
hdl_list = lang.entity_get_inst("add32")
lang.dump_hdl_vec_info(hdl_list, fp)
fp.close()

# end NPI
npisys.end()
```

Output (file "inst_info.txt"):

```
npiVhComponent, top.adder0, {example.vhd : 35}
npiVhComponent, top.adder1, {example.vhd : 42}
```

Signal

The Signal Library contains the routines to collect or dump the handles that are related to a signal (variable or net). Typical applications include tracing the driver or load of a signal. The following table outlines the APIs in this category.

API	Description
trace_driver2	Trace the drivers of a signal using the NPI Language model.
trace_driver_by_hdl2	Trace the drivers of a signal using the NPI Language model.
trace_driver_dump2	Dump the driver statements of a signal using the NPI Language model.
trace_driver_by_hdl_dump2	Dump the driver statements of a signal using the NPI Language model.
trace_load2	Trace the loads of a signal using the NPI Language model.
trace_load_by_hdl2	Trace the loads of a signal using the NPI Language model.
trace_load_dump2	Dump the load statements of a signal using the NPI Language model.
trace_load_by_hdl_dump2	Dump the load statements of a signal using the NPI Language model.
get_signal_define_typespec_by_hdl	Get the definition typespec of a signal.
get_signal_define_typespec	Get the definition typespec of a signal.

API	Description
dump_signal_define_typespec	Dump the definition typespec of a signal.
dump_signal_define_typespec_by_hdl	Dump the definition typespec of a signal.
get_bit_blasted_signal	Get all the bit-blasted signals of a signal with multiple bits.

trace_driver2

lang.trace_driver2(sig_hier_name, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)

Find the drivers of a signal handle (simple expression except operation) based on the NPI Language Model.

The driver signal and statement handles obtained by the full name sig_hier_name and the NPI Language Model will be collected into the returned list.

Parameters:

- **sig_hier_name** (*string*) – Specify the hierarchical name of the target signal.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or npivhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the list of boundary handles. The signals are not traced if their scope has been specified the boundary scope (default value: None, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: Return the list of DrvLoadStmt objects.

Return type: list of [DrvLoadStmt](#)

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
```

```
initial q = 1'b1;
table
// ena_data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
    input wire [3:0] mi, m2;
    output wire [3:0] mo;
    not n0[7:10] (mo, mi);
endmodule
```

```
module top;
    wire [2:5] e, f, h, k;
    wire [3:0] d;
    wire [1:4] a, b, c;
    wire [3:0] i, j, l;
    wire [1:4] g = i;

    assign b = g[a] + c;
    mod m0(h,
           g[d]+ b + b,
           f);
    nand nd(b, d, e);
    latch la1[3:0](j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

drv_load_stmt_list = lang.trace_driver2("top.f[2:3]")
```

```

for dl_stmt in drv_load_stmt_list:
    src_hdl = dl_stmt.get_src_hdl()
    scope_hdl = dl_stmt.get_scope_hdl()
    is_pass_thr = dl_stmt.get_is_pass_thr()
    num_sig_use = dl_stmt.get_num_sig_use()
    use_hdl = dl_stmt.get_use_hdl()
    sig_hdl_list = dl_stmt.get_sig_hdl_list()
    print("-----")
    print("srcHdl: "+lang.get_hdl_info(src_hdl))
    print("scope: "+lang.get_hdl_info(scope_hdl))
    print("isPass: "+str(is_pass_thr))
    if num_sig_use == 0:
        print("  Driver in child stmt.")
        print("  <D> "+lang.get_hdl_info(use_hdl))
    elif num_sig_use == 1:
        print("  <D> "+lang.get_hdl_info(use_hdl))
    else:
        print("  <"+str(num_sig_use)+"D> "+lang.get_hdl_info(use_hdl))
    for hdl in sig_hdl_list:
        print(lang.get_hdl_info(hdl))

# end NPI
npisys.end()

```

Output:

```

-----
srcHdl: npiParamSelect, f[2:3], {example.v : 22}
scope:  npiParamModule, top.m0, {example.v : 29}
isPass: True
  <D> npiParamPort, mo, {example.v : 15}
npiParamSelect, mo[3:2], {example.v : 17}
-----
srcHdl: npiParamSelect, mo[3:2], {example.v : 17}
scope:  npiParamModule, top.m0, {example.v : 29}
isPass: False
  <D> npiParamPrimTerm, (null), {example.v : 18}
npiNetBit, top.m0.mi[3], {example.v : 16}
-----
srcHdl: npiParamSelect, mo[3:2], {example.v : 17}
scope:  npiParamModule, top.m0, {example.v : 29}
isPass: False
  <D> npiParamPrimTerm, (null), {example.v : 18}
npiNetBit, top.m0.mi[2], {example.v : 16}

```

trace_driver_by_hdl2

***lang.trace_driver_by_hdl2(sig_hdl, is_pass_thr=True,
bnd_hdl_list=None, trc_opt=None)***

Find the drivers of a signal handle (simple expression except operation) based on the NPI Language Model.

The driver signal and statement handles obtained by the input signal handle and the NPI Language Model will be collected into the returned list.

Parameters:

- **sig_hdl** (*handle*) – Specify the target source signal handle.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or npivhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the list of boundary handle. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: Return the list of DrvLoadStmt objects.

Return type: list of [DrvLoadStmt](#)

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
    input wire [3:0] mi, m2;
    output wire [3:0] mo;
    not n0[7:10] (mo, mi);
endmodule

module top;
    wire [2:5] e, f, h, k;
    wire [3:0] d;
    wire [1:4] a, b, c;
    wire [3:0] i, j, l;
    wire [1:4] g = i;

    assign b = g[a] + c;
    mod m0(h,
           g[d]+ b + b,
           f);
    nand nd(b, d, e);
    latch la1[3:0](j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

target_hdl = lang.handle_by_name("top.f[2:3]", None)
drv_load_stmt_list = lang.trace_driver_by_hdl2(target_hdl)

for dl_stmt in drv_load_stmt_list:
    src_hdl = dl_stmt.get_src_hdl()
    scope_hdl = dl_stmt.get_scope_hdl()
    is_pass_thr = dl_stmt.get_is_pass_thr()
    num_sig_use = dl_stmt.get_num_sig_use()
    use_hdl = dl_stmt.get_use_hdl()
    sig_hdl_list = dl_stmt.get_sig_hdl_list()
    print("-----")
    print("srcHdl: "+lang.get_hdl_info(src_hdl))
```

```

print("scope: " + lang.get_hdl_info(scope_hdl))
print("isPass: " + str(is_pass_thr))
if num_sig_use == 0:
    print("  Driver in child stmt.")
    print("    <D> " + lang.get_hdl_info(use_hdl))
elif num_sig_use == 1:
    print("    <D> " + lang.get_hdl_info(use_hdl))
else:
    print("    <" + str(num_sig_use) + "> " + lang.get_hdl_info(use_hdl))
for hdl in sig_hdl_list:
    print(lang.get_hdl_info(hdl))

# end NPI
npisys.end()

```

Output:

```

scope: npiParam, top.m0, {example.v : 29}
isPass: False
    <D> npiParam, (null), {example.v : 18}
npiParam, top.m0.mi[3], {example.v : 16}
-----
srcHdl: npiParamSelect, mo[3:2], {example.v : 17}
scope: npiParam, top.m0, {example.v : 29}
isPass: False
    <D> npiParam, (null), {example.v : 18}
npiParam, top.m0.mi[2], {example.v : 16}

```

trace_driver_dump2

lang.trace_driver_dump2(sig_hier_name, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)

Dump the drivers of a signal handle (simple expression except operation) based on the NPI Language Model.

The driver signal and statement handles obtained by the full name sig_hier_name and the NPI Language Model will be dumped into the specified file object.

Parameters:

- **sig_hier_name** (*string*) – Specify the hierarchical name of the target signal.
- **file** (*file()*) – Specify the file object of the output file.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).

- **bnd_hdl_list** (*list of handle*) – Specify the list of boundary handle. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: the number of the dumped used statement.

Return type: int

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
  input wire [3:0] mi, m2;
  output wire [3:0] mo;
  not n0[7:10] (mo, mi);
endmodule
```

```
module top;
  wire [2:5] e, f, h, k;
  wire [3:0] d;
  wire [1:4] a, b, c;
  wire [3:0] i, j, l;
  wire [1:4] g = i;

  assign b = g[a] + c;
  mod m0(h,
        g[d]+ b + b,
        f);
  nand nd(b, d, e);
  latch la1[3:0](j, l, i);

endmodule
```


example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

lang.trace_driver_dump2("top.f[2:3]", sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npiPartSelect, f[2:3], {example.v : 22} /* results of trace driver */
Need pass through
<1> source: f[2:3], scope: top.m0
    pass through.
    <D> npiPort, mo, {example.v : 15}
        npiPartSelect, mo[3:2], {example.v : 17}
<2> source: mo[3:2], scope: top.m0
    <D> npiPrimTerm, (null), {example.v : 18}
        npiNetBit, top.m0.mi[3], {example.v : 16}
<3> source: mo[3:2], scope: top.m0
    <D> npiPrimTerm, (null), {example.v : 18}
        npiNetBit, top.m0.mi[2], {example.v : 16}
```

trace_driver_by_hdl_dump2

lang.trace_driver_by_hdl_dump2(sig_hdl, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)

Dump the drivers of a signal handle (simple expression except operation) based on the NPI Language Model.

The driver signal and statement handles obtained by the input signal handle and the NPI Language Model will be dumped into the file object.

Parameters:

- **sig_hdl** (*handle*) – Specify the target source signal handle.
- **file** (*file()*) – Specify the file object of the output file.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the pointer of boundary handle vector. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: the number of the dumped used statement.

Return type: int

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
  input wire [3:0] mi, m2;
  output wire [3:0] mo;
  not n0[7:10] (mo, mi);
endmodule
```

```
module top;
  wire [2:5] e, f, h, k;
  wire [3:0] d;
  wire [1:4] a, b, c;
  wire [3:0] i, j, l;
```

```
wire [1:4] g = i;

assign b = g[a] + c;
mod m0(h,
      g[d]+ b + b,
      f);
nand nd(b, d, e);
latch la1[3:0](j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

target_hdl = lang.handle_by_name("top.f[2:3]", None)
lang.trace_driver_by_hdl_dump2(target_hdl, sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npiPartSelect, f[2:3], {example.v : 22} /* results of trace driver */
Need pass through
<1> source: f[2:3], scope: top.m0
    pass through.
    <D> npiPort, mo, {example.v : 15}
        npiPartSelect, mo[3:2], {example.v : 17}
<2> source: mo[3:2], scope: top.m0
    <D> npiPrimTerm, (null), {example.v : 18}
        npiNetBit, top.m0.mi[3], {example.v : 16}
<3> source: mo[3:2], scope: top.m0
    <D> npiPrimTerm, (null), {example.v : 18}
        npiNetBit, top.m0.mi[2], {example.v : 16}
```

trace_load2

***lang.trace_load2(sig_hier_name, is_pass_thr=True,
bnd_hdl_list=None, trc_opt=None)***

Find the loads of a signal handle (simple expression except operation) based on the NPI Language Model.

The load signal and statement handles obtained by the full name sigHierName and the NPI Language Model will be collected into the returned list.

Parameters:

- **sig_hier_name** (*string*) – Specify the hierarchical name of the target signal.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the pointer of boundary handle vector. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: Return the list of DrvLoadStmt objects.

Return type: list of [DrvLoadStmt](#)

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
    input wire [3:0] mi, m2;
    output wire [3:0] mo;
    not n0[7:10] (mo, mi);
endmodule

module top;
    wire [2:5] e, f, h, k;
    wire [3:0] d;
    wire [1:4] a, b, c;
    wire [3:0] i, j, l;
    wire [1:4] g = i;

    assign b = g[a] + c;
    mod m0(h,
           g[d]+ b + b,
           f);
    nand nd(b, d, e);
    latch la1[3:0](j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

drv_load_stmt_list = lang.trace_load2("top.b")

for dl_stmt in drv_load_stmt_list:
    src_hdl = dl_stmt.get_src_hdl()
    scope_hdl = dl_stmt.get_scope_hdl()
    is_pass_thr = dl_stmt.get_is_pass_thr()
    num_sig_use = dl_stmt.get_num_sig_use()
    use_hdl = dl_stmt.get_use_hdl()
    sig_hdl_list = dl_stmt.get_sig_hdl_list()
    print("-----")
    print("srcHdl: "+lang.get_hdl_info(src_hdl))
    print("scope:  "+lang.get_hdl_info(scope_hdl))
```

```

print("isPass: "+str(is_pass_thr))
if num_sig_use == 0:
    print("  Load in upper stmt.")
    print("  <L> "+lang.get_hdl_info(use_hdl))
elif num_sig_use == 1:
    print("  <L> "+lang.get_hdl_info(use_hdl))
else:
    print("  <" +str(num_sig_use)+"L> "+lang.get_hdl_info(use_hdl))
for hdl in sig_hdl_list:
    print(lang.get_hdl_info(hdl))

# end NPI
npisys.end()

```

Output:

```

-----
srcHdl: npINet, top.b, {example.v : 24}
scope:  npIModule, top.m0, {example.v : 29}
isPass: True
  <2L> npIPort, m2, {example.v : 15}
npINet, top.m0.m2, {example.v : 16}

```

trace_load_by_hdl2

***lang.trace_load_by_hdl2(sig_hdl, is_pass_thr=True,
bnd_hdl_list=None, trc_opt=None)***

Find the loads of a signal handle (simple expression except operation) based on the NPI Language Model.

The load signal and statement handles obtained by the input signal handle and the NPI Language Model will be collected into the returned list.

Parameters:

- **sig_hdl** (*handle*) – Specify the target source signal handle.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the pointer of boundary handle vector. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: Return the list of `DrvLoadStmt` objects.

Return type: list of [DrvLoadStmt](#)

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
  input wire [3:0] mi, m2;
  output wire [3:0] mo;
  not n0[7:10] (mo, mi);
endmodule
```

```
module top;
  wire [2:5] e, f, h, k;
  wire [3:0] d;
  wire [1:4] a, b, c;
  wire [3:0] i, j, l;
  wire [1:4] g = i;

  assign b = g[a] + c;
  mod m0(h,
    g[d]+ b + b,
    f);
  nand nd(b, d, e);
  latch la1[3:0] (j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys
```

```
# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

target_hdl = lang.handle_by_name("top.b", None)
drv_load_stmt_list = lang.trace_load_by_hdl2(target_hdl)

for dl_stmt in drv_load_stmt_list:
    src_hdl = dl_stmt.get_src_hdl()
    scope_hdl = dl_stmt.get_scope_hdl()
    is_pass_thr = dl_stmt.get_is_pass_thr()
    num_sig_use = dl_stmt.get_num_sig_use()
    use_hdl = dl_stmt.get_use_hdl()
    sig_hdl_list = dl_stmt.get_sig_hdl_list()
    print("-----")
    print("srcHdl: "+lang.get_hdl_info(src_hdl))
    print("scope:  "+lang.get_hdl_info(scope_hdl))
    print("isPass: "+str(is_pass_thr))
    if num_sig_use == 0:
        print("  Load in upper stmt.")
        print("  <L> "+lang.get_hdl_info(use_hdl))
    elif num_sig_use == 1:
        print("  <L> "+lang.get_hdl_info(use_hdl))
    else:
        print("  <"+str(num_sig_use)+"L> "+lang.get_hdl_info(use_hdl))
    for hdl in sig_hdl_list:
        print(lang.get_hdl_info(hdl))

# end NPI
npisys.end()
```

Output:

```
srcHdl: npINet, top.b, {example.v : 24}
scope:  npIModule, top.m0, {example.v : 29}
isPass: True
  <2L> npIPort, m2, {example.v : 15}
npINet, top.m0.m2, {example.v : 16}
```

trace_load_dump2

lang.trace_load_dump2(sig_hier_name, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)

Dump the loads of a signal handle (simple expression except operation) based on the NPI Language Model. The load signal and statement handles obtained by the full name sig_hier_name and the NPI Language Model will be dumped into the specified file object.

Parameters:

- **sig_hier_name** (*string*) – Specify the hierarchical name of the target signal.
- **file** (*file()*) – Specify the file object of the output file.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the pointer of boundary handle vector. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: the number of the dumped used statement.

Return type: int

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
  input wire [3:0] mi, m2;
  output wire [3:0] mo;
  not n0[7:10] (mo, mi);
endmodule

module top;
  wire [2:5] e, f, h, k;
  wire [3:0] d;
  wire [1:4] a, b, c;
  wire [3:0] i, j, l;
  wire [1:4] g = i;

  assign b = g[a] + c;
  mod m0(h,
        g[d]+ b + b,
        f);
  nand nd(b, d, e);
  latch la1[3:0](j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

lang.trace_load_dump2("top.b", sys.stdout)

# end NPI
```

Output:

```
npiNet, top.b, {example.v : 24} /* results of trace load */
Need pass through
<1> source: b, scope: top.m0
    pass through.
<2L> npiPort, m2, {example.v : 15}
    npiNet, top.m0.m2, {example.v : 16}
```

trace_load_by_hdl_dump2

lang.trace_load_by_hdl_dump2(sig_hdl, file, is_pass_thr=True, bnd_hdl_list=None, trc_opt=None)

Dump the loads of a signal handle (simple expression except operation) based on the NPI Language Model.

The load signal and statement handles obtained by the input signal handle and the NPI Language Model will be dumped into the specified file object.

Parameters:

- **sig_hdl** (*handle*) – Specify the target source signal handle.
- **file** (*file()*) – Specify the file object of the output file.
- **is_pass_thr** (*bool*) – A Boolean value specifying whether or not to pass through the port of module and map element of vh component (false: the boundary of Module or VhComponent object will not be passed through when traversing the model, true: the boundary of Module or VhComponent object will be passed through when traversing the model, default value: true).
- **bnd_hdl_list** (*list of handle*) – Specify the pointer of boundary handle vector. The signals are not traced if their scope has been specified the boundary scope (default value: NULL, no boundary scope handle).
- **trc_opt** (*TrcOption*) – Specify the option of trace rules. Default is = { True, False, True, True, False };

Returns: the number of the dumped used statement.

Return type: int

Examples

example.v:

```
// Sequential Primitive
primitive latch (q, ena_, data);
output q; reg q;
input ena_, data;
initial q = 1'b1;
table
// ena_ data : q : q+
0 1 : ? : 1 ;
0 0 : ? : 0 ;
1 ? : ? : - ; // - = no change
endtable
endprimitive
```

```
module mod(mi, m2, mo);
    input wire [3:0] mi, m2;
    output wire [3:0] mo;
    not n0[7:10] (mo, mi);
endmodule

module top;
    wire [2:5] e, f, h, k;
    wire [3:0] d;
    wire [1:4] a, b, c;
    wire [3:0] i, j, l;
    wire [1:4] g = i;

    assign b = g[a] + c;
    mod m0(h,
        g[d]+ b + b,
        f);
    nand nd(b, d, e);
    latch la1[3:0] (j, l, i);

endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

target_hdl = lang.handle_by_name("top.b", None)
lang.trace_load_by_hdl_dump2(target_hdl, sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npinet, top.b, {example.v : 24} /* results of trace load */
Need pass through
```

```
<1> source: b, scope: top.m0
    pass through.
<2L> npiParam, m2, {example.v : 15}
    npiParam, top.m0.m2, {example.v : 16}
```

get_signal_define_typespec_by_hdl

lang.get_signal_define_typespec_by_hdl(sig_hdl)

Get the typespec (typespec means the class of Typespec and the class of VhType) handle of a signal based on the NPI Language Model.

The typespec or type handle of a signal, which is specified by its full hierarchy name, is retrieved. If the specified signal type is npiParamSelect, npiParamIndexedPartSelect, npiParamVhSliceName, or npiParamVhIndexName, this function will return the typespec or type handle of its parent.

Parameters: **sig_hdl** (*handle*) – Specify the target signal handle.

Returns: Return the typespec or the type handle of the target signal. Return NULL if the given signal is unable to get its typespec or type.

Return type: handle

Examples

example.v:

```
module TOP;
    wire [4:0] w1 [2:0];
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0
```

```
sig_hdl = lang.handle_by_name("TOP.w1[2:1]", None)
hdl = lang.get_signal_define_typespec_by_hdl(sig_hdl)
lang.dump_hdl_info(hdl, sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npisArrayTypespec, logic unsigned [4:0] $SIG [2:0];, (null)
```

get_signal_define_typespec

lang.get_signal_define_typespec(sig_hier_name)

Get the typespec (typespec means the class of Typespec and the class of VhType) handle of a signal based on the NPI Language Model.

The typespec or type handle of a signal, which is specified by its full hierarchy name, is retrieved. If the specified signal type is `npiParamSelect`, `npiParamIndexedPartSelect`, `npiVarSliceName`, or `npiVarIndexName`, this function will return the typespec or type handle of its parent.

Parameters: `sig_hier_name` (*string*) – Specify the hierarchical name of the target signal.

Returns: Return the typespec or the type handle of the target signal. Return NULL if the given signal is unable to get its typespec or type.

Return type: handle

Examples

example.v:

```
module TOP;
  wire [4:0] w1 [2:0];
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
```

```

        assert 0

    if not npisys.load_design(sys.argv):
        print("Error: Fail to load design.")
        assert 0

    hdl = lang.get_signal_define_typespec("TOP.w1[2:1]")
    lang.dump_hdl_info(hdl, sys.stdout)

# end NPI
npisys.end()

```

Output:

```

npiArrayTypeSpec, logic unsigned [4:0] $SIG [2:0];, (null)

```

dump_signal_define_typespec

lang.dump_signal_define_typespec(sig_hier_name, file)

Dump the typespec (typespec means the class of Typespec and the class of VhType) handle of a signal based on the NPI Language Model.

The typespec or type handle of a signal, which is specified by its full hierarchy name, is dumped. If the specified signal type is `npiPartSelect`, `npiIndexedPartSelect`, `npiVhSliceName`, or `npiVhIndexName`, this function will return the typespec or type handle of its parent.

Parameters:

- **sig_hier_name** (*string*) – Specify the hierarchical name of the target signal.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return 1 if getting the typespec or type handle of the target signal was successful; otherwise, return 0.

Return type: int

Examples

example.v:

```

module TOP;
    wire [4:0] w1 [2:0];
endmodule

```

example.py:

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

```

```
import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

lang.dump_signal_define_typespec("TOP.w1[2:1]", sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npisArrayTypespec, logic unsigned [4:0] $SIG [2:0];, (null)
```

dump_signal_define_typespec_by_hdl

lang.dump_signal_define_typespec_by_hdl(sig_hdl, file)

Dump the typespec (typespec means the class of Typespec and the class of VhType) handle of a signal based on the NPI Language Model.

The typespec or type handle of a signal, which is specified by its full hierarchy name, is dumped. If the specified signal type is `npiParamSelect`, `npiParamIndexedPartSelect`, `npiParamVhSliceName`, or `npiParamVhIndexName`, this function will dump the typespec or type information of its parent.

Parameters:

- **sig_hdl** (*handle*) – Specify the target signal handle.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return 1 if getting the typespec or type handle of the target signal was successful; otherwise, return 0.

Return type: int

Examples

example.v:

```
module TOP;
  wire [4:0] w1 [2:0];
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl = lang.handle_by_name("TOP.w1[2:1]", None)
lang.dump_signal_define_typespec_by_hdl(hdl, sys.stdout)

# end NPI
npisys.end()
```

Output:

```
npiArrayTypespec, logic unsigned [4:0] $SIG [2:0];, (null)
```

get_bit_blasted_signal

lang.get_bit_blasted_signal(sig_hdl)

Get all the bit-blasted signals of specified signal handle (see the definition of `ut_is_signal`) based on the NPI Language Model. However, any range selected signal like `PartSelect` is not an invalid input signal.

Parameters: `sig_hdl` (*handle*) – Specify the target signal.

Returns: Specify the list of bit-blasted signal handles.

Return type: list of handle

Examples

example.v:

```
module top;
  logic [1:0][2:0] a [3:0];
  logic [1:1] b;

  typedef struct packed {
    logic [1:0] a;
    logic b;
    logic [1:0][2:0] c [3:0];
  } st_t;

  st_t svar;
endmodule
```

Following are some examples for different kinds of signal and their results.

- Input signal “top.a”: return value = 24, hdlVec will contain 24 bit-blasted handles: a[3][1][2], a[3][1][1], a[3][1][0], a[3][0][2], ..., a[0][0][0].
- Input signal “top.a[1][1]”: return value = 3, hdlVec will contain 3 bit-blasted handles: a[1][1][2], top.a[1][1][1] and a[1][1][0]
- Input signal “top.a[1][1:0]”: return value = -1, hdlVec will contain nothing.
- Input signal “top.b”: return value = 1, hdlVec will contain 1 bit-blasted handle itself: b[1]
- Input signal “top.svar”: return value = 27, hdlVec will contain 27 bit-blasted handles: svar.a[1], svar.a[0], svar.b, svar.c[3][2][1], svar.c[3][2][0], svar.c[3][1][1], ..., svar.c[0][0][0]
- Input signal “top.svar.c”: return value = 24, hdlVec will contain 24 bit-blasted handles: svar.c[3][2][1], svar.c[3][2][0], svar.c[3][1][1], ..., svar.c[0][0][0]
- Input signal “top.svar.c[2:1]”: return value = -1, hdlVec will contain nothing.

example.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity top is
end top;

architecture rt_body of top is
  type MY_BOOL_2D_ARR is array (NATURAL range <>, NATURAL range <>) of
    BOOLEAN;
  type MY_SUB_2D_ARR is array (NATURAL range <>) of MY_BOOL_2D_ARR;
  signal a: std_logic_vector(31 downto 0);
  signal b: std_logic_vector(0 to 0);
  signal c: MY_SUB_2D_ARR(2 downto 0)(0 to 1, 4 to 5);
begin
end;
```

Following are some examples for different kinds of signal and their results.

- Input signal “top.a”: return value = 32, hdlVec will contain 24 bit-blasted handles: a(31), a(30), a(29), ..., a(0)
- Input signal “top.a(0)”: return value = 1, hdlVec will contain 1 bit-blasted handle itself: a(0)
- Input signal “top.b”: return value = 1, hdlVec will contain 1 bit-blasted handle itself: b(0)
- Input signal “top.c”: return value = 12, hdlVec will contain 12 bit-blasted handles: c(2)(0, 4), c(2)(0, 5), c(2)(1, 4), c(2)(1, 5), ..., c(0)(1, 5)
- Input signal “top.c(0)”, vhRngIdx = 1 r return value = 4, hdlVec will contain 4 bit-blasted handles: c(0)(0, 4), c(0)(0, 5), c(0)(1, 4), c(0)(1, 5), ..., c(0)(1, 5)
- Input signal “top.c(0)(0, 4)”: return value = 1, hdlVec will contain 1 bit-blasted handle itself: c(0)(0, 4)

List

The List Library contains the dumping routines during the design traversal. Some specific design components will be listed (dumped). The following table outlines the APIs in this category.

API	Description
list_all_module_define	List all module definitions in the design.

list_all_module_define

lang.list_all_module_define(root_scope, file)

List the defined module names from the target scope.

Find the module definitions from a specified scope. Print the defined module names, file name and line number into a specified file or stdout.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module is used if it is NULL.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return the total number of modules defined. If none is found, return 0.

Return type: int

Examples

example.v:

```
module MOD;
endmodule

module TOP;
  MOD inst();
  MOD inst2 [3:0] ();
endmodule
```

example.py:

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("mod_define_info.txt", 'w')
lang.list_all_module_define("TOP", fp);
fp.close()

# end NPI
npisys.end()
```

Output (file “mod_define_info.txt”):

```
MOD, {example.v : 1}
TOP, {example.v : 4}
```

Find

The Find Library contains the routines that can find instances, instance definition, and signals (nets or variables) in the design. The following table outlines the APIs in this category.

API	Description
find_inst_wildcard	Find the instances that are matched with the specified wildcard.
find_inst_wildcard_dump	Find and dump the instances that are matched with the specified wildcard.
find_inst_regex	Find the instances that are matched with the specified regular expression.
find_inst_regex_dump	Find and dump the instances that are matched with the specified regular expression.
find_inst_with_def_wildcard	Find the instances whose defined names are matched with the specified wildcard.
find_inst_with_def_wildcard_dump	Find and dump the instances whose defined names are matched with the specified wildcard.
find_inst_with_def_regex	Find the instances whose defined names are matched with the specified regular expression.
find_inst_with_def_regex_dump	Find and dump the instances whose defined names are matched with the specified regular expression.
find_signal_wildcard	Find the signals (nets or variables) that are matched with the specified wildcard.
find_signal_wildcard_dump	Find and dump the signals (nets or variables) that are matched with the specified wildcard.
find_signal_regex	Find the signals (nets or variables) that are matched with the specified regular expression.
find_signal_regex_dump	Find and dump the signals (nets or variables) that are matched with the specified regular expression.

find_inst_wildcard

lang.find_inst_wildcard(root_scope, inst_wildcard_list)

Find the instances that match the specified wildcard naming rules.

Find the instances that match a specified wildcard naming rules. Collect them into a list. The “instance” means the SV, VHDL and SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_wildcard_list** (*list of string*) – Specify the instance wildcard naming rules into the string list. Any instances should be collected if their names (not full names) match any of the specified naming rules.

Returns: the list of the matched instance handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
  esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(15 downto 0);
       eb   : in  std_logic_vector(15 downto 0);
       esum : out std_logic_vector(15 downto 0));
end entity;
```

```
architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = ["t*", "*[0-9]"]
hdl_list = lang.find_inst_wildcard("top", pat_str_list)
for hdl in hdl_list:
    print("Found handle: "+hdl.full_name())

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output:

```
Found handle: top
Found handle: top.a0
Found handle: top.a1
```

find_inst_wildcard_dump

lang.find_inst_wildcard_dump(root_scope, inst_wildcard_list, file)

Dump the instances that match the specified wildcard naming rules.

Find the instances that match the specified wildcard naming rules. Dump them into a specified file or stdout. The “instance” means the SV, VHDL and SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_wildcard_list** (*list of string*) – Specify the instance wildcard naming rules. Any instances should be dumped if their names (not full names) match any of the specified naming rules.
- **file** – Specify the file object of the output file.

Returns: Return the total number of matched instances. If none is found, return 0.

Return type: int

Examples

svTop.v:

```
module top;
    parameter cv = 32;
    logic [31:0] a;
    logic [31:0] b;
    logic [31:0] c;
    vhAdd32 #(.width(cv))    a0(a, b, c);
    vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
    vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
    generic(width: integer := 0);
    port(ea    : in  std_logic_vector(31 downto 0);
         eb    : in  std_logic_vector(31 downto 0);
         esum  : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
```



```
generic(width: integer := 0);
port(ea    : in  std_logic_vector(15 downto 0);
     eb    : in  std_logic_vector(15 downto 0);
     esum  : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = ["t*", "[0-9]"]
fp = open("output.log", 'w')
lang.find_inst_wildcard_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output (File "output.log"):

```
npiModule, top, {svTop.v : 1}
npiVhComponent, top.a0, {svTop.v : 6}
npiVhComponent, top.a1, {svTop.v : 7}
```

find_inst_regex

lang.find_inst_regex(root_scope, inst_regex_list)

Find the instances that match the specified regex (regular expression) naming rules.

Find the instances that match the specified regular expression naming rules. Collect them into a specified vector (C++) or list (Tcl). The “instance” means the SV, VHDL and SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_regex_list** (*list of string*) – Specify the instance regular expression naming rules into the vector. Any instances should be collected if their names (not full names) match any of the specified naming rules.

Returns: the list of the matched instance handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
  esum <= ea xor eb after 1 ns;
```

```
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea    : in  std_logic_vector(15 downto 0);
          eb    : in  std_logic_vector(15 downto 0);
          esum  : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = [".*n", ".*[0-9]"]
hdl_list = lang.find_inst_regex("top", pat_str_list)
for hdl in hdl_list:
    print("Found handle: "+hdl.full_name())

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output:

```
Found handle: top.a0
Found handle: top.an
Found handle: top.a1
```

find_inst_regex_dump

lang.find_inst_regex_dump(root_scope, inst_regex_list, file)

Dump the instances that match the specified regex (regular expression) naming rules.

Find the instances that match the specified regular expression naming rules. Dump them into a specified file or stdout. The “instance” means the SV, VHDL and SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_regex_list** (*list of string*) – Specify the instance regular expression naming rules into the vector. Any instances should be collected if their names (not full names) match any of the specified naming rules.
- **file** (*file()*) – Specify the file object of the output file.

Returns: the list of the matched instance handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea    : in  std_logic_vector(31 downto 0);
```

```

        eb    : in  std_logic_vector(31 downto 0);
        esum  : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea    : in  std_logic_vector(15 downto 0);
         eb    : in  std_logic_vector(15 downto 0);
         esum  : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;

```

example.py

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = [".*n", ".*[0-9]"]
fp = open("output.log", 'w')
lang.find_inst_regex_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()

```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output (File “output.log”):

```
npiVhComponent, top.a0, {svTop.v : 6}
npiVhComponent, top.an, {svTop.v : 8}
npiVhComponent, top.a1, {svTop.v : 7}
```

find_inst_with_def_wildcard

***lang.find_inst_with_def_wildcard(root_scope,
inst_def_wildcard_list)***

Find the instances whose defined names match the specified wildcard naming rule.

Find the instances whose defined names match the specified wildcard naming rules. Collect them into a list. The “instance” means the SV instance and VHDL instance. In the SystemVerilog Model, the defined name means the DefName of SV instance. In the VHDL Model, the defined name means the VhEntityName and VhArchName (in a form like “entityName(archName)”) of VHDL instance. In the SystemC Model, the defined name means the DefName of SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The module will be used if it is NULL.
- **inst_def_wildcard_list** (*list of string*) – Specify the instance defined wildcard naming rules into the list. Any instances should be collected if their defined names match any of the specified naming rules.

Returns: the list that collects the matched instance handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.(width(cv))    a0(a, b, c);
```

```

    vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
    vhAdd32 #(.width(cv))   an(a, b, c);
endmodule

```

add.vhd:

```

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
    generic(width: integer := 0);
    port(ea   : in  std_logic_vector(31 downto 0);
         eb   : in  std_logic_vector(31 downto 0);
         esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea   : in  std_logic_vector(15 downto 0);
         eb   : in  std_logic_vector(15 downto 0);
         esum : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;

```

example.py

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")

```

```

    assert 0

    pat_str_list = ["*16*", "*32*"]
    hdl_list = lang.find_inst_with_def_wildcard("top", pat_str_list)
    for hdl in hdl_list:
        print("Found handle: "+hdl.full_name())

# end NPI
npisys.end()

```

Execution command in python:

```

os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')

```

Output:

```

Found handle: top.a0
Found handle: top.an
Found handle: top.a1

```

find_inst_with_def_wildcard_dump

lang.find_inst_with_def_wildcard_dump(root_scope, inst_def_wildcard_list, file)

Dump the instances whose defined names match the specified wildcard naming rules.

Find the instances whose defined names match the specified wildcard naming rules. Dump them into a specified file or stdout. The “instance” means the SV instance and VHDL instance. In the SystemVerilog Model, the defined name means the DefName of SV instance. In the VHDL Model, the defined name means the VhEntityName and VhArchName (in a form like “entityName(archName)”) of VHDL instance. In the SystemC Model, the defined name means the DefName of SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_def_wildcard_list** (*list of string*) – Specify the instance defined wildcard naming rules into the list. Any instances should be collected if their defined names match any of the specified naming rules.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return the total number of matched instances. If none is found, return 0.

Return type: int

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
  esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(15 downto 0);
       eb   : in  std_logic_vector(15 downto 0);
       esum : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
  esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
```

```
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = ["*16*", "*32*"]
fp = open("output.log", 'w')
lang.find_inst_with_def_wildcard_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output (File “output.log”):

```
npiVhComponent, top.a0, {svTop.v : 6}
npiVhComponent, top.an, {svTop.v : 8}
npiVhComponent, top.a1, {svTop.v : 7}
```

find_inst_with_def_regex

lang.find_inst_with_def_regex(root_scope, inst_def_regex_list)

Find the instances whose defined names match the specified regex (regular expression) naming rules.

Find the instances whose defined names match the specified regular expression naming rules. Collect them into a list. The “instance” means the SV instance and the VHDL instance. In the SystemVerilog Model, the defined name means the DefName of SV instance. In the VHDL Model, the defined name means the VhEntityName and VhArchName (in a form like “entityName(archName)”) of VHDL instance. In the SystemC Model, the defined name means the npiDefName of SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_def_regex_list** (*list of string*) – Specify the instance defined regular expression naming rules into the list. Any instances should be collected if their defined names match any of the specified naming rules.

Returns: the list that will collect the matched instance handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
  esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(15 downto 0);
       eb   : in  std_logic_vector(15 downto 0);
       esum : out std_logic_vector(15 downto 0));
end entity;
```

```
architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = ["t.*", ".s*"]
hdl_list = lang.find_inst_with_def_regex("top", pat_str_list)
for hdl in hdl_list:
    print("Found handle: "+hdl.full_name())

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output:

```
Found handle: top
Found handle: top.a0
Found handle: top.an
Found handle: top.a1
```

find_inst_with_def_regex_dump

lang.find_inst_with_def_regex_dump(root_scope, inst_def_regex_list, file)

Dump the instances whose defined names match the specified regex (regular expression) naming rules.

Find the instances whose defined names match the specified regular expression naming rules. Dump them into a specified file or stdout. The “instance” means the the SV instance and VHDL instance. In the SystemVerilog Model, the defined name means the DefName of SV instance. In the VHDL Model, the defined name means the VhEntityName and VhArchName (in a form like “entityName(archName)”) of VHDL instance. In the SystemC Model, the defined name means the npDefName of SC instance.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **inst_def_regex_list** (*list of string*) – Specify the instance defined regular expression naming rules into the list. Any instances should be collected if their defined names match any of the specified naming rules.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return the total number of matched instances. If none is found, return 0.

Return type: int

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
```

```
    port(ea      : in  std_logic_vector(31 downto 0);
          eb      : in  std_logic_vector(31 downto 0);
          esum    : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea      : in  std_logic_vector(15 downto 0);
          eb      : in  std_logic_vector(15 downto 0);
          esum    : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = ["t.*", ".*s"]
fp = open("output.log", 'w')
lang.find_inst_with_def_regex_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output (File “output.log”):

```
npiModule, top, {svTop.v : 1}
npiVhComponent, top.a0, {svTop.v : 6}
npiVhComponent, top.an, {svTop.v : 8}
npiVhComponent, top.a1, {svTop.v : 7}
```

find_signal_wildcard

lang.find_signal_wildcard(root_scope, sig_wildcard_list)

Find the signals that match the specified wildcard naming rule.

Find the signals that match the specified wildcard naming rules. Collect them into a list. The “signal” here contains “SV nets”, “SV variables”, “VHDL signals”, and “VHDL variables”.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **sig_wildcard_list** (*list of string*) – Specify the signal wildcard naming rules into the list. Any signals should be collected if their names (not full names) match any of the specified naming rules.

Returns: the list that will collect the matched signal handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv)) a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv)) an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
    generic(width: integer := 0);
    port(ea    : in  std_logic_vector(31 downto 0);
         eb    : in  std_logic_vector(31 downto 0);
         esum   : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
    signal s0: std_logic;
    signal s1: integer;
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea    : in  std_logic_vector(15 downto 0);
         eb    : in  std_logic_vector(15 downto 0);
         esum   : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
    signal s0: std_logic;
    signal s1: integer;
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
```



```

    assert 0

pat_str_list = ["*0", "[1-9]"]
hdl_list = lang.find_signal_wildcard("top", pat_str_list)
for hdl in hdl_list:
    print("Found signal: "+hdl.full_name()+" (" +hdl.file()+" :
        "+str(hdl.line_no())+")")

# end NPI
npisys.end()

```

Execution command in python:

```

os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')

```

Output:

```

Found signal: top.a0.s0 (add.vhd : 12)
Found signal: top.a0.s1 (add.vhd : 13)
Found signal: top.an.s0 (add.vhd : 12)
Found signal: top.an.s1 (add.vhd : 13)
Found signal: top.a1.s0 (add.vhd : 29)
Found signal: top.a1.s1 (add.vhd : 30)

```

find_signal_wildcard_dump

lang.find_signal_wildcard_dump(root_scope, sig_wildcard_list, file)

Dump the signals that match the specified wildcard naming rule.

Find the signals that match the specified wildcard naming rule. Dump them into a specified file or stdout. The “signal” here contains “SV nets”, “SV variables”, “VHDL signals”, and “VHDL variables”.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **sig_wildcard_list** (*list of string*) – Specify the signal wildcard naming rules into the list. Any signals should be collected if their names (not full names) match any of the specified naming rules.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return the total number of matched signals. If none is found, return 0.

Return type: int

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
  signal s0: std_logic;
  signal s1: integer;
begin
  esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(15 downto 0);
       eb   : in  std_logic_vector(15 downto 0);
       esum : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
  signal s0: std_logic;
  signal s1: integer;
begin
  esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("output.log", 'w')
pat_str_list = ["*0", "*[1-9]"]
hdl_list = lang.find_signal_wildcard_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output (File “output.log”):

```
npiVhSig, top.a0.s0, {add.vhd : 12}
npiVhSig, top.a0.s1, {add.vhd : 13}
npiVhSig, top.an.s0, {add.vhd : 12}
npiVhSig, top.an.s1, {add.vhd : 13}
npiVhSig, top.a1.s0, {add.vhd : 29}
npiVhSig, top.a1.s1, {add.vhd : 30}
```

find_signal_regex

lang.find_signal_regex(root_scope, sig_regex_list)

Find the signals that match the specified regex (regular expression) naming rules.

Find the signals that match the specified regular expression naming rules. Collect them into a list. The “signal” here contains “SV nets”, “SV variables”, “VHDL signals”, and “VHDL variables”.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **sig_regex_list** (*list of string*) – Specify the signal regular expression naming rules into the list. Any signals should be collected if their names (not full names) match any of the specified naming rules.

Returns: the list that will collect the matched signal handles.

Return type: list of handle

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
  signal s0: std_logic;
  signal s1: integer;
begin
  esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(15 downto 0);
       eb   : in  std_logic_vector(15 downto 0);
```

```

        esum : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
    signal s0: std_logic;
    signal s1: integer;
begin
    esum <= ea nor eb after 1 ns;
end architecture;

```

example.py

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

pat_str_list = [".*0", ".*[1-9]"]
hdl_list = lang.find_signal_regex("top", pat_str_list)
for hdl in hdl_list:
    print("Found signal: "+hdl.full_name()+" (" +hdl.file()+" : "+str(hdl.line_no())+")")

# end NPI
npisys.end()

```

Execution command in python:

```

os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')

```

Output:

```

npiVhSig, top.a0.s0, {add.vhd : 12}
npiVhSig, top.a0.s1, {add.vhd : 13}
npiVhSig, top.an.s0, {add.vhd : 12}
npiVhSig, top.an.s1, {add.vhd : 13}
npiVhSig, top.a1.s0, {add.vhd : 29}
npiVhSig, top.a1.s1, {add.vhd : 30}

```

find_signal_regex_dump

lang.find_signal_regex_dump(root_scope, sig_regex_list, file)

Dump the signals that match the specified regex (regular expression) naming rule

Find the signals that match the specified regular expression naming rule. Dump them into a specified file or stdout. The “signal” here contains “SV nets”, “SV variables”, “VHDL signals”, and “VHDL variables”.

Parameters:

- **root_scope** (*string*) – Specify the hierarchical name of a root scope. The function will search the handle from the specified scope. The top module will be used if it is NULL.
- **sig_regex_list** (*list of string*) – Specify the signal regular expression naming rule into the list. Any signals should be collected if their names (not full names) match any of the specified naming rules.
- **file** (*file()*) – Specify the file object of the output file.

Returns: Return the total number of matched signals. If none is found, return 0.

Return type: int

Examples

svTop.v:

```
module top;
  parameter cv = 32;
  logic [31:0] a;
  logic [31:0] b;
  logic [31:0] c;
  vhAdd32 #(.width(cv))    a0(a, b, c);
  vhAdd16 #(.width(cv/2)) a1(a[31:16], b[31:16], null);
  vhAdd32 #(.width(cv))    an(a, b, c);
endmodule
```

add.vhd:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd32 is
  generic(width: integer := 0);
  port(ea   : in  std_logic_vector(31 downto 0);
       eb   : in  std_logic_vector(31 downto 0);
       esum : out std_logic_vector(31 downto 0));
end entity;

architecture circuits of vhAdd32 is
```

```
    signal s0: std_logic;
    signal s1: integer;
begin
    esum <= ea xor eb after 1 ns;
end architecture;

library IEEE;
use IEEE.std_logic_1164.all;

entity vhAdd16 is
    generic(width: integer := 0);
    port(ea      : in  std_logic_vector(15 downto 0);
          eb      : in  std_logic_vector(15 downto 0);
          esum    : out std_logic_vector(15 downto 0));
end entity;

architecture circuits of vhAdd16 is
    signal s0: std_logic;
    signal s1: integer;
begin
    esum <= ea nor eb after 1 ns;
end architecture;
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

fp = open("output.log", 'w')
pat_str_list = [".*0", ".*[1-9]"]
hdl_list = lang.find_signal_regex_dump("top", pat_str_list, fp)
fp.close()

# end NPI
npisys.end()
```

Execution command in python:

```
os.system('rm -rf *.lib++ *Log')
os.system('vhdlcom -vhdl08 add.vhd -lib svTop')
os.system('vericom -sv svTop.v -lib svTop')
os.system('./example.py -lib svTop')
```

Output:

```
npiVhSig, top.a0.s0, {add.vhd : 12}
npiVhSig, top.a0.s1, {add.vhd : 13}
npiVhSig, top.an.s0, {add.vhd : 12}
npiVhSig, top.an.s1, {add.vhd : 13}
npiVhSig, top.a1.s0, {add.vhd : 29}
npiVhSig, top.a1.s1, {add.vhd : 30}
```

Utilities

The Utility Library contains the basic utility routines, such as get handle information, dump handle information, and query utilities. The following table outlines the APIs in this category

API	Description
dump_hdl_info	Dump the information of the npiHandle object.
get_hdl_info	Get the information of the npiHandle object.
dump_hdl_vec_info	Dump the handle vector information.
expr_decompile	Decompile the handle object and return a string with a functionally equivalent to the original source code.
expr_trv	Traverse the expression tree from the given expression handle.
expr_trv_register_cb	Register callback function for expr_trv.
expr_trv_reset_cb	Reset callback function for expr_trv.

dump_hdl_info

lang.dump_hdl_info(hdl, file, is_compcose_full_name=False, const_size=False)

Dump the information of the npiHandle object.

The API `dump_hdl_info` will call `get_hdl_info` internally and then dump the information into a file.

Parameters:

- **hdl** (*handle*) – The handle to an object.
- **file** (*file()*) – Specify the file object of the output file.
- **is_compose_full_name** (*bool*) – If this variable is set true, the name information will always show the input handle's hierarchy even though the input handle doesn't have the property, `FullName`. Otherwise, the name information will show the input handle's hierarchy only when the input handle has the property, `npifullname`. The default value is false.
- **const_size** (*int*) – If this variable is true, the value of size will append to the front of constant. The default value is false.

Returns: None.

Examples

example.v:

```
module top;
    wire w1, w2, w3;
    assign w1 = w2 & w3;
endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

inst_hdl = lang.handle_by_name("top", None)
net_hdl_list = inst_hdl.net_handles()

fp = open("output.log", 'w')
# dump the handle information using L1 API
```

```
lang.dump_hdl_info(inst_hdl, fp)
for hdl in net_hdl_list:
    lang.dump_hdl_info(hdl, fp)
fp.close()

# end NPI
npisys.end()
```

Output (file “output.log”):

```
npiParam, top, {example.v : 1}
npiParam, top.w1, {example.v : 2}
npiParam, top.w2, {example.v : 2}
npiParam, top.w3, {example.v : 2}
```

get_hdl_info

lang.get_hdl_info(hdl, is_compose_full_name=False, const_size=False)

Get the information of the npiParam object.

The API `get_hdl_info` will return a string containing information about 1) the object type; 2) full hierarchical name, name, or decompile string; 3) file, and line number.

Parameters:

- **hdl** (*handle*) – The handle to an object.
- **is_compose_full_name** (*bool*) – If this variable is set true, the name information will always show the input handle’s hierarchy even though the input handle doesn’t have the property, `FullName`. Otherwise, the name information will show the input handle’s hierarchy only when the input handle has the property, `npiParamFullName`. The default value is false.
- **const_size** (*int*) – If this variable is true, the value of size will append to the front of constant. The default value is false.

Returns: A string containing the information of the input handle.

Return type: string

Examples

example.v:

```
module top;
    wire w1, w2, w3;
    assign w1 = w2 & w3;
endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

inst_hdl = lang.handle_by_name("top", None)
net_hdl_list = inst_hdl.net_handles()

# get and print handle information using L1 API
print(lang.get_hdl_info(inst_hdl))
for hdl in net_hdl_list:
    print(lang.get_hdl_info(hdl))

# end NPI
npisys.end()
```

Output (file “output.log”):

```
npiModule, top, {example.v : 1}
npiNet, top.w1, {example.v : 2}
npiNet, top.w2, {example.v : 2}
npiNet, top.w3, {example.v : 2}
```

dump_hdl_vec_info

***lang.dump_hdl_vec_info(hdl_list, file, is_release=False,
is_compose_full_name=False, const_size=False)***

Dump the information of the handle list.

The API `dump_hdl_vec_info` will call `dump_hdl_info` internally to dump every element in the handle vector.

Parameters:

- **hdl_list** (*list of handle*) – Specify the list of type handle.
- **is_release** (*bool*) – Specify a Boolean value specifying whether or not the handles in the vector should be released and the vector should be cleared after the dump process (true: release, false: no release, default value: false).
- **is_compose_full_name** (*bool*) – If this variable is set true, the name information will always show the input handle's hierarchy even though the input handle doesn't have the property, FullName. Otherwise, the name information will show the input handle's hierarchy only when the input handle has the property, npifullName. The default value is false.
- **const_size** (*int*) – If this variable is true, the value of size will append to the front of constant. The default value is false.

Returns: None.

Examples

example.v:

```
module top;
  wire w1, w2, w3;
  assign w1 = w2 & w3;
endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

inst_hdl = lang.handle_by_name("top", None)
net_hdl_list = inst_hdl.net_handles()

fp = open("output.log", 'w')
# dump the handle information using L1 API
```

```
lang.dump_hdl_info(inst_hdl, fp)
# dump and release the handles in the hdl vector
lang.dump_hdl_vec_info(net_hdl_list, fp, True)
fp.close()

# end NPI
npisys.end()
```

Output (file “output.log”):

```
npiParam, top, {example.v : 1}
npiParam, top.w1, {example.v : 2}
npiParam, top.w2, {example.v : 2}
npiParam, top.w3, {example.v : 2}
```

expr_decompile

lang.expr_decompile(expr_hdl, const_size=False, res_const_opr=False)

Decompile the target expression. The expression includes SV expression and VHDL expression.

The API `expr_decompile` will decompile the `npiParam` object and return a string with a functionally equivalent to the original source code.

Parameters:

- **expr_hdl** (*handle*) – Specify the handle of the target expression.
- **const_size** (*bool*) – If this variable is true, the value of size will append to the front of constant. The default value is false.
- **res_const_opr** (*bool*) – Option to resolve value of generate variables, parameters, and operations that are composed by constants or the foregoing two types. The default value is false.

Returns: A string representing the decompiled result of the given expression handle. If the input handle is not an expression, an empty string (“”) will be returned.

Return type: string

Examples

example.v:

```
module top;

parameter P1 = 31;
parameter P2 = 4'b1111;
integer a = P1 - P2 + 3.2 / 2 + P1;
```

```
endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# Usage of constSize
param_hdl = lang.handle_by_name("top.P2", None)
param_rhs_hdl = param_hdl.expr_handle()
print("RHS of P2: "+lang.expr_decompile(param_rhs_hdl))
print("RHS of P2 with constant size: "+lang.expr_decompile(param_rhs_hdl,
    True))

# Usage of resConstOpr
var_hdl = lang.handle_by_name("top.a", None)
var_rhs_hdl = var_hdl.expr_handle()
print("RHS of a: "+lang.expr_decompile(var_rhs_hdl))
print("RHS of a and resolve operation: "+lang.expr_decompile(var_rhs_hdl,
    False, True))

# end NPI
npisys.end()
```

Output:

```
RHS of P2: 'b1111'
RHS of P2 with constant size: 4'b1111
RHS of a: (((P1 - P2) + (3.200000 / 2)) + P1)
RHS of a and resolve operation: 48.600000
```

expr_trv_register_cb

lang.expr_trv_register_cb(cb_func, cb_data)

Register callback function for expr_trv.

The API `expr_trv_register_cb` will create association between use-defined callback function/data and `npi_expr_trv`. The registered callback function will be called by `npi_expr_trv` when each sub-expression handle is traversed. Also refer to `expr_trv` and `expr_trv_reset_cb`.

Note:

- The object (handle) will be passed as an argument to the callback function and be released when the execution of callback function is done. If users want to preserve the object, `set_permanent_handle` can be used to set the object as "permanent".
- The `expr_trv` will stop to traverse if this callback function returns 0.

Parameters:

- **cb_func** – Specify the pointer to the callback function.
- **cb_data** – Specify the pointer to the callback data.

Returns: Returns 1 if the callback function is valid to be registered; otherwise, returns 0.

Return type: int

Examples

example.v:

```
module top;

parameter P1 = 31;
parameter P2 = 4'b1111;
integer a = P1 - P2 + 3.2 / 2 + P1;

endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# check otype of operation and append "add operation" to list
def expr_cb(hdl, cb_data):
    if hdl.type() != "npiOperation":
        return 0
    if hdl.op_type() == "npiAddOp":
        hdl_list.append(hdl)
        hdl.set_permanent()
```

```

        return 1

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# traverse expression
var_hdl = lang.handle_by_name("top.a", None)
expr_hdl = var_hdl.expr_handle()
hdl_list = []
lang.expr_trv_register_cb(expr_cb, hdl_list)

lang.expr_trv(expr_hdl)
for hdl in hdl_list:
    print(lang.get_hdl_info(hdl))
print("Add op cnt = "+str(len(hdl_list)))

# reset and traverse expression again
hdl_list = []
lang.expr_trv_reset_cb()
lang.expr_trv(expr_hdl)
print("Add op cnt = "+str(len(hdl_list)))

# end NPI
npisys.end()

```

Output:

```

npiOperation, (((P1 - P2) + (3.200000 / 2)) + P1), {example.v : 5}
npiOperation, ((P1 - P2) + (3.200000 / 2)), {example.v : 5}
Add op cnt = 2
Add op cnt = 0

```

expr_trv_reset_cb

lang.expr_trv_reset_cb()

Reset callback function for expr_trv.

The API expr_reset_cb will remove the association between user-registered callback function/data and expr_trv. When expr_trv_reset_cb is executed, expr_trv will not execute the callback function registered before. Also refer to expr_trv and expr_trv_register_cb.

Parameters: None.

Returns: None.

Examples

example.v:

```
module top;

parameter P1 = 31;
parameter P2 = 4'b1111;
integer a = P1 - P2 + 3.2 / 2 + P1;

endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# check otype of operation and append "add operation" to list
def expr_cb(hdl, cb_data):
    if hdl.type() != "npiOperation":
        return 0
    if hdl.op_type() == "npiAddOp":
        hdl_list.append(hdl)
        hdl.set_permanent()
    return 1

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# traverse expression
var_hdl = lang.handle_by_name("top.a", None)
expr_hdl = var_hdl.expr_handle()
hdl_list = []
lang.expr_trv_register_cb(expr_cb, hdl_list)

lang.expr_trv(expr_hdl)
for hdl in hdl_list:
    print(lang.get_hdl_info(hdl))
print("Add op cnt = "+str(len(hdl_list)))

# reset and traverse expression again
hdl_list = []
```

```
lang.expr_trv_reset_cb()
lang.expr_trv(expr_hdl)
print("Add op cnt = "+str(len(hdl_list)))

# end NPI
npisys.end()
```

Output:

```
npiOperation, (((P1 - P2) + (3.200000 / 2)) + P1), {example.v : 5}
npiOperation, ((P1 - P2) + (3.200000 / 2)), {example.v : 5}
Add op cnt = 2
Add op cnt = 0
```

expr_trv

lang.expr_trv(expr_hdl)

Traverse the expression tree from the given expression `npiHandle`. The expression includes SV expression and VHDL expression.

The API `expr_trv` will traverse the expression tree and execute the callback functions registered by the user. Also refer to `expr_trv_register_cb` and `expr_trv_reset_cb`.

Parameters: `expr_hdl` (*handle*) – Specify the handle of the target expression.

Returns: Returns 0 if the input handle is not an expression; otherwise, returns 1.

Return type: int

Examples

`example.v`:

```
module top;

parameter P1 = 31;
parameter P2 = 4'b1111;
integer a = P1 - P2 + 3.2 / 2 + P1;

endmodule
```

`example.py`

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys
```

```
# check otype of operation and append "add operation" to list
def expr_cb(hdl, cb_data):
    if hdl.type() != "npiOperation":
        return 0
    if hdl.op_type() == "npiAddOp":
        hdl_list.append(hdl)
        hdl.set_permanent()
    return 1

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

# traverse expression
var_hdl = lang.handle_by_name("top.a", None)
expr_hdl = var_hdl.expr_handle()
hdl_list = []
lang.expr_trv_register_cb(expr_cb, hdl_list)

lang.expr_trv(expr_hdl)
for hdl in hdl_list:
    print(lang.get_hdl_info(hdl))
print("Add op cnt = "+str(len(hdl_list)))

# reset and traverse expression again
hdl_list = []
lang.expr_trv_reset_cb()
lang.expr_trv(expr_hdl)
print("Add op cnt = "+str(len(hdl_list)))

# end NPI
npisys.end()
```

Output:

```
npiOperation, (((P1 - P2) + (3.200000 / 2)) + P1), {example.v : 5}
npiOperation, ((P1 - P2) + (3.200000 / 2)), {example.v : 5}
Add op cnt = 2
Add op cnt = 0
```

scope_explore_register_cb

lang.scope_explore_register_cb(obj_type, cb_func, cb_data)

Register callback function for scope_explore.

The API `scope_explore_register_cb` will create the association between user-defined callback function/data and `scope_explore`. Object types that can be iterated from `scope` are allowed to register callback functions. The registered callback function will be called by `scope_explore` when the corresponding object type is traversed.

Also refer to `scope_explore` and `scope_explore_reset_cb`.

Note:

The object (`npiParam`) will be passed as an argument to the callback function and be released when the execution of callback function is done. If users want to preserve the object, `set_permanent_handle` can be used to set the object as “permanent”.

SV object types recognized by `npiParam_scope_explore_register_cb`:

<code>npiParamAlways</code>	<code>npiParamArrayNet</code>	<code>npiParamArrayTypespec</code>	<code>npiParamArrayVar</code>
<code>npiParamAssignStmt</code>	<code>npiParamAssignment</code>	<code>npiParamBegin</code>	<code>npiParamBitTypespec</code>
<code>npiParamBitVar</code>	<code>npiParamBreak</code>	<code>npiParamByteTypespec</code>	<code>npiParamByteVar</code>
<code>npiParamCase</code>	<code>npiParamContAssign</code>	<code>npiParamContinue</code>	<code>npiParamDeassign</code>
<code>npiParamDelayControl</code>	<code>npiParamDisable</code>	<code>npiParamDoWhile</code>	<code>npiParamEnumNet</code>
<code>npiParamEnumTypespec</code>	<code>npiParamEnumVar</code>	<code>npiParamEventControl</code>	<code>npiParamEventStmt</code>
<code>npiParamFor</code>	<code>npiParamForce</code>	<code>npiParamForever</code>	<code>npiParamFork</code>
<code>npiParamFuncCall</code>	<code>npiParamFunction</code>	<code>npiParamGate</code>	<code>npiParamGenScope</code>
<code>npiParamIODecl</code>	<code>npiParamIf</code>	<code>npiParamIfElse</code>	<code>npiParamInitial</code>
<code>npiParamIntTypespec</code>	<code>npiParamIntVar</code>	<code>npiParamIntegerNet</code>	<code>npiParamIntegerTypespec</code>
<code>npiParamIntegerVar</code>	<code>npiParamInterface</code>	<code>npiParamLogicTypespec</code>	<code>npiParamLongIntTypespec</code>
<code>npiParamLongIntVar</code>	<code>npiParamModport</code>	<code>npiParamModule</code>	<code>npiParamNamedBegin</code>
<code>npiParamNamedEvent</code>	<code>npiParamNamedEventArray</code>	<code>npiParamNamedFork</code>	<code>npiParamNet</code>
<code>npiParamNullStmt</code>	<code>npiParamPackage</code>	<code>npiParamPackedArrayNet</code>	<code>npiParamPackedArrayTypespec</code>
<code>npiParamPackedArrayVar</code>	<code>npiParamParamAssign</code>	<code>npiParamParameter</code>	<code>npiParamPort</code>
<code>npiParamRealTypespec</code>	<code>npiParamRealVar</code>	<code>npiParamReg</code>	<code>npiParamRelease</code>
<code>npiParamRepeat</code>	<code>npiParamReturnStmt</code>	<code>npiParamShortIntTypespec</code>	<code>npiParamShortIntVar</code>

npShortRealTypespec	npShortRealVar	npStringTypespec	npStructNet
npStructTypespec	npStructVar	npSwitch	npSysFuncCall
npSysTaskCall	npTask	npTaskCall	npTimeNet
npTimeTypespec	npTimeVar	npTypeParameter	npUdp
npUnionTypespec	npUnionVar	npWait	npWhile
npLangInterface	npProgram	npClassDefn	npMethodFuncCall
npClassTypespec	npVirtualInterfaceVar	npClassVar	npClassObj
npClockingBlock	npClockingIODecl	npNettypeTypespec	npForeachStmt
npStringVar	npRefObj	npExtRef	npImportItem
npExportItem	npAssert	npAssume	npCover
npRestrict	npImmediateAssert	npImmediateAssume	npImmediateCover
npPropertyDecl	npSequenceDecl	npCovergroup	npCoverageOption
npCoverPoint	npCoverCross	npBins	

VHDL object types recognized by `np_scope_explore_register_cb`:

npVhRoot	npVhComponent	npVhPackage	npVhBlock
npVhGenerate	npVhProcedure	npVhFunction	npLangInterface
npVhSelectSigAssign	npVhConcProcCall	npVhSimpleSigAssign	npVhCondSigAssign
npVhSeqAssert	npVhForLoop	npVhConcAssert	npVhProcess
npVhSeqSigAssign	npVhSeqProcCall	npVhWhileLoop	npVhForeverLoop
npVhReport	npVhIf	npVhVarAssign	npVhWait
npVhExit	npVhNext	npVhCase	npVhReturn
npVhNull	npVhEnumTypeDecl	npVhIntTypeDecl	npVhSubTypeDecl
npVhArrayTypeDecl	npVhPhysTypeDecl	npVhFloatTypeDecl	npVhRecordTypeDecl
npVhProcDecl	npVhFuncDecl	npPort	npVhSig

npiVhVar	npiVhConst	npiVhGeneric	npiVhSigParam
npiVhVarParam	npiVhConstParam	npiVhAttrDecl	npiVhCompDecl

SC object types recognized by `npi_scope_explore_register_cb`:

`npiScInstance`

`npiScMethodProcess`

Parameters:

- **obj_type** (*int* or *enum*) – Specify the object type that should be associated with the callback function.
- **cb_func** (*pointer*) – Specify the pointer to the callback function.
- **cb_data** – Specify the pointer to the callback data.

Returns: Returns 1 if the input object type is valid to be registered; otherwise, returns 0.

Examples

example.v:

```

struct {
    int a;
    int b;
} struct_t1;

union packed {
    bit [7:0] a;
    bit [1:0][3:0] b;
} union_t2;

typedef enum { a, b, c } enum_t3;

interface i;
    logic data, res;
    modport master(input data, output res);
    modport slave(input data, output res);
    task automatic tsk(output dup,
                       input data,
                       ref byte packet [1:0]);
        dup = data + packet[0][0];
    endtask
endinterface

package p;
    typedef enum { FALSE, TRUE } bool_t;
    integer pi;

```

```

parameter pp = 0;
class pkgCls;
endclass
endpackage

module top;
  import p::*; // import item
  i_intf();
  wire t1 = intf.data; // external reference
  wire t2 = struct_t1.a; // external reference
  wire t3 = union_t2.a; // external reference
  enum_t3 t4 = a; // external reference
  integer t5 = pi; // external reference
  logic d1, d2;
  bit signed [7:0] pk [1:0];
  always@(intf.data) // external reference
    intf.tsk(d1, d2, pk); // external reference
endmodule

module dut;
  top top_inst();
  initial force top_inst.t1 = 0; // external reference
endmodule

```

example.py:

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
    hdl.set_permanent()
    cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiExtRef', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()

```

```
# reset the callback function
lang.scope_explore_reset_cb()
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " external reference(s)")

# reset callback type and explore scope again
hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiImportItem', cb_collect_hdl, hdl_list)
lang.scope_explore_register_cb('npiExportItem', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()
# reset the callback function
lang.scope_explore_reset_cb()
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " import/export item(s)")

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()
```

Output:

```
npiExtRef, intf.data, {example.v : 35}
npiExtRef, struct_t1.a, {example.v : 36}
npiExtRef, union_t2.a, {example.v : 37}
npiExtRef, intf.data, {example.v : 42}
npiExtRef, a, {example.v : 38}
npiExtRef, pi, {example.v : 39}
npiExtRef, top_inst.t1, {example.v : 48}
Found 7 external reference(s)
npiImportItem, p::*, {example.v : 33}
Found 1 import/export item(s)
```

scope_explore_reset_cb

lang.scope_explore_reset_cb()

Reset callback function for npi_scope_explore.

The API `scope_explore_reset_cb` will remove the association between user-registered callback function/data and `scope_explore`. When `scope_explore_reset_cb` is executed, `scope_explore` will not execute the callback function registered before.

Also refer to `scope_explore` and `scope_explore_register_cb`.

Parameters: None.

Returns: None.

Examples

example.v:

```
struct {
    int a;
    int b;
} struct_t1;

union packed {
    bit [7:0] a;
    bit [1:0][3:0] b;
} union_t2;

typedef enum { a, b, c } enum_t3;

interface i;
    logic data, res;
    modport master(input data, output res);
    modport slave(input data, output res);
    task automatic tsk(output dup,
                        input data,
                        ref byte packet [1:0]);
        dup = data + packet[0][0];
    endtask
endinterface

package p;
    typedef enum { FALSE, TRUE } bool_t;
    integer pi;
    parameter pp = 0;
    class pkgCls;
    endclass
endpackage

module top;
    import p::*; // import item
    i intf();
    wire t1 = intf.data; // external reference
    wire t2 = struct_t1.a; // external reference
    wire t3 = union_t2.a; // external reference
    enum_t3 t4 = a; // external reference
    integer t5 = pi; // external reference
    logic d1, d2;
    bit signed [7:0] pk [1:0];
    always@(intf.data) // external reference
        intf.tsk(d1, d2, pk); // external reference
```

```
endmodule

module dut;
    top top_inst();
    initial force top_inst.t1 = 0; // external reference
endmodule
```

example.py

```
#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
    hdl.set_permanent()
    cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiExtRef', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()
# reset the callback function
lang.scope_explore_reset_cb()
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " external reference(s)")

# reset callback type and explore scope again
hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiImportItem', cb_collect_hdl, hdl_list)
lang.scope_explore_register_cb('npiExportItem', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()
# reset the callback function
lang.scope_explore_reset_cb()
```

```
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " import/export item(s)")

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()
```

Output:

```
npiExtRef, intf.data, {example.v : 35}
npiExtRef, struct_t1.a, {example.v : 36}
npiExtRef, union_t2.a, {example.v : 37}
npiExtRef, intf.data, {example.v : 42}
npiExtRef, a, {example.v : 38}
npiExtRef, pi, {example.v : 39}
npiExtRef, top_inst.t1, {example.v : 48}
Found 7 external reference(s)
npiImportItem, p::*, {example.v : 33}
Found 1 import/export item(s)
```

scope_explore

lang.scope_explore(scope_hier_name=None, depth=0)

Explore scope and report handle(s) in specified type.

The API `scope_explore` will explore the specified scope and execute the callback functions registered by the user. Also refer to `scope_explore_register_cb` and `scope_explore_reset_cb`.

Parameters:

- **scope_hier_name** (*string*) – Specify the full name of the target scope. If not specified, traverse all scopes.
- **depth** (*int*) – Specify the traverse level. If not specified or specifying 0, traverse all levels.

Returns: Return 0 if the corresponding scope cannot be found; otherwise, return 1.

Examples

example.v:

```
struct {
    int a;
```

```

    int b;
} struct_t1;

union packed {
    bit [7:0] a;
    bit [1:0][3:0] b;
} union_t2;

typedef enum { a, b, c } enum_t3;

interface i;
    logic data, res;
    modport master(input data, output res);
    modport slave(input data, output res);
    task automatic tsk(output dup,
                        input data,
                        ref byte packet [1:0]);
        dup = data + packet[0][0];
    endtask
endinterface

package p;
    typedef enum { FALSE, TRUE } bool_t;
    integer pi;
    parameter pp = 0;
    class pkgCls;
    endclass
endpackage

module top;
    import p::*; // import item
    i intf();
    wire t1 = intf.data; // external reference
    wire t2 = struct_t1.a; // external reference
    wire t3 = union_t2.a; // external reference
    enum_t3 t4 = a; // external reference
    integer t5 = pi; // external reference
    logic d1, d2;
    bit signed [7:0] pk [1:0];
    always@(intf.data) // external reference
        intf.tsk(d1, d2, pk); // external reference
endmodule

module dut;
    top top_inst();
    initial force top_inst.t1 = 0; // external reference
endmodule

```

example.py

```

#!/global/freeware/Linux/2.X/python-3.6.0/bin/python

import sys, os

```

```
pynpi_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(pynpi_lib_path))
from pynpi import lang
from pynpi import npisys

# callback function
def cb_collect_hdl(hdl, cb):
    hdl.set_permanent()
    cb.append(hdl)

# initialize NPI and load design example.v
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI.")
    assert 0

if not npisys.load_design(sys.argv):
    print("Error: Fail to load design.")
    assert 0

hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiExtRef', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()
# reset the callback function
lang.scope_explore_reset_cb()
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " external reference(s)")

# reset callback type and explore scope again
hdl_list = []
# register callback function and data
lang.scope_explore_register_cb('npiImportItem', cb_collect_hdl, hdl_list)
lang.scope_explore_register_cb('npiExportItem', cb_collect_hdl, hdl_list)
# traverse all scopes
lang.scope_explore()
# reset the callback function
lang.scope_explore_reset_cb()
# dump the result
for hdl in hdl_list:
    hdl_info = lang.get_hdl_info(hdl)
    print(hdl_info)
print("Found "+str(len(hdl_list))+ " import/export item(s)")

# release all handles (including permanent handles)
lang.release_all_handles()

# end NPI
npisys.end()
```

Output:

```
npiExtRef, intf.data, {example.v : 35}  
npiExtRef, struct_t1.a, {example.v : 36}  
npiExtRef, union_t2.a, {example.v : 37}  
npiExtRef, intf.data, {example.v : 42}  
npiExtRef, a, {example.v : 38}  
npiExtRef, pi, {example.v : 39}  
npiExtRef, top_inst.t1, {example.v : 48}  
Found 7 external reference(s)  
npiImportItem, p::*, {example.v : 33}  
Found 1 import/export item(s)
```

A

Appendix

This appendix chapter contains the following topics:

- [Data Structure](#)
 - [TrcOption](#)
 - [DrvLoadStmt](#)

Data Structure

NPI Language Model defined data structure are as follows:

API	Description
TrcOption	Class for option of trace rules.
DrvLoadStmt	Class for the collected driver statement and the signal handles.

TrcOption

class lang.TrcOption(edge_check=True, trc_val_idx=False, report_data=True, report_control=True, trc_stmt_prop=False)

Class for option of trace rules.

There are five members:

- edgeCheck (bool): Only report the edge trigger signals of event control.
- trcVarIdx (bool): Treat the signal of variable index as driver.
- reportData (bool): Report the data signals.
- reportControl (bool): Report the control signals.
- trcStmtProp (bool): Trace through statement.

set_edge_check(*bool_val*)

Assign a boolean value to “edge_check”.

Parameters: **bool_val** (*bool*) – the boolean value to be written to edge_check.

Returns: None.

get_edge_check()

Get value of “edge_check”.

Parameters: None.

Returns: boolean value of “edge_check”.

Return type: bool

set_trc_var_idx(*bool_val*)

Assign a boolean value to “trc_var_idx”.

Parameters: **bool_val** (*bool*) – the boolean value to be written to trc_var_idx.

Returns: None.

get_trc_var_idx()

Get value of “trc_var_idx”.

Parameters: None.

Returns: boolean value of “trc_var_idx”.

Return type: bool

set_report_data(*bool_val*)

Assign a boolean value to “report_data”.

Parameters: **bool_val** (*bool*) – the boolean value to be written to report_data.

Returns: None.

get_report_data()

Get value of “report_data”.

Parameters: None.

Returns: boolean value of “report_data”.

Return type: bool

set_report_control(*bool_val*)

Assign a boolean value to “report_control”.

Parameters: **bool_val** (*bool*) – the boolean value to be written to report_control.

Returns: None.

get_report_control()

Get value of “report_control”.

Parameters: None.

Returns: boolean value of “report_control”.

Return type: bool

set_trc_stmt_prop(*bool_val*)

Assign a boolean value to “trc_stmt_prop”.

Parameters: **bool_val** (*bool*) – the boolean value to trace through statement.

Returns: None.

get_trc_stmt_prop()

Get value of “trc_stmt_prop”.

Parameters: None.

Returns: boolean value of “trc_stmt_prop”.

Return type: bool

set_trc_option(*edge_check*, *trc_val_idx*, *report_data*, *report_control*, *trc_stmt_prop*)

Assign boolean value to TrcOption members.

Parameters:

- **edge_check** (bool) – the boolean value to be written to edge_check.
- **trc_val_idx** (bool) – the boolean value to be written to trc_val_idx.
- **report_data** (bool) – the boolean value to be written to report_data.
- **report_control** (bool) – the boolean value to be written to report_control.
- **trc_stmt_prop** (bool) – the boolean value to trace through statement.

Returns: None.

DrvLoadStmt

class lang.DrvLoadStmt(src_hdl, scope_hdl, is_pass_thr, num_sig_use, use_hdl, sig_hdl_list)

Class for the collected driver statement and signal handles.

The class “DrvLoadStmt” contains six members,

- src_hdl (handle): The tracing source handle of this use handle.
- scope_hdl (handle): The scope handle of this use handle.
- is_pass_thr (bool): Indicate this use handle is whether or not passed through.
- num_sig_use (int): Indicate how many times of source handle be traced in this use handle. If this value is 0, it means this useHdl is traced by other statement (statement structure based trace).
- use_hdl (handle): Target use handle. The definition of use handle, refer to Use and Vh Use.
- sig_hdl_list (list of handle): The driver signal handles of this use handle.

get_src_hdl()

Get handle object of “src_hdl”.

Parameters: None.

Returns: handle object of “src_hdl”.

Return type: handle

get_scope_hdl()

Get handle object of “scope_hdl”.

Parameters: None.

Returns: handle object of “scope_hdl”.

Return type: handle

get_is_pass_thr()

Get value of “is_pass_thr”.

Parameters: None.

Returns: boolean value of “is_pass_thr”.

Return type: bool

get_num_sig_use()

Get value of “num_sig_use”.

Parameters: None.

Returns: value of “num_sig_use”.

Return type: int

get_use_hdl()

Get handle object of “use_hdl”.

Parameters: None.

Returns: handle object of “use_hdl”.

Return type: handle

get_sig_hdl_list()

Get handle list “sig_hdl_list”.

Parameters: None.

Returns: handle list “sig_hdl_list”.

Return type: list of handle

set_src_hdl(*src_hdl*)

Internal Use: set src_hdl.

set_scope_hdl(*scope_hdl*)

Internal Use: set scope_hdl.

set_is_pass_thr(*is_pass_thr*)

Internal Use: set is_pass_thr.

set_num_sig_use(*num_sig_use*)

Internal Use: set num_sig_use.

set_use_hdl(*use_hdl*)

Internal Use: set use_hdl.

set_sig_hdl_list(*sig_hdl_list*)

Internal Use: set sig_hdl_list.